

ML

August 15, 2022

```
[ ]: !pip install import-ipynb
      !pip install eli5
      !pip install shap
      !pip install lime
      !pip install yellowbrick
```

```
[ ]: !pip install catboost
```

```
[ ]: # importing the tools to call google drive
      from google.colab import drive

      import pandas as pd
      import numpy as np
      import matplotlib.pyplot as plt
      import seaborn as sns
      import plotly.express as px
      from scipy.stats import uniform

      from sklearn.model_selection import GridSearchCV, RandomizedSearchCV, ↵
          ↵train_test_split
      from sklearn.pipeline import Pipeline
      from sklearn.compose import ColumnTransformer
      from sklearn.preprocessing import StandardScaler, OneHotEncoder
      from sklearn.pipeline import FeatureUnion

      from sklearn.model_selection import cross_val_score
      from sklearn.metrics import precision_score, log_loss, roc_curve, roc_auc_score, ↵
          ↵precision_recall_curve
      from sklearn.linear_model import LogisticRegression
      import xgboost
      from xgboost import XGBClassifier
      from lightgbm import LGBMClassifier
      from sklearn.ensemble import VotingClassifier
      from catboost import CatBoostClassifier
```

```
import pickle as pkl
```

```
import eli5
```

```
import shap
```

```
import lime
```

```
import yellowbrick
```

```
import import_ipynb
```

```
[ ]: # mount our directory
mount='/content/gdrive'
drive.mount(mount, force_remount=True)
```

Mounted at /content/gdrive

```
[ ]: # Switch to the directory on the VM for our purposes
drive_root=mount + '/My Drive/Colab Notebooks/Thesis'
%cd $drive_root
%pwd
```

/content/gdrive/My Drive/Colab Notebooks/Thesis

```
[ ]: '/content/gdrive/My Drive/Colab Notebooks/Thesis'
```

```
[ ]: import Pitch_Plot
import Half_Pitch
import data
from data import obtain_seasons,fetch_url,Game
from Pitch_Plot import pitchplot
from Half_Pitch import halfpitchplot
```

importing Jupyter notebook from Pitch_Plot.ipynb

importing Jupyter notebook from Half_Pitch.ipynb

importing Jupyter notebook from data.ipynb

```
[ ]: df_laliga = pd.read_pickle('./laliga_data.pkl')
```

```
[ ]: df_laliga['shot distance'] = round(np.sqrt((df_laliga['x location_
↪shot']-120)**2+(df_laliga['y location shot']-40)**2),1)
```

```
[ ]: df_laliga['gk distance'] = round(np.sqrt((df_laliga['x gk_
↪location']-120)**2+(df_laliga['y gk location']-40)**2),1)
```

```
[ ]: Goal_label = []
binary_outcome = []
for outcome in df_laliga['outcome'].values:
    if outcome == 'Goal':
        Goal_label.append('Goal')
        binary_outcome.append(1)
    else:
```

```

        Goal_label.append('Not Goal')
        binary_outcome.append(0)
df_laliga['Goal_label'] = Goal_label
df_laliga['binary_outcome'] = binary_outcome

```

```
[ ]: df_laliga = df_laliga.set_index("shot id")
```

```
[ ]: df_laliga['type of shot'].unique()
```

```
[ ]: array(['Open Play', 'Penalty', 'Free Kick', 'Corner'], dtype=object)
```

```
[ ]: df_laliga['body part']
```

```
[ ]: shot id
859f0f2d-6b33-49a3-9397-64d2ab927a73    Right Foot
7a1b1dcb-4e09-4bfa-9827-9d81b6944bc5    Left Foot
5078ef4c-43ea-4e5e-90cc-8c288349b3e3    Left Foot
3b87f877-45b9-425c-8d0c-035975f7f0cf    Right Foot
06c3d431-8ac3-43fa-9585-d9689f7a44f3    Right Foot
...
4468faac-ae6a-4637-928b-fc7dd448411b    Left Foot
5f613006-c00e-4581-882c-a7a81978ebf0    Left Foot
717c9351-6eff-4f71-969a-e25bab5bfcb6    Right Foot
0c914938-af14-479e-8043-8e4bfdfd474d    Right Foot
d9533117-d840-4973-91cd-9472a76d705e    Head
Name: body part, Length: 12841, dtype: object
```

```
[ ]: body_part_list=[]
for bodypart in df_laliga['body part'].values:
    if (bodypart=='Right Foot') or (bodypart=='Left Foot'):
        body_part_list.append('Foot')
    elif (bodypart=='Head'):
        body_part_list.append('Head')
    else:
        body_part_list.append('Other')
```

```
[ ]: df_laliga['body part']=body_part_list
df_laliga['body part'].value_counts()
```

```
[ ]: Foot      11197
Head       1611
Other        33
Name: body part, dtype: int64
```

```
[ ]: df_laliga=df_laliga[df_laliga['body part'] != 'Other']
df_laliga['body part'].value_counts()
```

```
[ ]: Foot      11197
      Head      1611
      Name: body part, dtype: int64
```

```
[ ]: df_laliga['play pattern']
```

```
[ ]: shot id
      859f0f2d-6b33-49a3-9397-64d2ab927a73      From Counter
      7a1b1dcb-4e09-4bfa-9827-9d81b6944bc5      From Throw In
      5078ef4c-43ea-4e5e-90cc-8c288349b3e3      Regular Play
      3b87f877-45b9-425c-8d0c-035975f7f0cf      From Throw In
      06c3d431-8ac3-43fa-9585-d9689f7a44f3      From Throw In
      ...
      4468faac-ae6a-4637-928b-fc7dd448411b      From Free Kick
      5f613006-c00e-4581-882c-a7a81978ebf0      From Throw In
      717c9351-6eff-4f71-969a-e25bab5bfcfb6      From Throw In
      0c914938-af14-479e-8043-8e4bfdfd474d      From Throw In
      d9533117-d840-4973-91cd-9472a76d705e      From Free Kick
      Name: play pattern, Length: 12808, dtype: object
```

```
[ ]: key_pass_pattern_list = []
      for pattern in df_laliga['play pattern'].values:
          if pattern=='None':
              key_pass_pattern_list.append('None')
          else:
              if (pattern=='Regular Play'):
                  key_pass_pattern_list.append('Regular Play')
              else:
                  key_pass_pattern_list.append('Non Regular Play')
```

```
[ ]: df_laliga['play pattern']=key_pass_pattern_list
      df_laliga['play pattern'].value_counts()
```

```
[ ]: Non Regular Play      8292
      Regular Play          4516
      Name: play pattern, dtype: int64
```

```
[ ]: df_laliga['type of shot'].value_counts()
```

```
[ ]: Open Play      11670
      Free Kick       973
      Penalty        164
      Corner          1
      Name: type of shot, dtype: int64
```

```
[ ]: df_laliga = df_laliga[df_laliga['type of shot']=='Open Play']
      df_laliga['type of shot'].value_counts()
```

```
[ ]: Open Play      11670
      Name: type of shot, dtype: int64
```

```
[ ]: df_laliga.isna().sum()
```

```
[ ]: play pattern      0
      x location shot   0
      y location shot   0
      duration          0
      outcome           0
      technique used    0
      first time        0
      x gk location     0
      y gk location     0
      body part         0
      type of shot      0
      Number of opponents in 5 yards  0
      Players between goal  0
      player name       0
      Team Name         0
      official xg       0
      Pass id           0
      Pass Type         0
      game_id           0
      season_id         0
      distance_from_goalpost_a  0
      distance_from_goalpost_b  0
      shot_angle        0
      shot distance     0
      gk distance       0
      Goal_label        0
      binary_outcome    0
      dtype: int64
```

1 Creating training, testing and validation data

```
[ ]: features = df_laliga.drop(["binary_outcome"], axis=1)
      targets = df_laliga["binary_outcome"]

      X_train, X_test, y_train, y_test = train_test_split(features, targets,
      ↪test_size=0.2)
      X_train, X_valid, y_train, y_valid = train_test_split(features.loc[X_train.
      ↪index], targets[X_train.index], test_size=0.2)

      train = pd.merge(X_train, y_train, left_index=True, right_index=True)
      validate = pd.merge(X_valid, y_valid, left_index=True, right_index=True)
```

```
test = pd.merge(X_test, y_test, left_index=True, right_index=True)
```

```
[ ]: train.columns
```

```
[ ]: Index(['play pattern', 'x location shot', 'y location shot', 'duration',
          'outcome', 'technique used', 'first time', 'x gk location',
          'y gk location', 'body part', 'type of shot',
          'Number of opponents in 5 yards', 'Players between goal', 'player name',
          'Team Name', 'official xg', 'Pass id', 'Pass Type', 'game_id',
          'season_id', 'distance_from_goalpost_a', 'distance_from_goalpost_b',
          'shot_angle', 'shot distance', 'gk distance', 'Goal_label',
          'binary_outcome'],
          dtype='object')
```

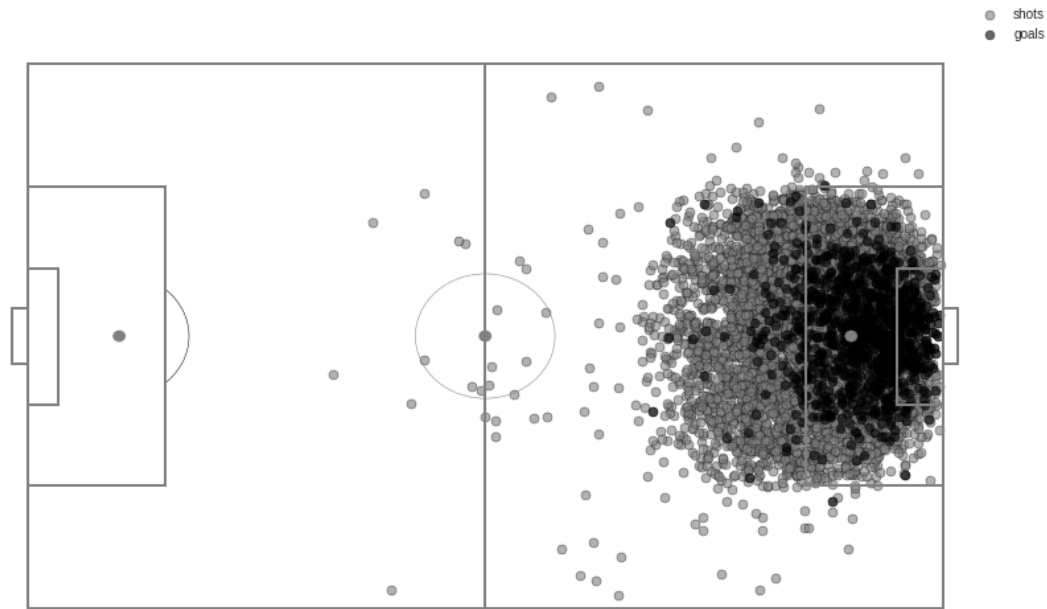
```
[ ]: with open('laliga.pickle', 'rb') as handle:
      laliga = pickle.load(handle)
```

```
[ ]: df_regular_play = train[train['type of shot']=='Open Play']
      df_regular_play
```

Let's check how distance has an affect on expected goals followed by angle.

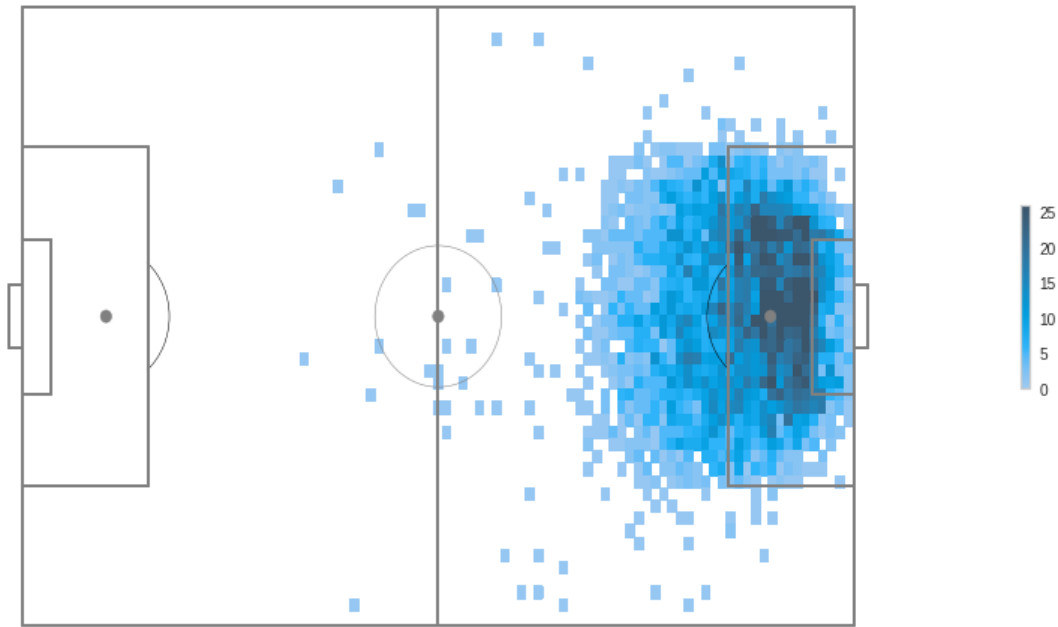
```
[ ]: def goals(fig, locations, color, label):
      x = []
      y = []
      for location in locations:
          x.append(location[0])
          y.append(location[1])
      plt.scatter(x, y, c=color, alpha=0.6, label=label, edgecolor='black', linewidth=0.
      ↪5)
      plt.axis('off')
      return fig
```

```
[ ]: pitch = pitchplot()
      pitch = goals(plt, train[['x location shot', 'y location shot']].
      ↪values, 'grey', 'shots')
      pitch = goals(plt, train[train['Goal_label']=='Goal'][['x location shot', 'y_
      ↪location shot']].values, 'black', 'goals')
      plt.legend(loc='upper right')
      plt.show()
```



```
[ ]: def allgoals(fig,df):
    sns.histplot(data=df_regular_play, x="x location shot", y="y location_
↪shot",cbar=True,cbar_kws=dict(shrink=.25), pmax=0.85, palette='Reds')
    #px.density_heatmap(df_regular_play, x="x location shot",y ="y location_
↪shot")
    plt.axis('off')
    return plt
```

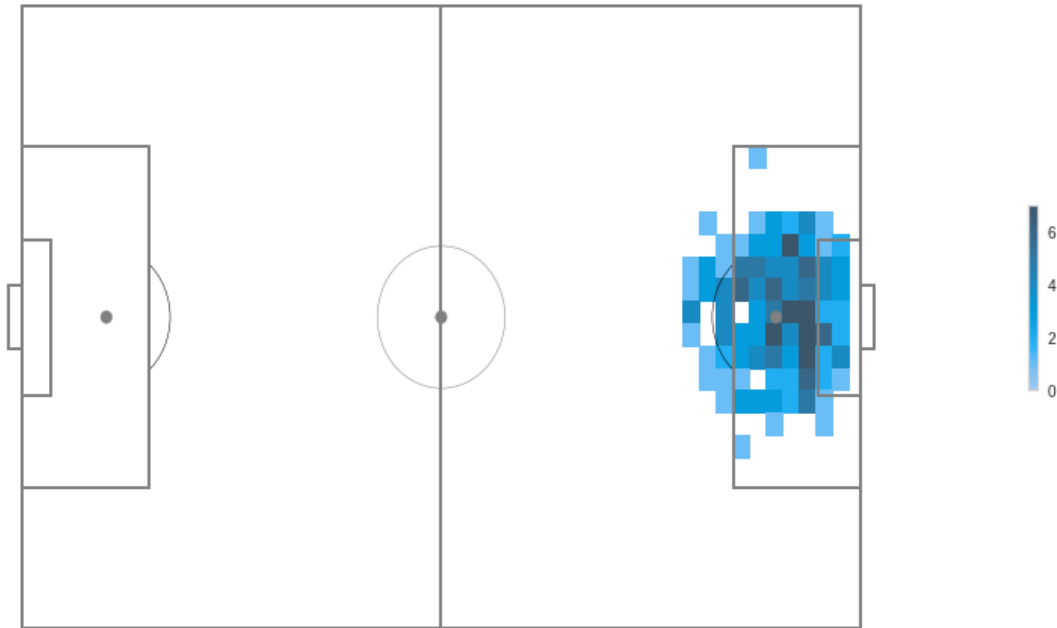
```
[ ]: df_goals = df_regular_play[df_regular_play['Goal_label']=='Goal']
pitch = pitchplot()
pitch = allgoals(pitch,df_goals)
plt.show()
```



```
[ ]: def playergoals(fig,df,player):
    df_player = df[df['player name'] == player]
    sns.histplot(data=df_player, x="x location shot", y="y location_
↪shot",cbar=True,cbar_kws=dict(shrink=.25), pmax=1)
    plt.title(player, fontsize = 30)
    #px.density_heatmap(df_regular_play, x="x location shot",y ="y location_
↪shot")
    plt.axis('off')
    return plt
```

```
[ ]: df_goals = df_regular_play[df_regular_play['Goal_label']=='Goal']
pitch = pitchplot()
pitch = playergoals(pitch,df_goals,'Lionel Andrés Messi Cuccittini')
plt.show()
```

Lionel Andrés Messi Cuccittini



```
[ ]: distance = train[["shot distance", "binary_outcome"]]  
distance["shot distance"] = distance["shot distance"].floordiv(5)  
distance["binary_outcome"] = pd.to_numeric(distance["binary_outcome"]) * 100  
outcome = distance.groupby("shot distance").mean().reset_index().iloc[0:9]
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

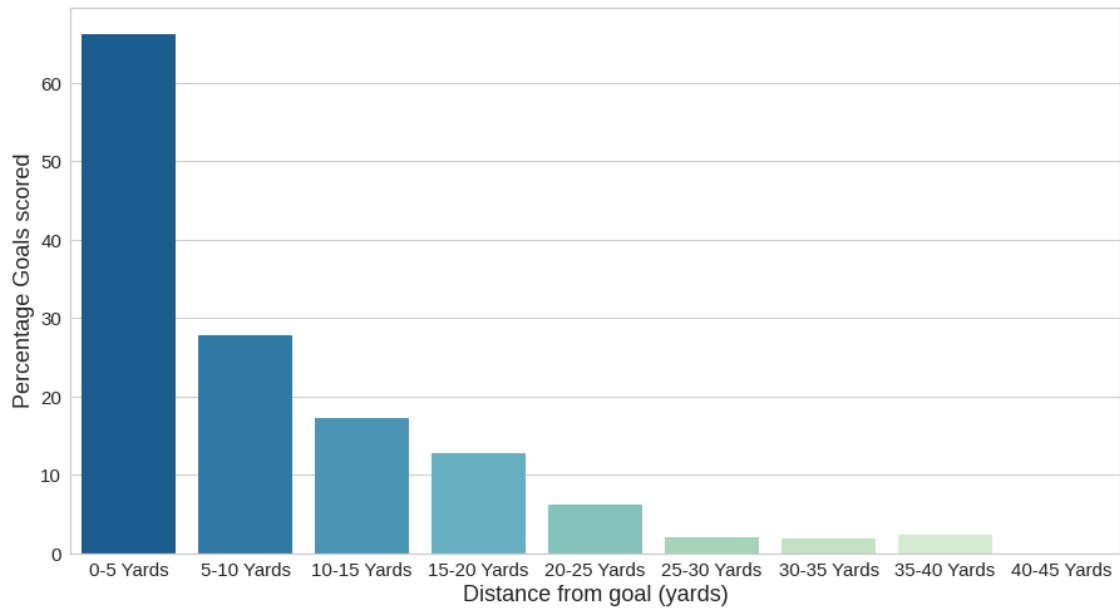
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
[ ]: fig, ax = plt.subplots(1, 1, figsize=(15, 8))  
sns.barplot(data=outcome,x="shot distance", y="binary_outcome", ax=ax,  
           palette='GnBu_r')
```

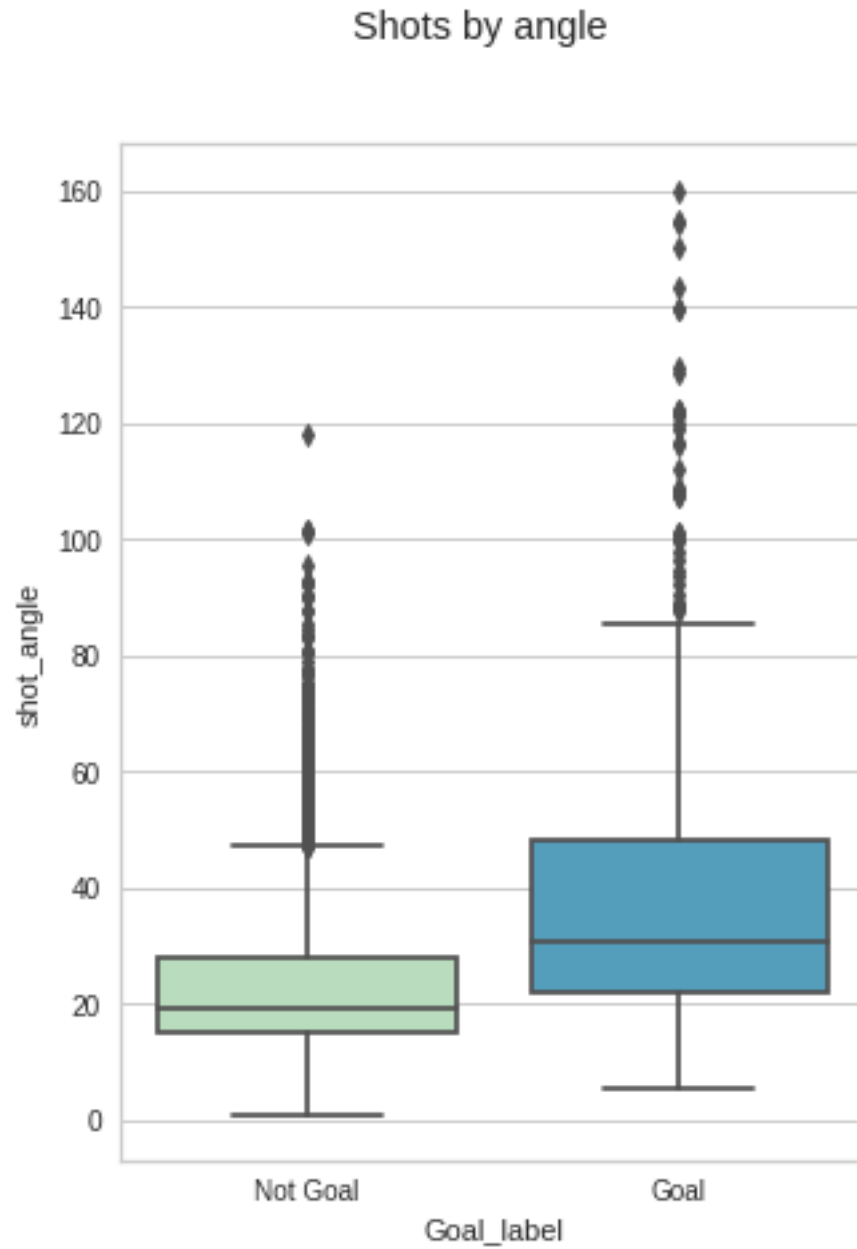
```
plt.xticks(np.arange(9), ("0-5 Yards", "5-10 Yards", "10-15 Yards", "15-20_Yards", "20-25 Yards", "25-30 Yards", "30-35 Yards", "35-40 Yards", "40-45_Yards"))
plt.xlabel("Distance from goal (yards)", size=18)
plt.ylabel("Percentage Goals scored", size=18)
plt.tick_params(labelsize=15)
```



Clearly the number of goals are reducing as the distance is increasing. Most of the goals are scored from within the box.

```
[ ]: fig,ax=plt.subplots(1,1,figsize=(5,7))
fig.suptitle('Shots by angle')
sns.boxplot(x='Goal_label',y='shot_angle',data=df_regular_play, palette =_GnBu')
```

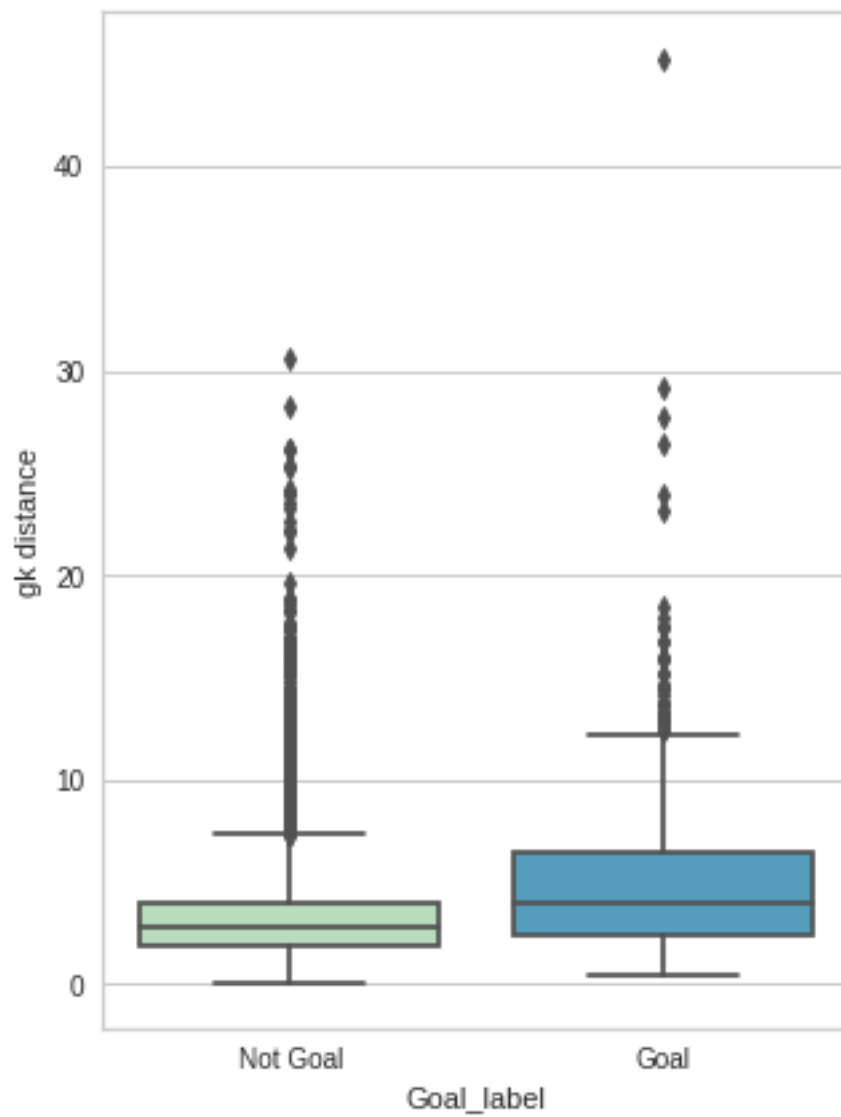
```
[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7fbc12865ed0>
```



```
[ ]: fig,ax=plt.subplots(1,1,figsize=(5,7))
fig.suptitle('Shots by Goal Keeper Distance')
sns.boxplot(x='Goal_label',y='gk distance',data=df_regular_play, palette =_
↪ 'GnBu')
```

```
[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7fbc127ec590>
```

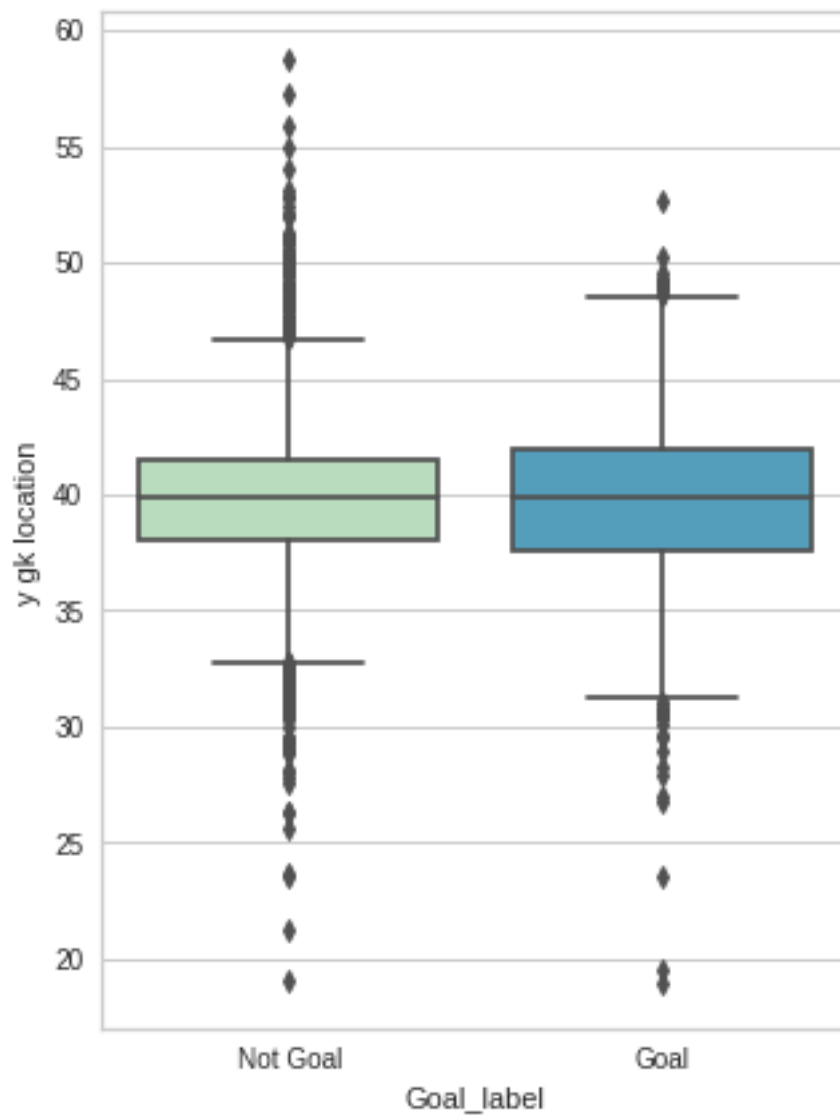
Shots by Goal Keeper Distance



```
[ ]: fig,ax=plt.subplots(1,1,figsize=(5,7))
fig.suptitle('Shots by Goal Keeper Location')
sns.boxplot(x='Goal_label',y='y gk location',data=df_regular_play, palette =_
↪ 'GnBu')
```

```
[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7fbc127f5050>
```

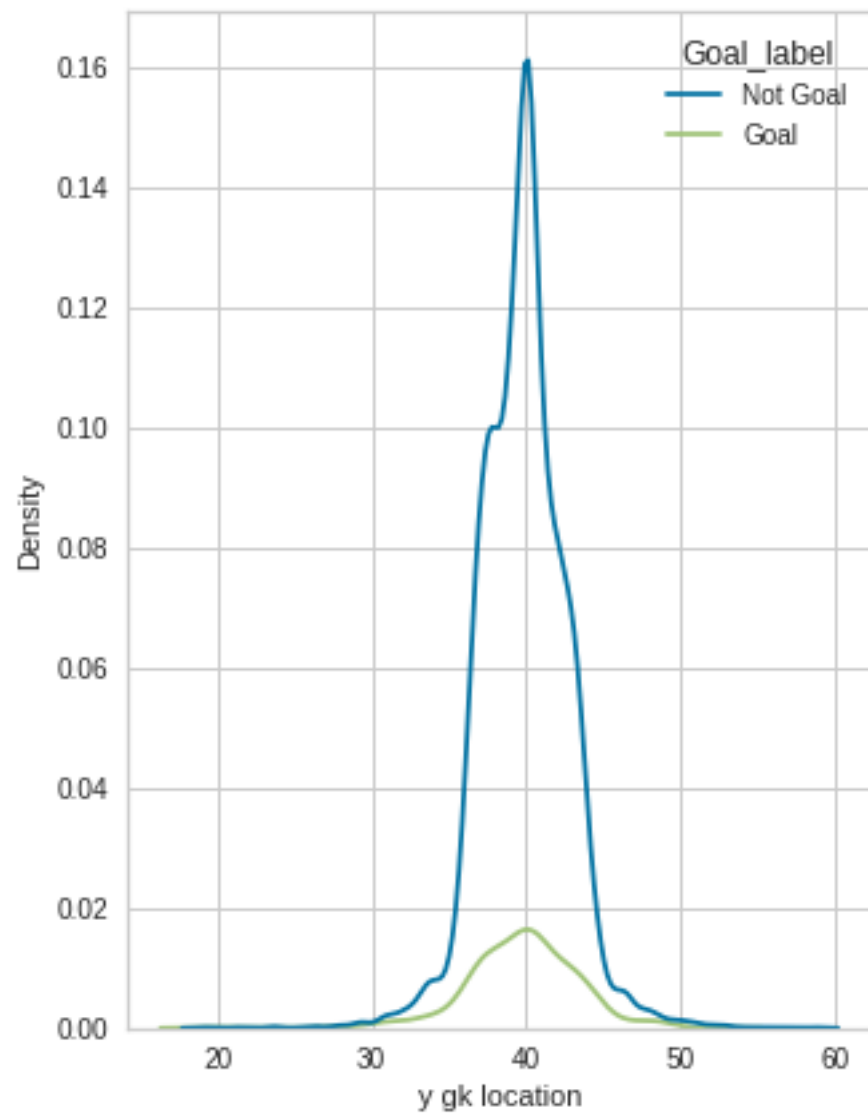
Shots by Goal Keeper Location



```
[ ]: fig,ax=plt.subplots(1,1,figsize=(5,7))
fig.suptitle('Shots by Goal Keeper Y coordinates')
#sns.boxplot(x='Goal_label',y='y gk location',data=df_regular_play, palette =_
↪ 'GnBu')
sns.kdeplot(data=df_regular_play, x="y gk location", hue="Goal_label")
```

```
[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7fbc1270e890>
```

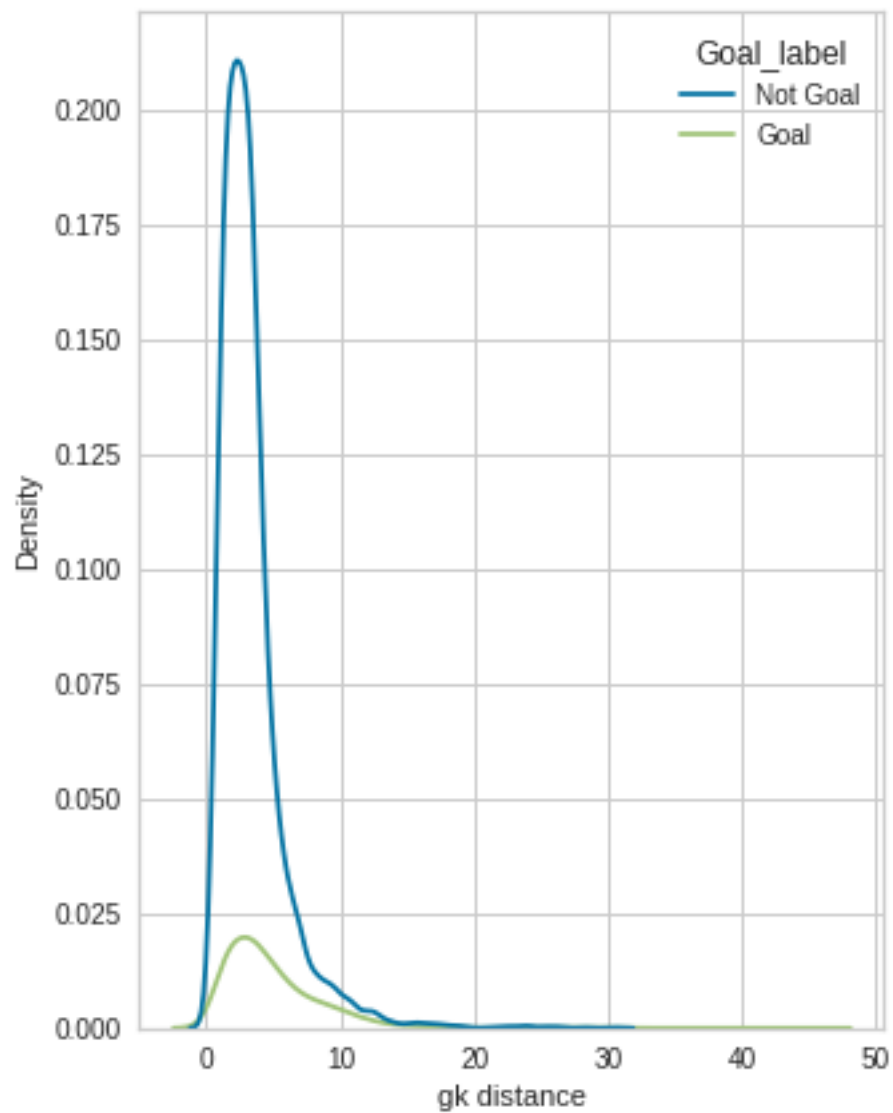
Shots by Goal Keeper Y coordinates



```
[ ]: fig,ax=plt.subplots(1,1,figsize=(5,7))
fig.suptitle('Shots by Goal Keeper Distance')
#sns.boxplot(x='Goal_label',y='y gk location',data=df_regular_play, palette =_
↪ 'GnBu')
sns.kdeplot(data=df_regular_play, x="gk distance", hue="Goal_label")
```

```
[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7fbc12621a90>
```

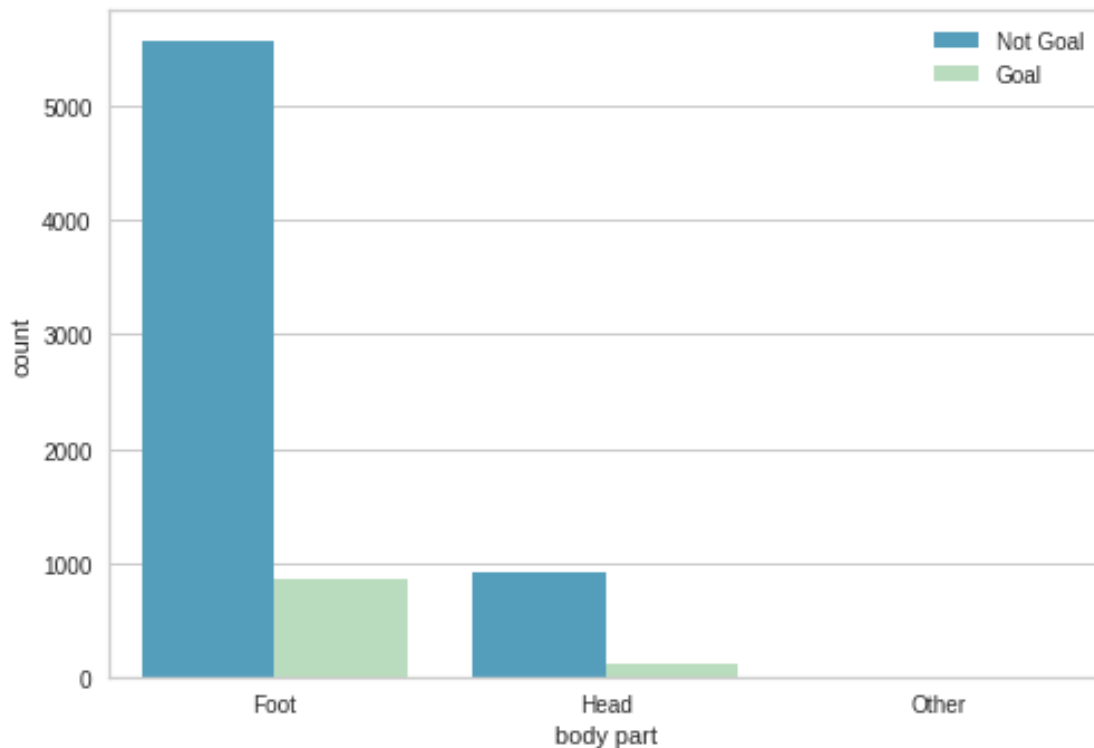
Shots by Goal Keeper Distance



```
[ ]: fig,ax=plt.subplots(1,1)
fig.suptitle('Shots by bodypart')
sns.countplot(x='bodypart',order=['Foot','Head','Other'],hue='Goal_label',data=df_regular_play,palette = 'GnBu_r')
plt.legend(loc='upper right')
```

```
[ ]: <matplotlib.legend.Legend at 0x7fbc126fc410>
```

Shots by bodypart



```
[ ]: footed_shots = (df_regular_play['body part']=='Foot')
shots_from_foot = df_regular_play[footed_shots].shape[0]
goals_from_foot = sum(df_regular_play[footed_shots]['binary_outcome'])

headed_shots = (df_regular_play['body part']=='Head')
shots_from_head = df_regular_play[headed_shots].shape[0]
goals_from_head = sum(df_regular_play[headed_shots]['binary_outcome'])

feetperc = goals_from_foot/shots_from_foot
headperc = goals_from_head/shots_from_head

print(feetperc*100,'%')
print(headperc*100,'%')
```

```
13.378967019290602 %
11.346153846153847 %
```

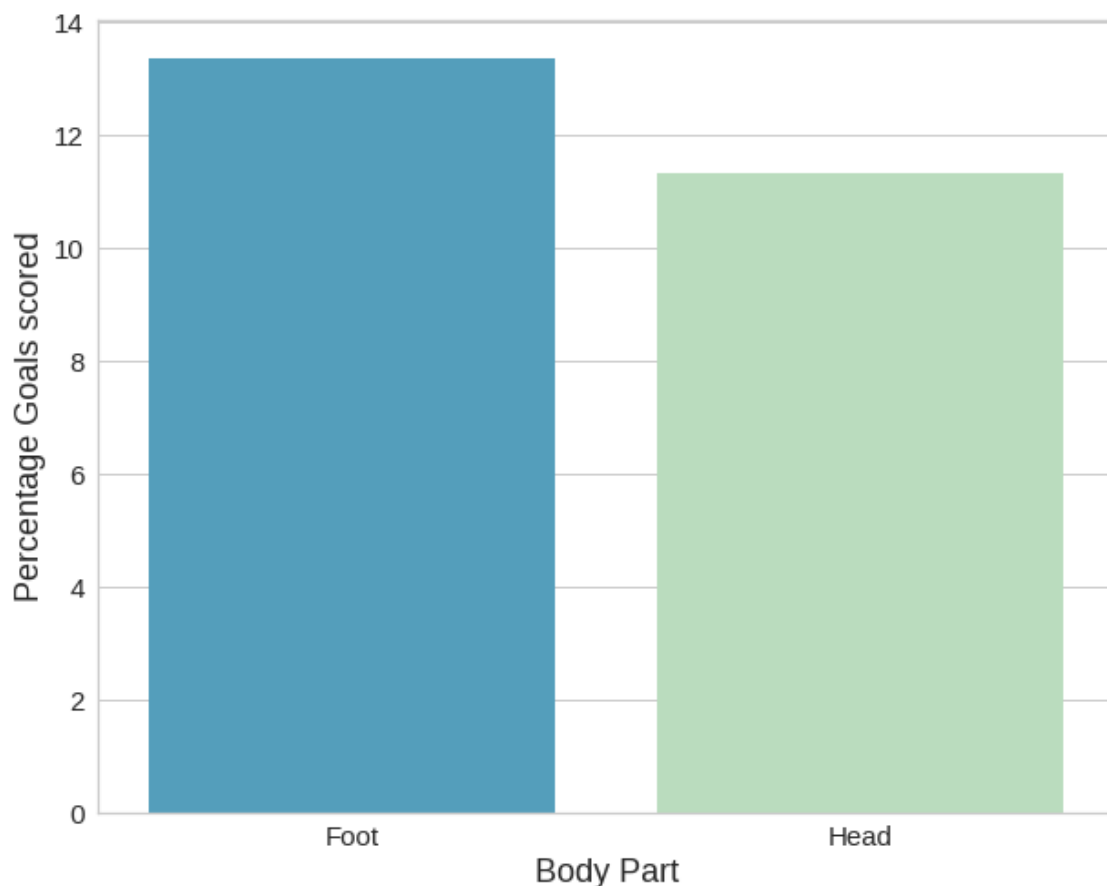
```
[ ]: shots_from_foot = df_regular_play[footed_shots]
shots_from_foot = shots_from_foot[["body part", "binary_outcome"]]
shots_from_head = df_regular_play[headed_shots]
shots_from_head = shots_from_head[["body part", "binary_outcome"]]
```

```
bodydf = pd.concat([shots_from_head, shots_from_foot], ignore_index=True)
```

```
[ ]: bodydf["binary_outcome"] = pd.to_numeric(bodydf["binary_outcome"]) * 100
outcome = bodydf.groupby("body part").mean().reset_index().iloc[0:9]
outcome
```

```
[ ]:  body part  binary_outcome
0     Foot      13.378967
1     Head      11.346154
```

```
[ ]: fig, ax = plt.subplots(1, 1, figsize=(10, 8))
sns.barplot(data=outcome, x="body part", y="binary_outcome", ax=ax,
            palette='GnBu_r')
plt.xticks(np.arange(2), ("Foot", "Head"))
plt.xlabel("Body Part", size=18)
plt.ylabel("Percentage Goals scored", size=18)
plt.tick_params(labelsize=15)
```

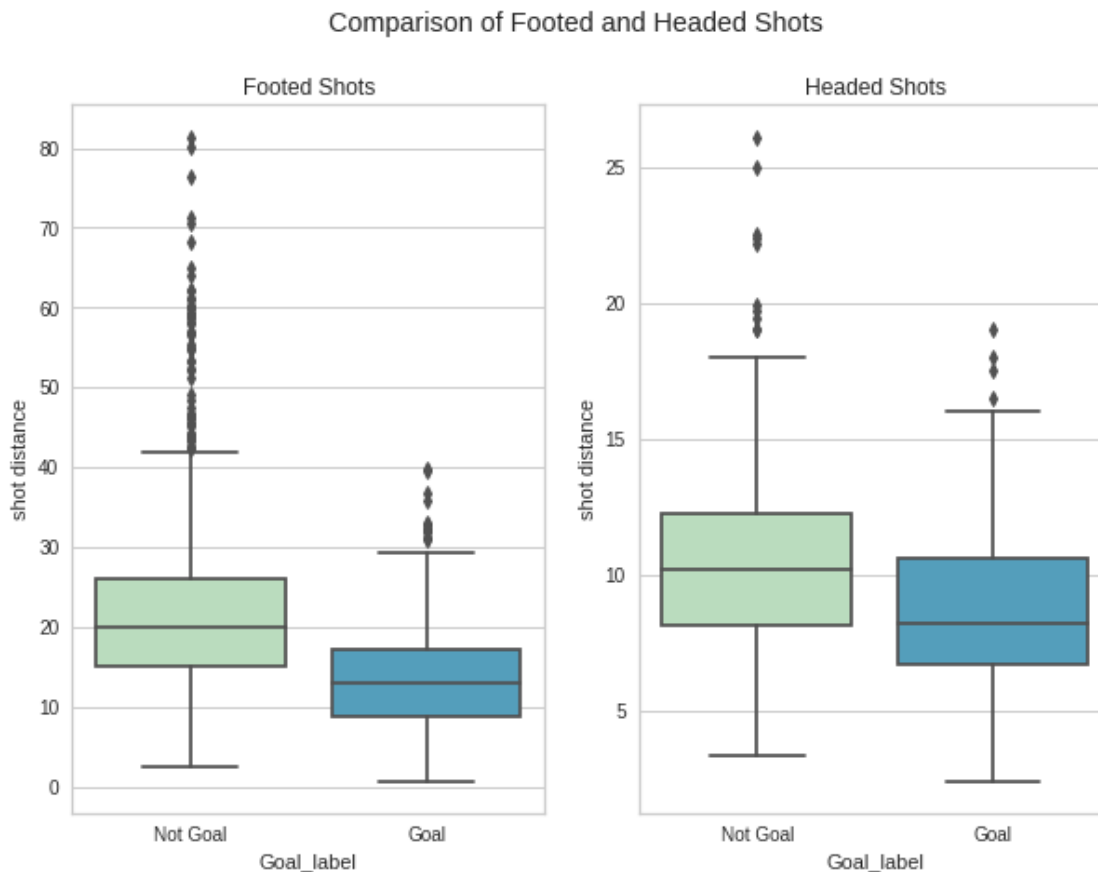


This shows that headed goals are more effective than footed goals. There is a skewness that we

need to consider first.

```
[ ]: fig,axis = plt.subplots(1,2, figsize=(10,7))
fig.suptitle('Comparison of Footed and Headed Shots')
sns.boxplot(x='Goal_label',y='shot_↵
↵distance',data=df_regular_play[footed_shots],ax=axis[0],palette='GnBu')
sns.boxplot(x='Goal_label',y='shot_↵
↵distance',data=df_regular_play[headed_shots],ax=axis[1],palette='GnBu')
axis[0].set_title('Footed Shots')
axis[1].set_title('Headed Shots')
```

```
[ ]: Text(0.5, 1.0, 'Headed Shots')
```



```
[ ]: eighteen_yardf = df_regular_play[footed_shots]['shot distance']<18
footed18_shots = df_regular_play[footed_shots][eighteen_yardf].shape[0]
footed18_goals = ↵
↵sum(df_regular_play[footed_shots][eighteen_yardf]['binary_outcome'])

eighteen_yardh = df_regular_play[headed_shots]['shot distance']<18
```

```

headed18_shots = df_regular_play[headed_shots][eighteen_yardh].shape[0]
headed18_goals = □
↳sum(df_regular_play[headed_shots][eighteen_yardh]['binary_outcome'])

```

This figure shows that almost all headed shots are taken from a maximum distance of 18 yards.

```

[ ]: foot18_shots = df_regular_play[footed_shots][eighteen_yardf]
foot18_shots = foot18_shots[["body part", "binary_outcome"]]
head18_shots = df_regular_play[headed_shots][eighteen_yardh]
head18_shots = head18_shots[["body part", "binary_outcome"]]
bodydf = pd.concat([head18_shots, foot18_shots], ignore_index=True)

```

```

[ ]: bodydf["binary_outcome"] = pd.to_numeric(bodydf["binary_outcome"]) * 100
outcome = bodydf.groupby("body part").mean().reset_index().iloc[0:9]
outcome

```

```

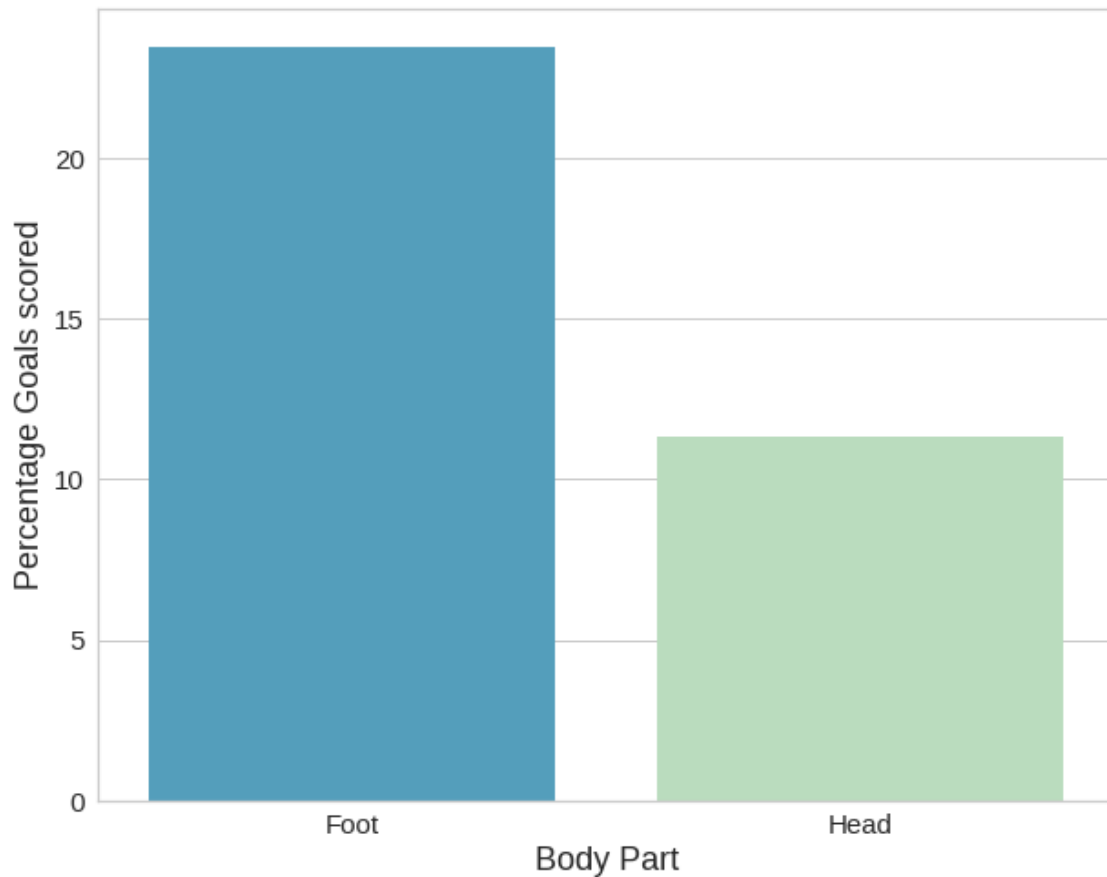
[ ]:   body part  binary_outcome
0      Foot      23.461538
1      Head      11.306043

```

```

[ ]: fig, ax = plt.subplots(1, 1, figsize=(10, 8))
sns.barplot(data=outcome, x="body part", y="binary_outcome", ax=ax, □
↳palette='GnBu_r')
plt.xticks(np.arange(2), ("Foot", "Head"))
plt.xlabel("Body Part", size=18)
plt.ylabel("Percentage Goals scored", size=18)
plt.tick_params(labelsize=15)

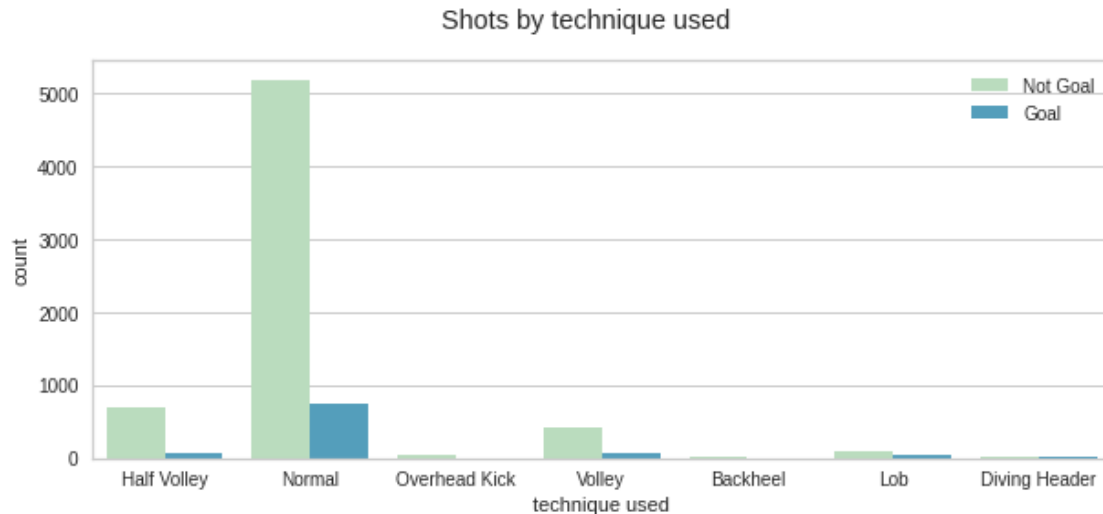
```



After checking the goals from the 18 yard distance, it is clear that the footed goals are more effective.

```
[ ]: fig,ax=plt.subplots(1,1, figsize=(10,4))
fig.suptitle('Shots by technique used')
sns.countplot(x='technique used',hue='Goal_label',data=df_regular_play, palette='GnBu')
plt.legend(loc='upper right')
```

```
[ ]: <matplotlib.legend.Legend at 0x7fbc1238f390>
```



```
[ ]: normal_shots = (df_regular_play['technique used']=='Normal')
volley_shots = (df_regular_play['technique used']=='Volley')
halfvolley_shots = (df_regular_play['technique used']=='Half Volley')
other_shots = (df_regular_play['technique used']=='Lob technique used') |
    ↳ (df_regular_play['technique used']!='Backheel') |
    ↳ (df_regular_play['technique used']!='Diving Header') |
    ↳ (df_regular_play['technique used']!='Overhead Kick')
```

```
[ ]: shots_normal = df_regular_play[normal_shots].shape[0]
goals_normal = sum(df_regular_play[normal_shots]['binary_outcome'])
perc_normal = goals_normal/shots_normal*100

shots_volley = df_regular_play[volley_shots].shape[0]
goals_volley = sum(df_regular_play[volley_shots]['binary_outcome'])
perc_volley = goals_volley/shots_volley*100

shots_halfvolley = df_regular_play[halfvolley_shots].shape[0]
goals_halfvolley = sum(df_regular_play[halfvolley_shots]['binary_outcome'])
perc_halfvolley = goals_halfvolley/shots_halfvolley*100

shots_other = df_regular_play[other_shots].shape[0]
goals_other = sum(df_regular_play[other_shots]['binary_outcome'])
perc_other = goals_other/shots_other*100

print(perc_normal)
print(perc_volley)
print(perc_halfvolley)
print(perc_other)
```

12.727272727272727

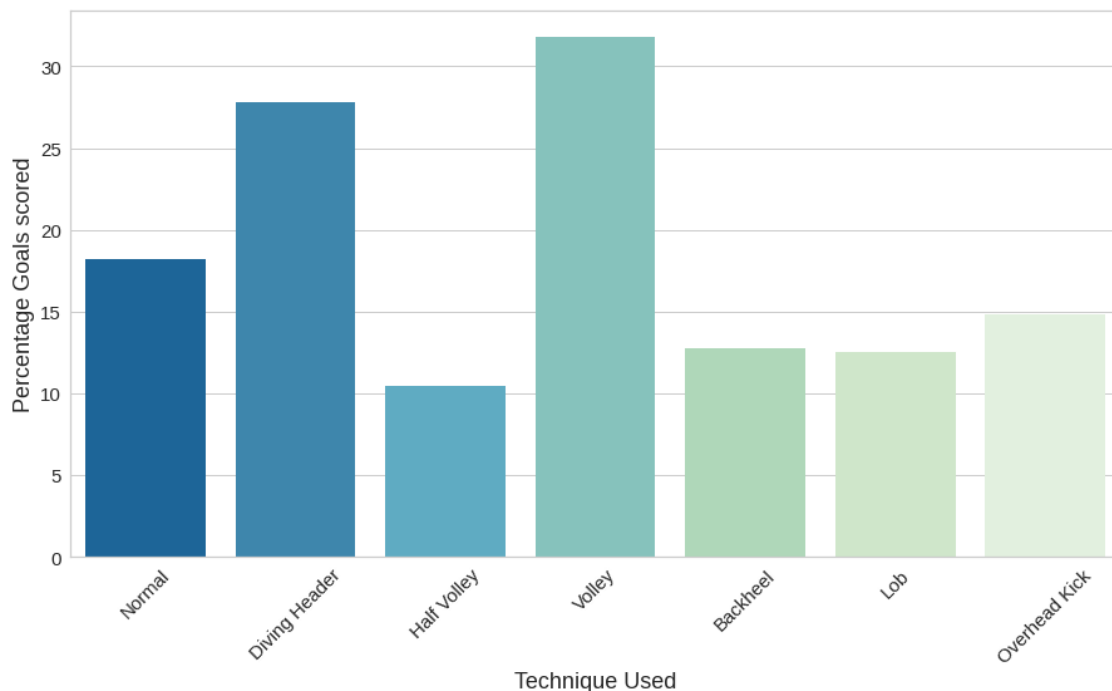
```
14.814814814814813
10.424710424710424
13.095875736475628
```

```
[ ]: play_pattern = train[["technique used", "binary_outcome"]]
play_pattern["binary_outcome"] = pd.to_numeric(play_pattern["binary_outcome"])
↳* 100
outcome = play_pattern.groupby("technique used").mean().reset_index().iloc[0:9]
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

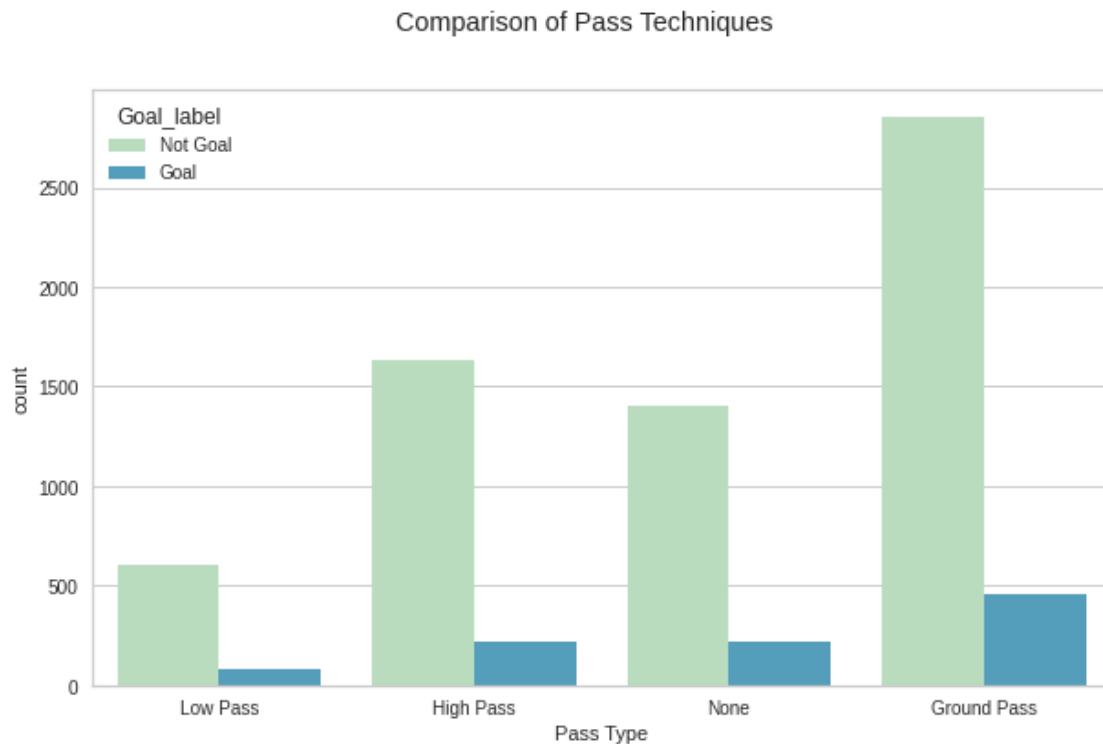
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
[ ]: fig, ax = plt.subplots(1, 1, figsize=(15, 8))
sns.barplot(data=outcome, x="technique used", y="binary_outcome", ax=ax,
↳palette='GnBu_r')
plt.xticks(np.arange(7), ("Normal", "Diving Header", "Half Volley", "Volley",
↳"Backheel", "Lob", "Overhead Kick"), rotation=45)
plt.xlabel("Technique Used", size=18)
plt.ylabel("Percentage Goals scored", size=18)
plt.tick_params(labelsize=15)
```



```
[ ]: fig,axis = plt.subplots(1,1, figsize=(10,6))
fig.suptitle('Comparison of Pass Techniques')
sns.countplot(x='Pass Type',data=df_regular_play,hue='Goal_label',ax=axis,palette='GnBu')
```

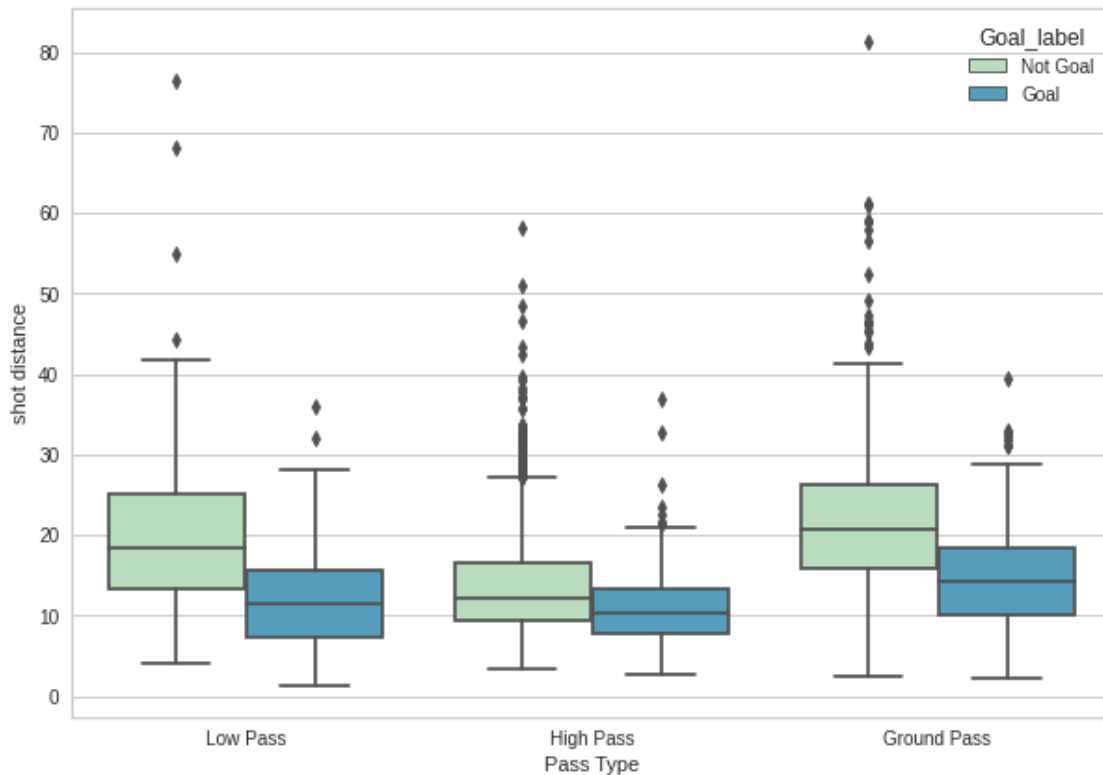
```
[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7fbc1222b9d0>
```



```
[ ]: fig,axis = plt.subplots(1,1, figsize=(10,7))
fig.suptitle('Comparison of Passes and Shot Distance')
df_none = df_regular_play[df_regular_play['Pass Type']!='None']
sns.boxplot(x='Pass Type',y='shot_distance',data=df_none,hue='Goal_label',ax=axis,palette='GnBu')
```

```
[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7fbc1219a7d0>
```

Comparison of Passes and Shot Distance



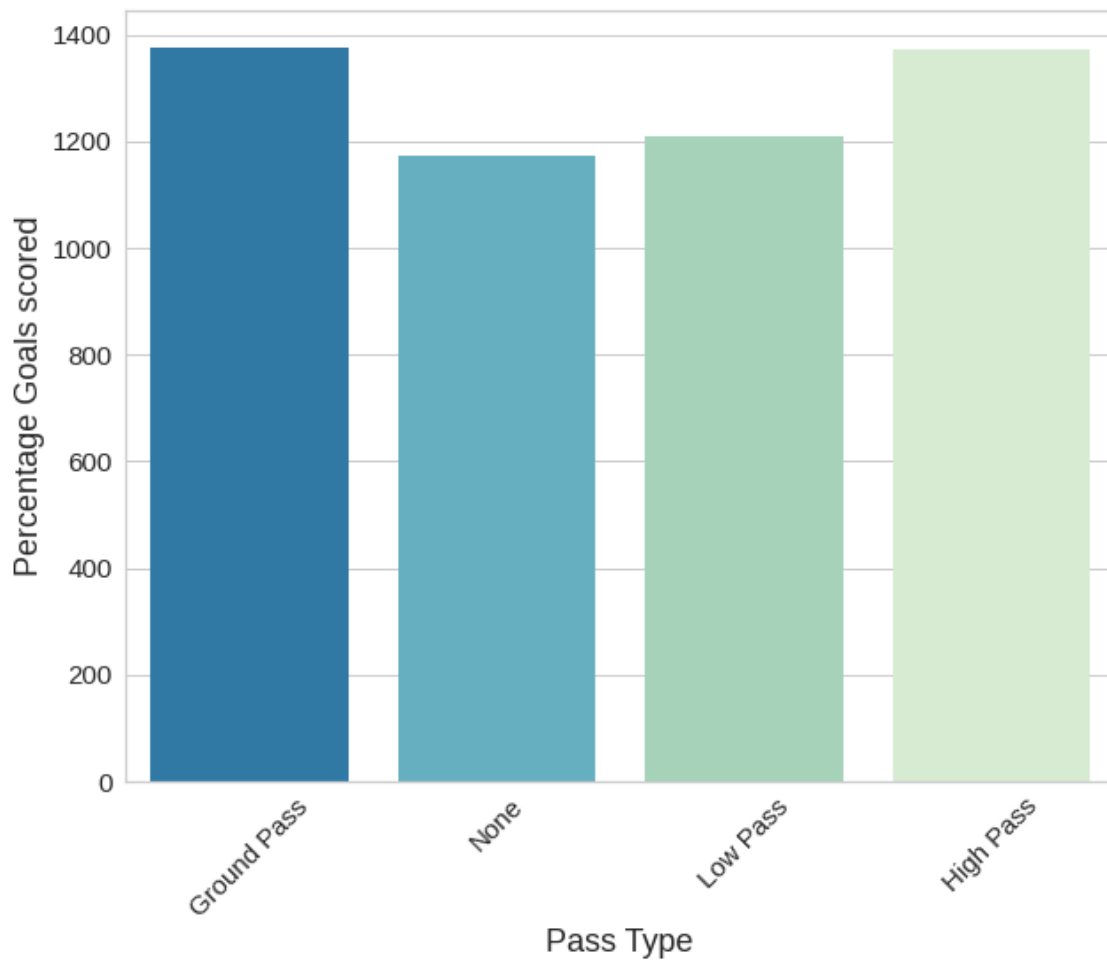
```
[ ]: pass_type = train[["Pass Type", "binary_outcome"]]
pass_type["binary_outcome"] = pd.to_numeric(play_pattern["binary_outcome"]) * 100
outcome = pass_type.groupby("Pass Type").mean().reset_index().iloc[0:9]
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

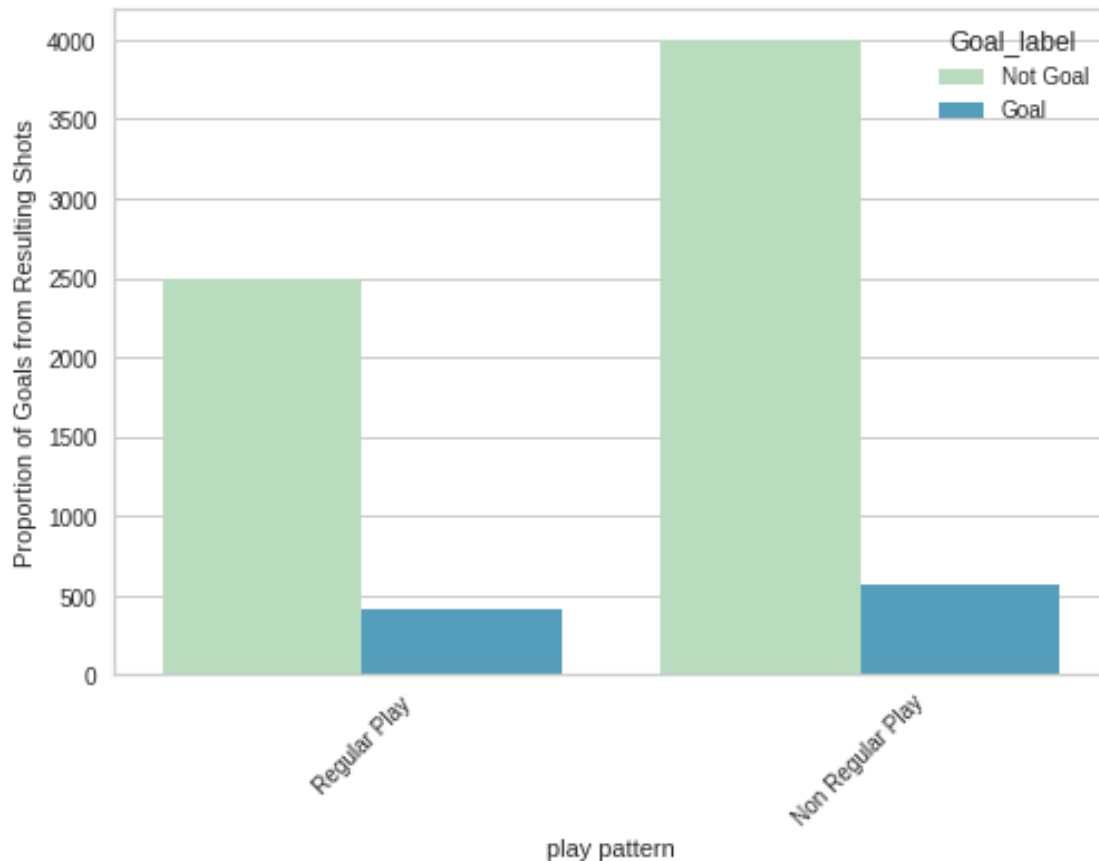
```
[ ]: fig, ax = plt.subplots(1, 1, figsize=(10, 8))
sns.barplot(data=outcome, x="Pass Type", y="binary_outcome", ax=ax, palette='GnBu_r')
plt.xticks(np.arange(4), ("Ground Pass", "None", "Low Pass", "High Pass"), rotation=45)
plt.xlabel("Pass Type", size=18)
plt.ylabel("Percentage Goals scored", size=18)
```

```
plt.tick_params(labelsize=15)
```



```
[ ]: sns.countplot(hue='Goal_label',x='play pattern',data=train,palette='GnBu')  
plt.xticks(rotation=45)  
plt.ylabel('Proportion of Goals from Resulting Shots')
```

```
[ ]: Text(0, 0.5, 'Proportion of Goals from Resulting Shots')
```



```
[ ]: train['play pattern'].value_counts()
```

```
[ ]: Non Regular Play    4565
      Regular Play      2903
      Name: play pattern, dtype: int64
```

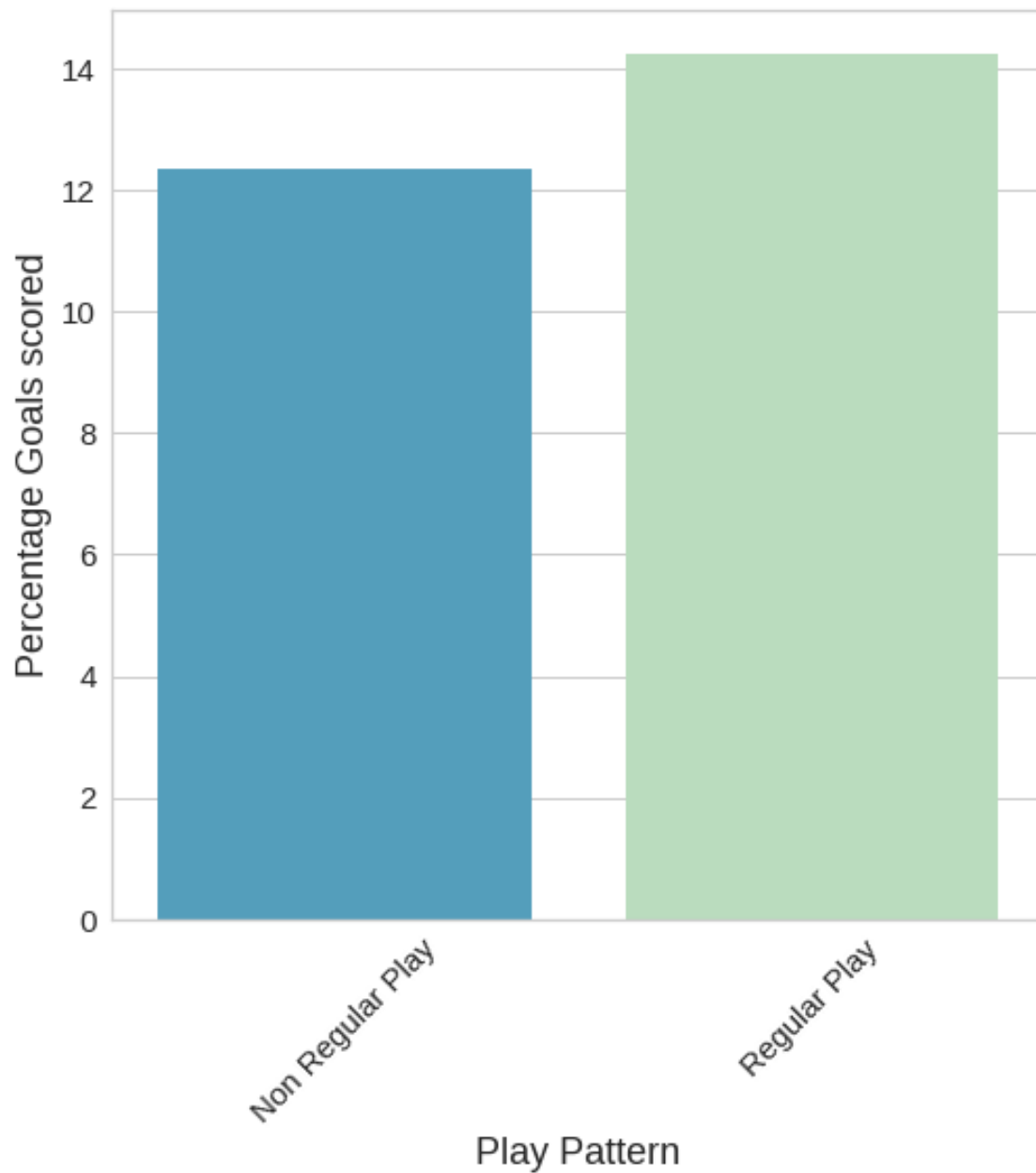
```
[ ]: play_pattern = train[["play pattern", "binary_outcome"]]
      play_pattern["binary_outcome"] = pd.to_numeric(play_pattern["binary_outcome"])*100
      outcome = play_pattern.groupby("play pattern").mean().reset_index().iloc[0:9]
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
[ ]: fig, ax = plt.subplots(1, 1, figsize=(8, 8))
```

```
sns.barplot(data=outcome,x="play pattern", y="binary_outcome", ax=ax,
            palette='GnBu_r')
plt.xticks(np.arange(2), ("Non Regular Play", "Regular Play"),rotation=45)
plt.xlabel("Play Pattern", size=18)
plt.ylabel("Percentage Goals scored", size=18)
plt.tick_params(labelsize=15)
```



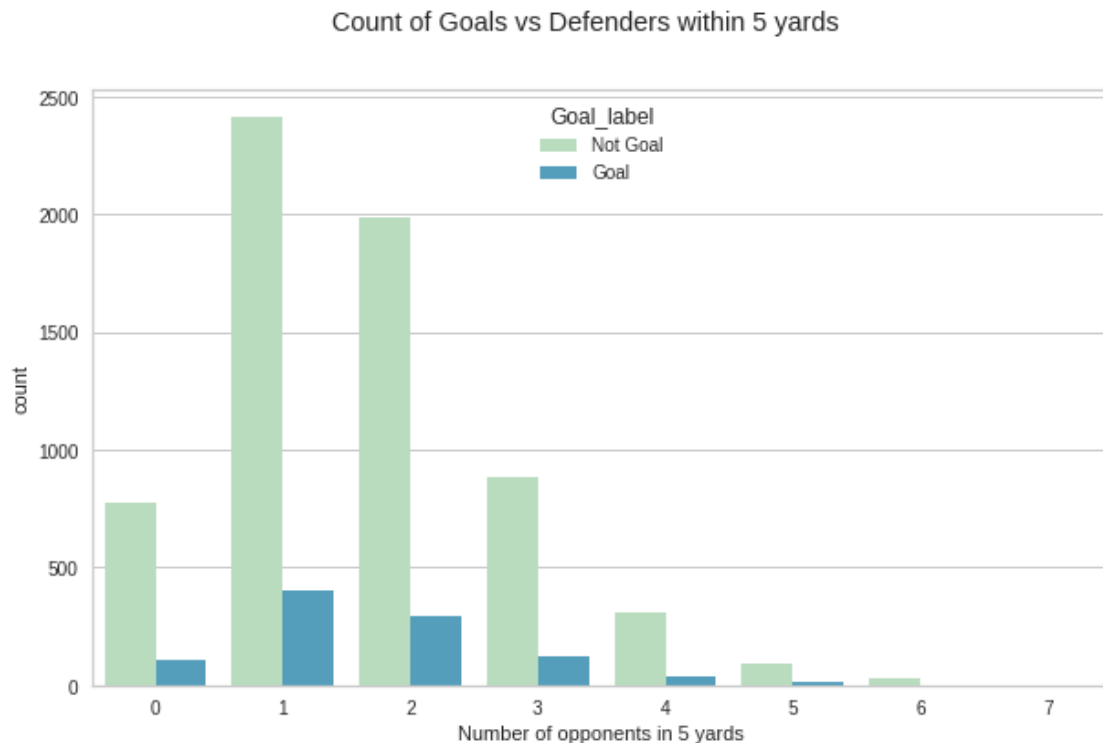
```
[ ]: shot_type = train[["type of shot", "binary_outcome"]]
shot_type["binary_outcome"] = pd.to_numeric(shot_type["binary_outcome"]) * 100
outcome = shot_type.groupby("type of shot").mean().reset_index().iloc[0:9]
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

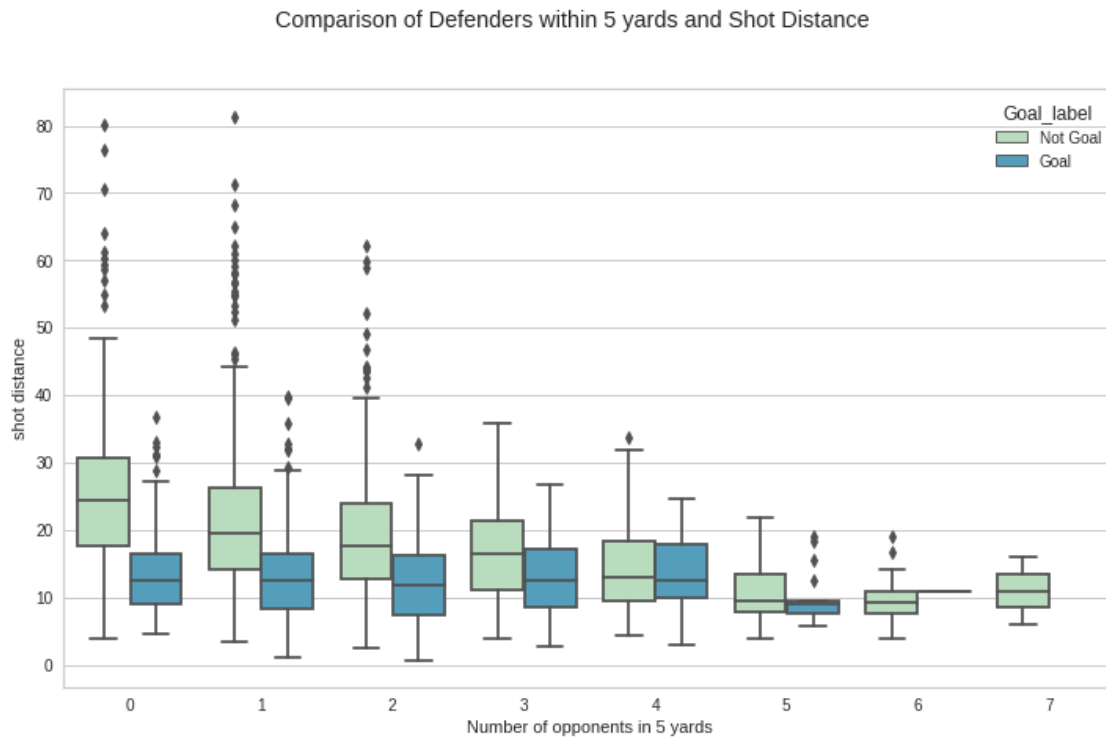
```
[ ]: fig,axis = plt.subplots(1,1, figsize=(10,6))
fig.suptitle('Count of Goals vs Defenders within 5 yards')
sns.countplot(x='Number of opponents in 5 yards',data=train,hue='Goal_label',ax=axis, palette='GnBu')
```

```
[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7fbc11f675d0>
```



```
[ ]: fig,axis = plt.subplots(1,1, figsize=(12,7))
fig.suptitle('Comparison of Defenders within 5 yards and Shot Distance')
df_none = df_regular_play[df_regular_play['Pass Type']!='None']
sns.boxplot(x='Number of opponents in 5 yards',y='shot_distance',data=train,hue='Goal_label',ax=axis, palette='GnBu')
```

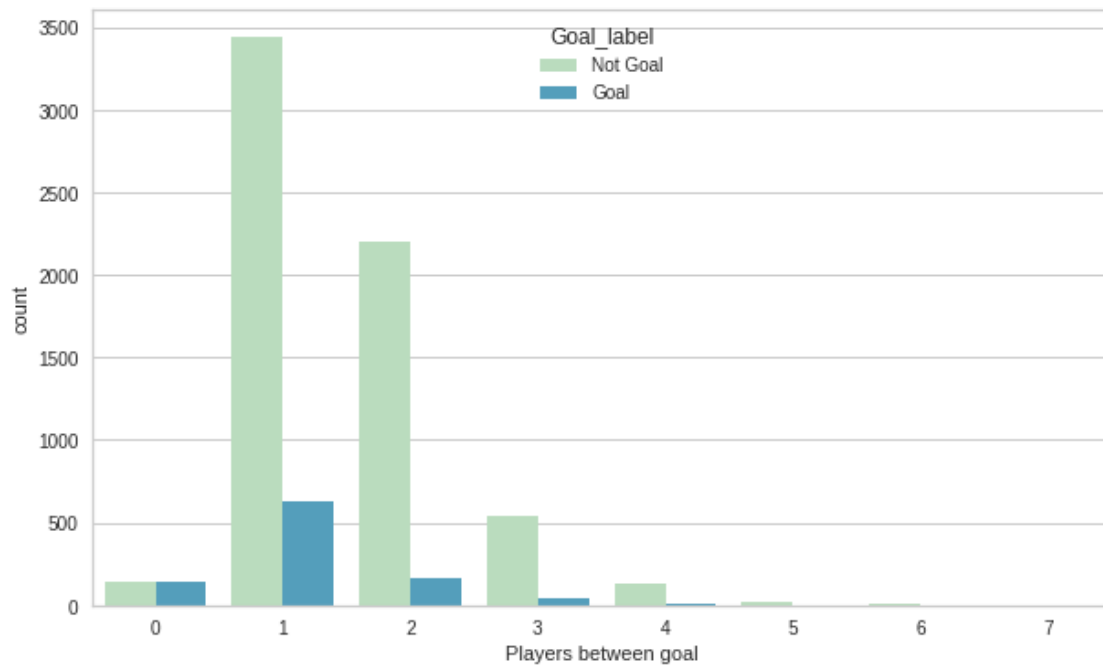
```
[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7fbc11f0c090>
```



```
[ ]: fig,axis = plt.subplots(1,1, figsize=(10,6))
fig.suptitle('Count of Goals vs Players between Goal')
sns.countplot(x='Players between goal',data=train,hue='Goal_label',ax=axis,palette='GnBu')
```

```
[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7fbc11d3fd10>
```

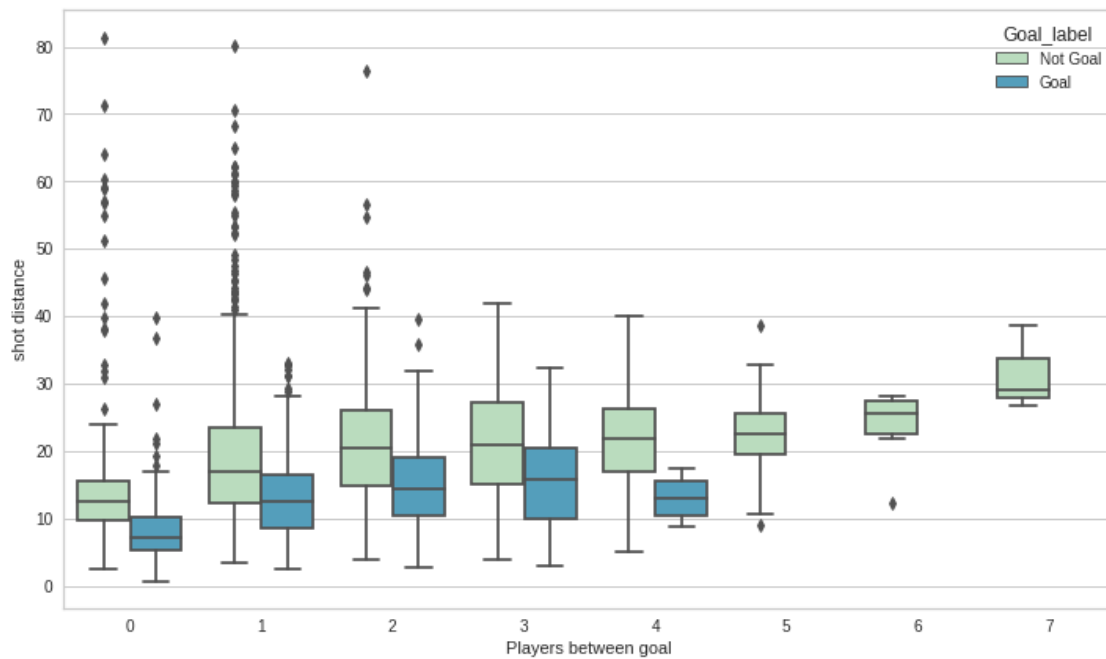
Count of Goals vs Players between Goal



```
[ ]: fig,axis = plt.subplots(1,1, figsize=(12,7))
fig.suptitle('Comparison of Players between Goal and Shot Distance')
sns.boxplot(x='Players between goal',y='shot_↵
↵distance',data=train,hue='Goal_label',ax=axis,palette='GnBu')
```

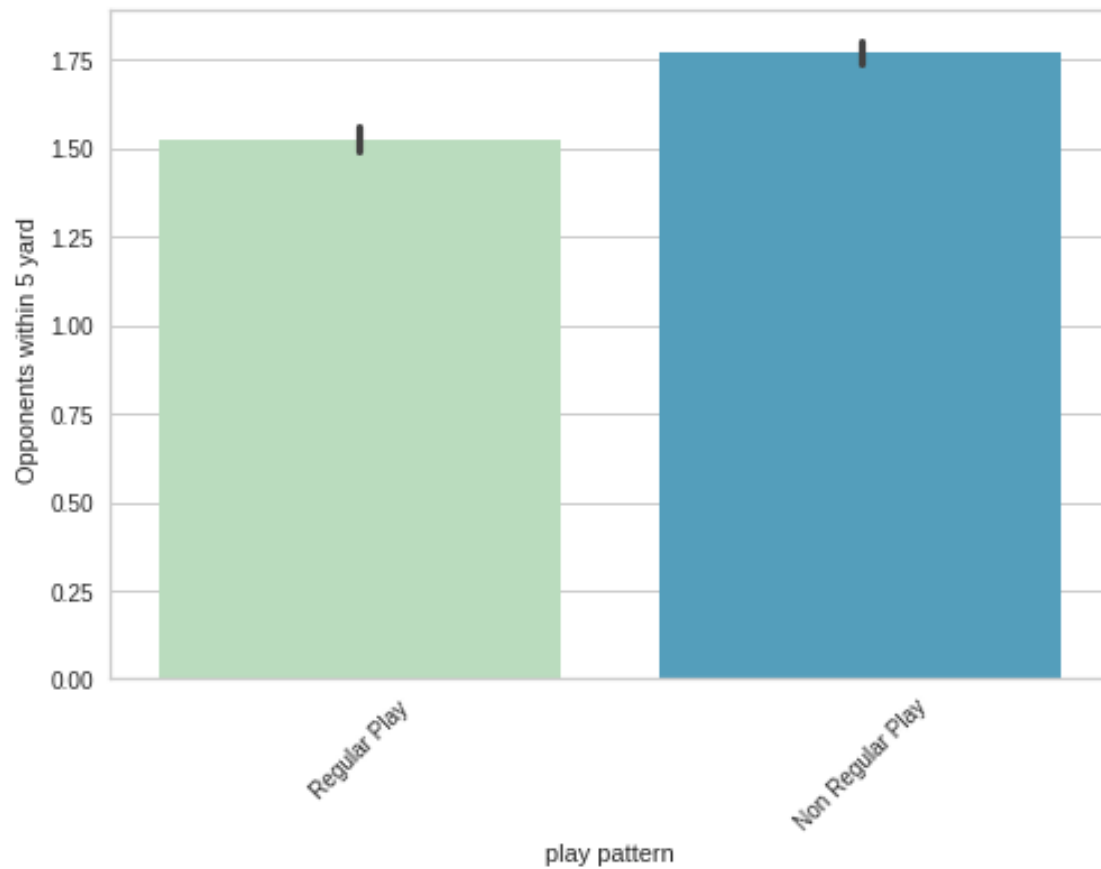
```
[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7fbc11c65050>
```

Comparison of Players between Goal and Shot Distance



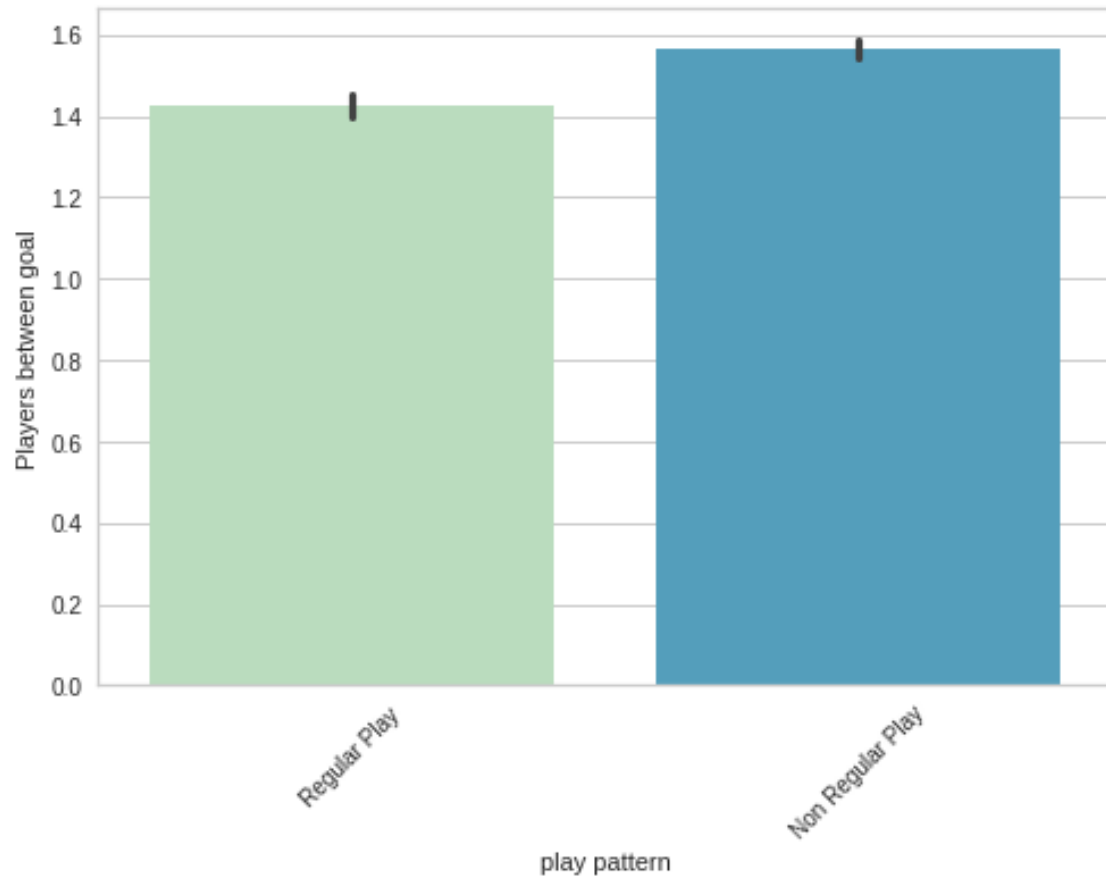
```
[ ]: sns.barplot(y='Number of opponents in 5 yards',x='play_
    ↪pattern',data=train,palette='GnBu')
plt.xticks(rotation=45)
plt.ylabel('Opponents within 5 yard ')
```

```
[ ]: Text(0, 0.5, 'Opponents within 5 yard ')
```



```
[ ]: sns.barplot(y='Players between goal',x='play pattern',data=train,palette='GnBu')
plt.xticks(rotation=45)
plt.ylabel('Players between goal')

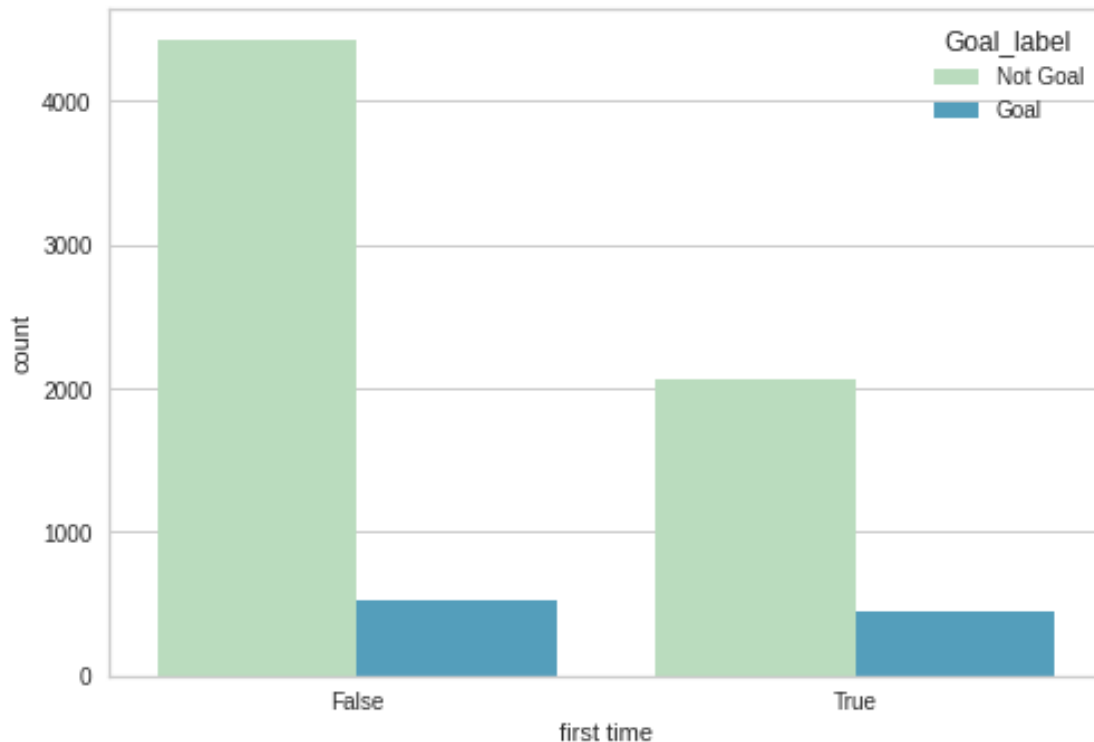
[ ]: Text(0, 0.5, 'Players between goal')
```



As we can see that the number of players between the goal and the shot taker is the least when the play type is a counter attack.

```
[ ]: sns.countplot(x='first time',data=train,palette='GnBu', hue = 'Goal_label')
```

```
[ ]: <matplotlib.axes._subplots.AxesSubplot at 0x7fbc11a58f10>
```

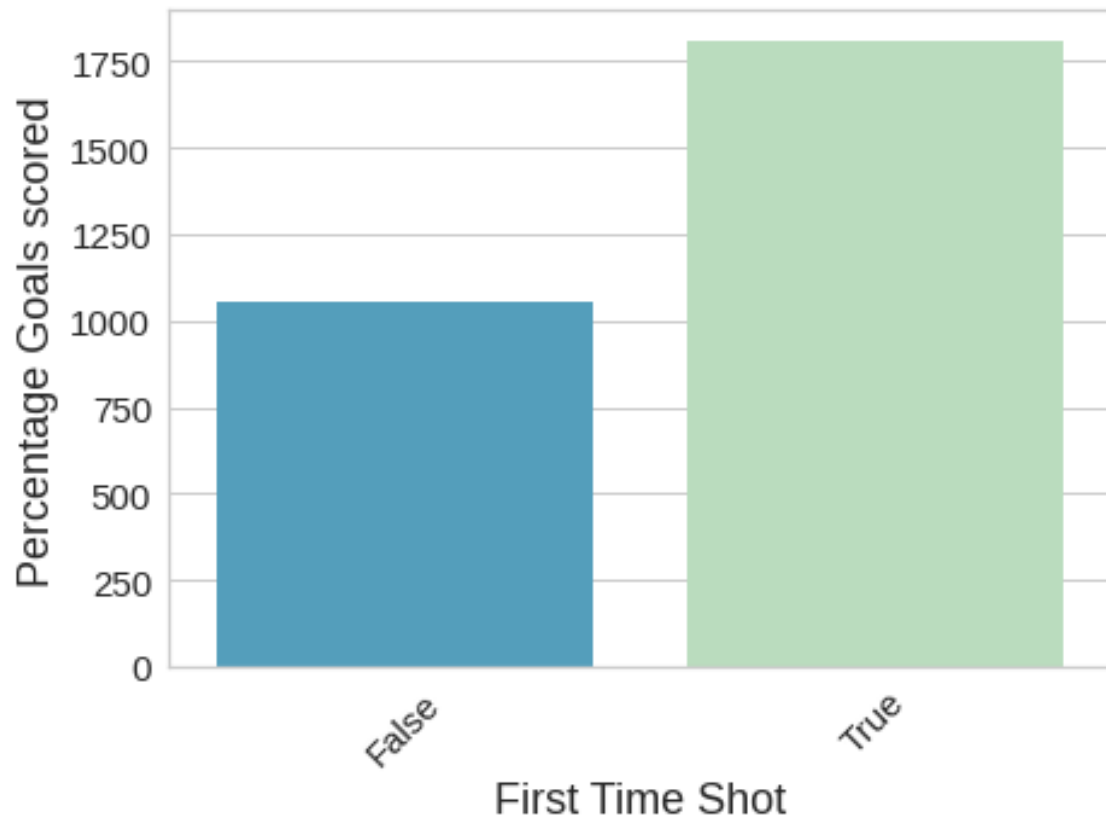


```
[ ]: first_time = train[["first time", "binary_outcome"]]
first_time["binary_outcome"] = pd.to_numeric(play_pattern["binary_outcome"]) * 100
outcome = first_time.groupby("first time").mean().reset_index().iloc[0:9]
```

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
[ ]: fig, ax = plt.subplots(1, 1, figsize=(7, 5))
sns.barplot(data=outcome, x="first time", y="binary_outcome", ax=ax, palette='GnBu_r')
plt.xticks(np.arange(2), ("False", "True"), rotation=45)
plt.xlabel("First Time Shot", size=18)
plt.ylabel("Percentage Goals scored", size=18)
plt.tick_params(labelsize=15)
```



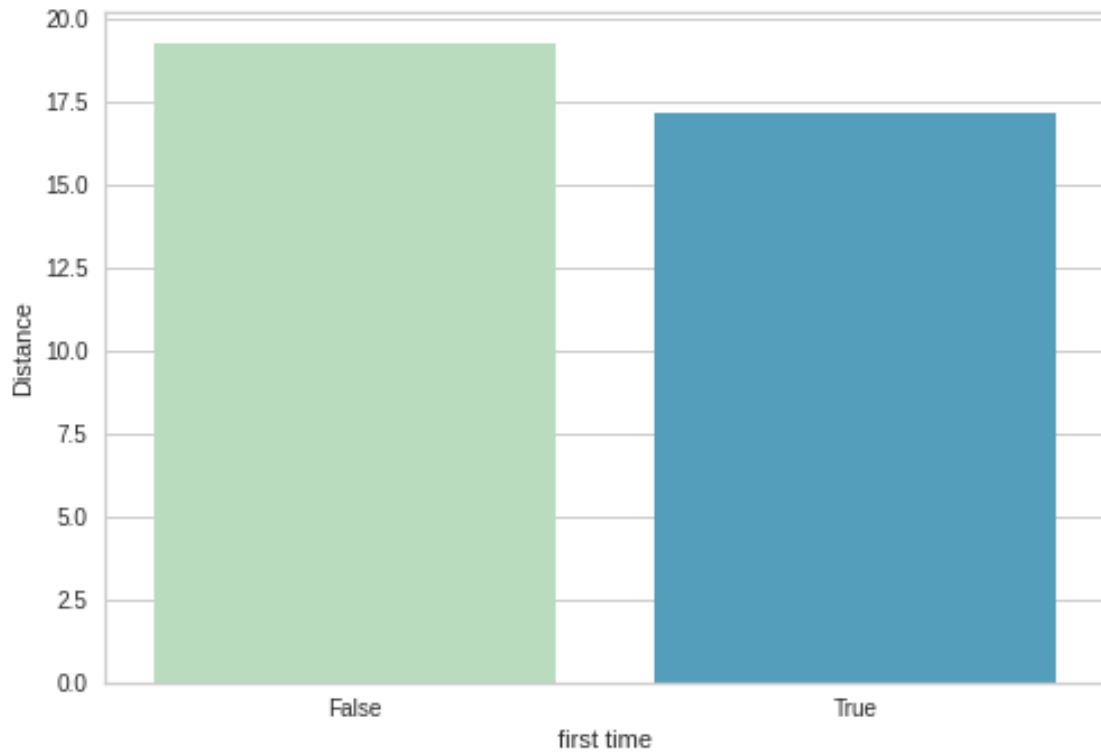
It looks like shots taken first time result in goals on a higher rate. Let's analyze this further.

```
[ ]: train.groupby('first time', as_index=False)['shot distance'].mean()
```

```
[ ]:   first time  shot distance
0      False      19.222691
1       True      17.142517
```

```
[ ]: sns.barplot(y='shot distance',x='first time',data=train,palette='GnBu',ci=False)
plt.ylabel('Distance')
```

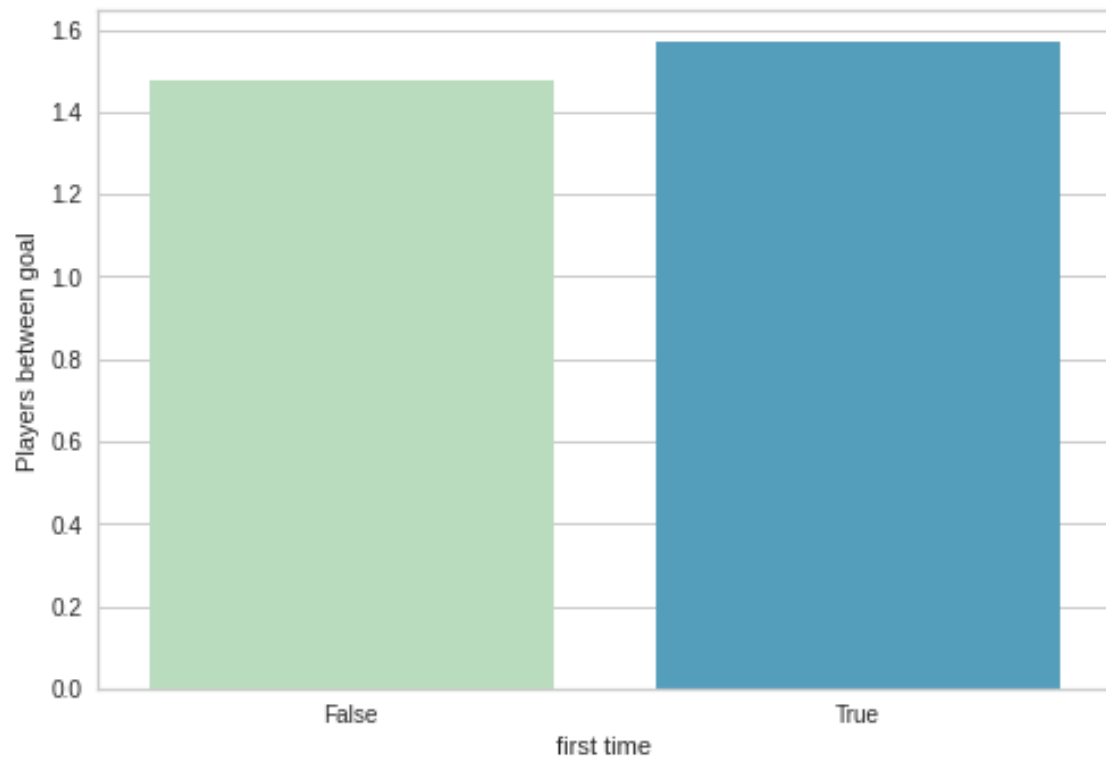
```
[ ]: Text(0, 0.5, 'Distance')
```



On an average, first time goals are shot from a distance of ~17 yards, while goals that are not scored are shot from ~20 yards.

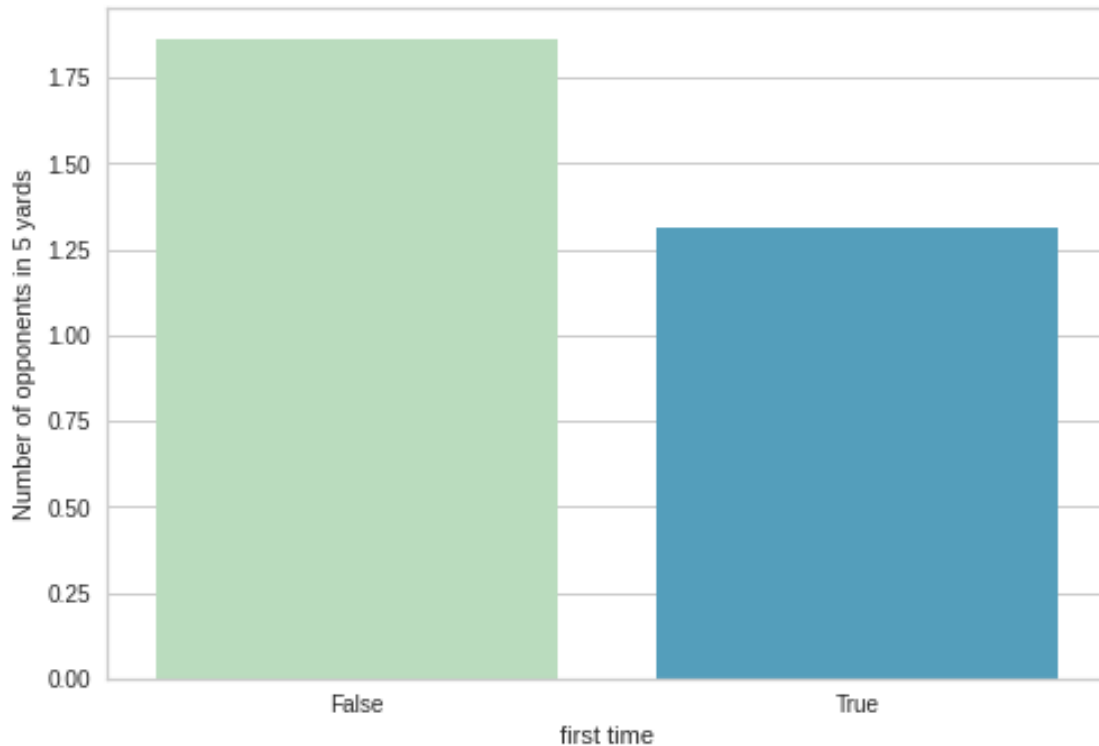
```
[ ]: sns.barplot(y='Players between goal',x='first time',data=train,palette='GnBu',ci=False)
plt.ylabel('Players between goal')
```

```
[ ]: Text(0, 0.5, 'Players between goal')
```



```
[ ]: sns.barplot(y='Number of opponents in 5 yards',x='first_time',data=train,palette='GnBu', ci=False)
plt.ylabel('Number of opponents in 5 yards')
```

```
[ ]: Text(0, 0.5, 'Number of opponents in 5 yards')
```



```
[ ]: train['official xg']
```

```
[ ]: shot id
690a4746-a671-47e0-bece-6a44a980e032    0.018941
eeb3e791-4c29-47a5-9a5d-6fd235139e3f    0.027518
a8a8caaa-5a8c-4d30-9df2-28e9266d27a1    0.032669
8bc51050-28de-4c1c-b25f-9a64a5c3c93b    0.257564
d95a7ee7-06c9-47c1-9e7c-b2462c53eaf8    0.752807
...
3681dbba-aa3a-402a-aa29-baebad73adc8    0.039551
ff77f470-47ee-428d-b018-0f2203ac6641    0.908923
42293ce4-e3d6-4e65-9040-e7f6413fbe44    0.076067
b69a6316-0c39-4623-a255-7c89212b68e5    0.239924
41e31614-c207-4ef7-b0f5-97ff717b3eb2    0.064953
Name: official xg, Length: 7468, dtype: float64
```

##Let us begin with the preprocessing pipeline for Machine Learning

First we define our categorical and numerical attributes in our data. We will use the following, 1. One Hot Encoder : For the categorical data 2. Standard Scaler : For numerical data 3. Column Transformer : To encode categorical columns and scale numerical columns

```
[ ]: cat_attributes = ['body part','Pass Type','play pattern','first time']
num_attributes = ['shot distance','shot_angle','Number of opponents in 5
↳yards', 'Players between goal','x location shot', 'y location shot','gk_
↳distance','x gk location','y gk location']
encoder = OneHotEncoder()
scaler = StandardScaler()
transformers = [('cat',encoder,cat_attributes),('num',scaler,num_attributes)]
main_pipeline = ColumnTransformer(transformers)
```

```
[ ]: processed_X_train = main_pipeline.fit_transform(train)
processed_X_test = main_pipeline.transform(validate)
```

```
[ ]: processed_X_train
```

```
[ ]: array([[ 1.          ,  0.          ,  1.          , ..., -0.35437032,
          -0.01124191, -0.2076158 ],
          [ 1.          ,  0.          ,  0.          , ..., -0.42773371,
           0.07343243,  0.27391948],
          [ 1.          ,  0.          ,  0.          , ..., -0.31768863,
           0.20044395, -0.58596495],
          ...,
          [ 1.          ,  0.          ,  1.          , ..., -0.86791405,
           0.58147851, -0.13882505],
          [ 1.          ,  0.          ,  1.          , ..., -0.13428015,
          -0.30760213,  0.20512872],
          [ 1.          ,  0.          ,  0.          , ..., -0.46441541,
           0.66615286,  0.75545476]])
```

```
[ ]: cat_transformed = main_pipeline.transformers[0][1].fit(X_train[cat_attributes],
↳y_train).get_feature_names_out(cat_attributes)
```

```
[ ]: all_features_transformed = list(num_attributes) + list(cat_transformed)
```

```
[ ]: len(all_features_transformed)
```

```
[ ]: 19
```

```
[ ]: classifiers = {
    'catboost' : (CatBoostClassifier(),[{"learning_rate": uniform(0,0.5),
↳"max_depth": np.arange(1,15), "n_estimators": np.arange(1, 250)}]),
    'lightgbm' : (LGBMClassifier(),[{"max_depth" : np.arange(1,
↳15),"num_leaves" : np.arange(0, 250),"learning_rate" : uniform(0, 0.5)}]),
    'xgboost' : (XGBClassifier(),[{"max_depth" : np.arange(1, 15),
↳"num_leaves" : np.arange(0, 250), "eta" : uniform(0, 1)}]),
    'logistic regression' : (LogisticRegression(max_iter=10000), [{"C" :
↳uniform(0, 20000)}])
}
```

```
[ ]: def hyperparameter_opt(classifier, hyperparameter_dist, processed_X_train,
    ↪y_train, scoring="neg_log_loss", n_iter=20):

    print("Optimizing hyperparameters for {} model...".format(type(classifier).
    ↪__name__))
    random_search = RandomizedSearchCV(estimator=classifier,
    ↪param_distributions=hyperparameter_dist, cv=10, verbose=0, scoring=scoring,
    ↪n_iter=n_iter)
    random_search.fit(processed_X_train, y_train)
    print("Best hyperparameters {0}".format(random_search.best_params_))
    print("Best cross validation score {0}\n".format(random_search.best_score_))

    return pd.DataFrame(random_search.cv_results_), random_search.
    ↪best_estimator_
```

```
[ ]: random_cv_results = {'catboost' : None,
    'lightgbm' : None,
    'xgboost' : None,
    'logistic regression' : None
    }
```

```
[ ]: def plot_roc_curve(y,y_model_proba,y_statsbomb_proba,model_label):
    fpr,tpr,thresholds = roc_curve(y,y_model_proba[:,1]) # extracting
    ↪probability for positive class
    fpr_sb,tpr_sb,thresholds_sb = roc_curve(y,y_statsbomb_proba) # statsbomb_xg
    ↪values
    plt.plot(fpr,tpr,label=model_label)
    plt.plot(fpr_sb,tpr_sb,label='statsbomb')
    plt.plot([0,1],[0,1])
    plt.ylim([0,1])
    plt.xlim([0,1])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.legend(loc='lower right');
```

```
[ ]: lr = LogisticRegression()
    lr.fit(processed_X_train,y_train)
```

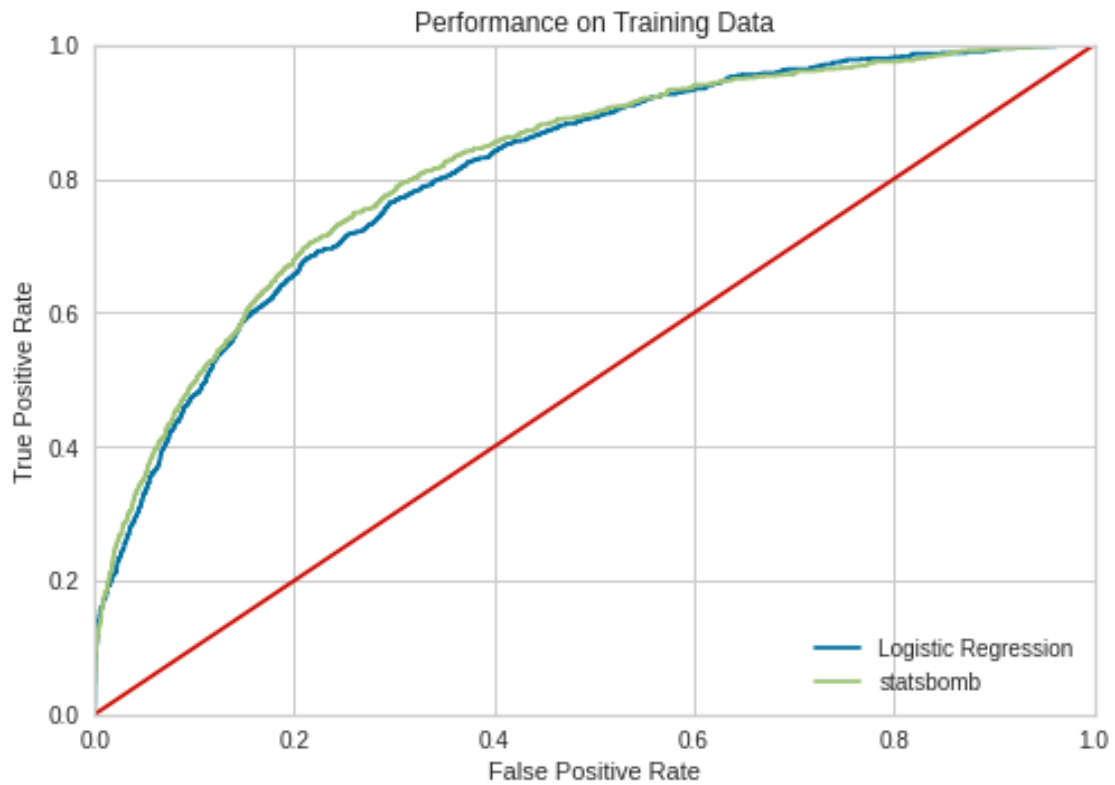
```
[ ]: LogisticRegression()
```

```
[ ]: y_train_pred = lr.predict(processed_X_train)
    y_train_proba = lr.predict_proba(processed_X_train)
```

```
[ ]: statsbomb_proba = train['official xg']
```

```
[ ]: plot_roc_curve(y_train,y_train_proba,statsbomb_proba,model_label='Logistic
    ↪Regression')
```

```
plt.title('Performance on Training Data');
```

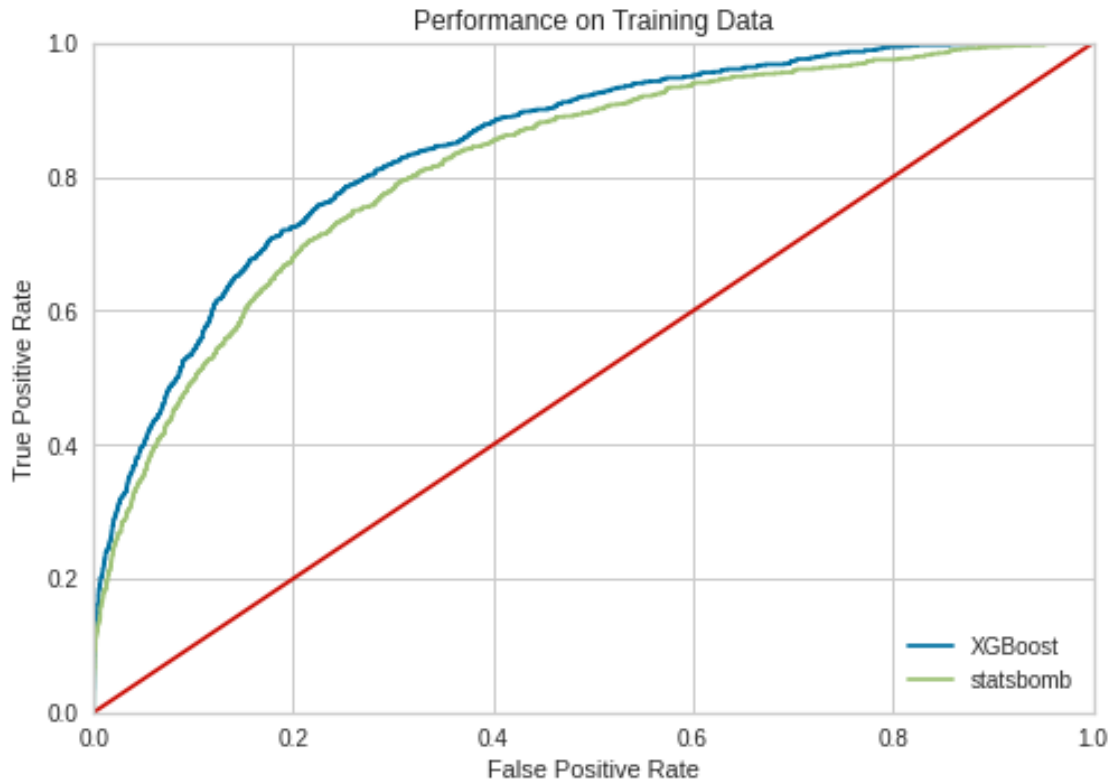


```
[ ]: xgb = XGBClassifier()  
xgb.fit(processed_X_train,y_train)
```

```
[ ]: XGBClassifier()
```

```
[ ]: y_train_pred = xgb.predict(processed_X_train)  
y_train_proba = xgb.predict_proba(processed_X_train)
```

```
[ ]: plot_roc_curve(y_train,y_train_proba,statsbomb_proba,model_label='XGBoost')  
plt.title('Performance on Training Data');
```



```
[ ]: print('The auc score for the XgBoost model on the training data is {}'.
        ↪format(round(roc_auc_score(y_train,y_train_proba[:,1]),2)))
print('The auc score for the Statsbomb model on the training data is {}'.
      ↪format(round(roc_auc_score(y_train,statsbomb_proba),2)))
```

The auc score for the XgBoost model on the training data is 0.84
 The auc score for the Statsbomb model on the training data is 0.82

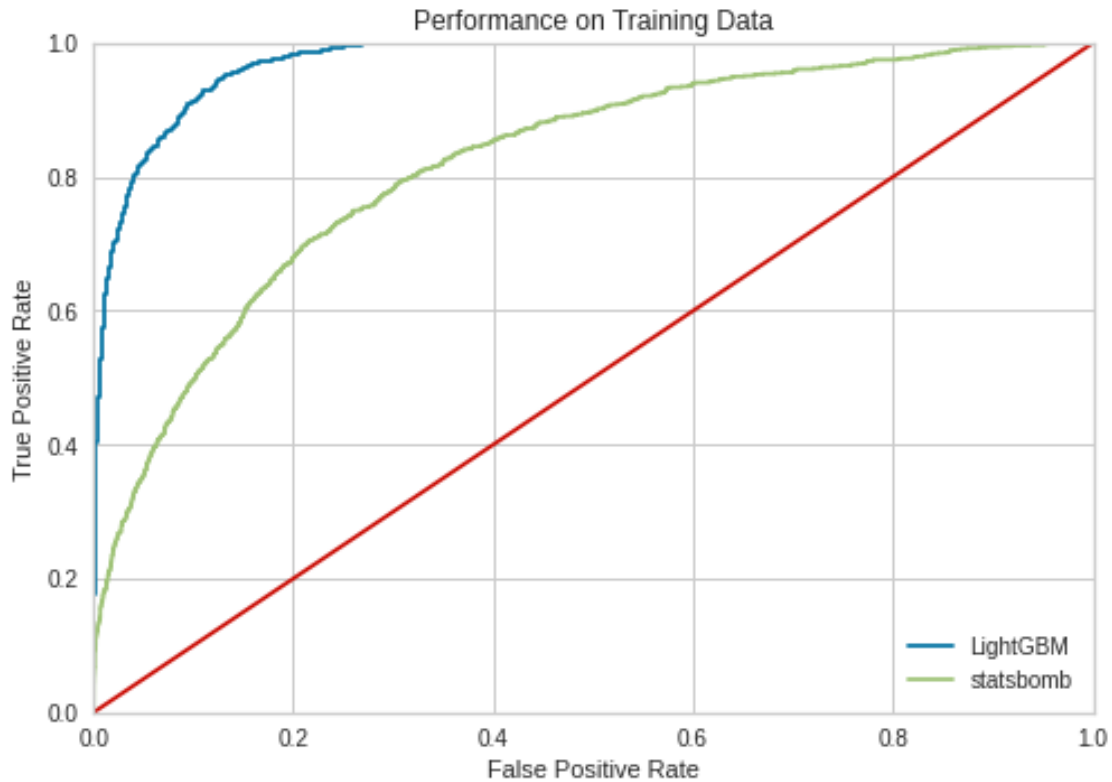
```
[ ]: X_test_preprocessed = main_pipeline.fit_transform(test)
```

```
[ ]: lgb = LGBMClassifier()
lgb.fit(processed_X_train,y_train)
```

```
[ ]: LGBMClassifier()
```

```
[ ]: y_train_pred = lgb.predict(processed_X_train)
y_train_proba = lgb.predict_proba(processed_X_train)
```

```
[ ]: plot_roc_curve(y_train,y_train_proba,statsbomb_proba,model_label='LightGBM')
plt.title('Performance on Training Data');
```



```
[ ]: print('The auc score for the LightGBM model on the training data is {}'.
        ↳format(round(roc_auc_score(y_train,y_train_proba[:,1]),2)))
print('The auc score for the Statsbomb model on the training data is {}'.
      ↳format(round(roc_auc_score(y_train,statsbomb_proba),2)))
```

The auc score for the LightGBM model on the training data is 0.97
The auc score for the Statsbomb model on the training data is 0.82

```
[ ]: cat = CatBoostClassifier()
cat.fit(processed_X_train,y_train)
```

Learning rate set to 0.02431

0:	learn: 0.6716891	total: 66.9ms	remaining: 1m 6s
1:	learn: 0.6511342	total: 72.3ms	remaining: 36.1s
2:	learn: 0.6312753	total: 81.8ms	remaining: 27.2s
3:	learn: 0.6121121	total: 94ms	remaining: 23.4s
4:	learn: 0.5957359	total: 98.7ms	remaining: 19.6s
5:	learn: 0.5791868	total: 107ms	remaining: 17.7s
6:	learn: 0.5640810	total: 115ms	remaining: 16.4s
7:	learn: 0.5499338	total: 123ms	remaining: 15.3s
8:	learn: 0.5356123	total: 132ms	remaining: 14.5s
9:	learn: 0.5224245	total: 140ms	remaining: 13.8s
10:	learn: 0.5100616	total: 148ms	remaining: 13.3s

11:	learn: 0.4983385	total: 153ms	remaining: 12.6s
12:	learn: 0.4878963	total: 157ms	remaining: 11.9s
13:	learn: 0.4780887	total: 165ms	remaining: 11.7s
14:	learn: 0.4687367	total: 170ms	remaining: 11.1s
15:	learn: 0.4595851	total: 174ms	remaining: 10.7s
16:	learn: 0.4519406	total: 180ms	remaining: 10.4s
17:	learn: 0.4445855	total: 185ms	remaining: 10.1s
18:	learn: 0.4378039	total: 189ms	remaining: 9.75s
19:	learn: 0.4313478	total: 193ms	remaining: 9.46s
20:	learn: 0.4255171	total: 197ms	remaining: 9.19s
21:	learn: 0.4195813	total: 201ms	remaining: 8.95s
22:	learn: 0.4139965	total: 205ms	remaining: 8.72s
23:	learn: 0.4096924	total: 209ms	remaining: 8.52s
24:	learn: 0.4049292	total: 214ms	remaining: 8.33s
25:	learn: 0.4003534	total: 217ms	remaining: 8.12s
26:	learn: 0.3952768	total: 221ms	remaining: 7.96s
27:	learn: 0.3910565	total: 225ms	remaining: 7.81s
28:	learn: 0.3870104	total: 229ms	remaining: 7.67s
29:	learn: 0.3830234	total: 233ms	remaining: 7.54s
30:	learn: 0.3789623	total: 237ms	remaining: 7.42s
31:	learn: 0.3754010	total: 242ms	remaining: 7.31s
32:	learn: 0.3719316	total: 246ms	remaining: 7.2s
33:	learn: 0.3685089	total: 250ms	remaining: 7.1s
34:	learn: 0.3657569	total: 255ms	remaining: 7.02s
35:	learn: 0.3630972	total: 260ms	remaining: 6.95s
36:	learn: 0.3602177	total: 264ms	remaining: 6.88s
37:	learn: 0.3571268	total: 269ms	remaining: 6.8s
38:	learn: 0.3548543	total: 273ms	remaining: 6.73s
39:	learn: 0.3526564	total: 277ms	remaining: 6.65s
40:	learn: 0.3506388	total: 281ms	remaining: 6.58s
41:	learn: 0.3487444	total: 287ms	remaining: 6.56s
42:	learn: 0.3466843	total: 295ms	remaining: 6.56s
43:	learn: 0.3449814	total: 299ms	remaining: 6.5s
44:	learn: 0.3431654	total: 303ms	remaining: 6.44s
45:	learn: 0.3410612	total: 308ms	remaining: 6.38s
46:	learn: 0.3396805	total: 313ms	remaining: 6.35s
47:	learn: 0.3378424	total: 318ms	remaining: 6.3s
48:	learn: 0.3365787	total: 322ms	remaining: 6.25s
49:	learn: 0.3352572	total: 326ms	remaining: 6.2s
50:	learn: 0.3339085	total: 331ms	remaining: 6.15s
51:	learn: 0.3327743	total: 333ms	remaining: 6.07s
52:	learn: 0.3316046	total: 337ms	remaining: 6.03s
53:	learn: 0.3305371	total: 342ms	remaining: 5.98s
54:	learn: 0.3293750	total: 346ms	remaining: 5.94s
55:	learn: 0.3282540	total: 350ms	remaining: 5.9s
56:	learn: 0.3272778	total: 355ms	remaining: 5.87s
57:	learn: 0.3261717	total: 362ms	remaining: 5.88s
58:	learn: 0.3251692	total: 367ms	remaining: 5.85s

59:	learn: 0.3242786	total: 371ms	remaining: 5.8s
60:	learn: 0.3234536	total: 375ms	remaining: 5.77s
61:	learn: 0.3227718	total: 379ms	remaining: 5.73s
62:	learn: 0.3220631	total: 383ms	remaining: 5.7s
63:	learn: 0.3213037	total: 387ms	remaining: 5.66s
64:	learn: 0.3206978	total: 392ms	remaining: 5.63s
65:	learn: 0.3198994	total: 396ms	remaining: 5.6s
66:	learn: 0.3194115	total: 400ms	remaining: 5.57s
67:	learn: 0.3188256	total: 404ms	remaining: 5.54s
68:	learn: 0.3181613	total: 408ms	remaining: 5.51s
69:	learn: 0.3172792	total: 412ms	remaining: 5.48s
70:	learn: 0.3165673	total: 417ms	remaining: 5.45s
71:	learn: 0.3159024	total: 421ms	remaining: 5.42s
72:	learn: 0.3154345	total: 425ms	remaining: 5.4s
73:	learn: 0.3149447	total: 430ms	remaining: 5.38s
74:	learn: 0.3144740	total: 434ms	remaining: 5.35s
75:	learn: 0.3139809	total: 438ms	remaining: 5.33s
76:	learn: 0.3135294	total: 443ms	remaining: 5.3s
77:	learn: 0.3129903	total: 447ms	remaining: 5.29s
78:	learn: 0.3125504	total: 451ms	remaining: 5.26s
79:	learn: 0.3122828	total: 456ms	remaining: 5.24s
80:	learn: 0.3118090	total: 461ms	remaining: 5.23s
81:	learn: 0.3112969	total: 465ms	remaining: 5.21s
82:	learn: 0.3108705	total: 470ms	remaining: 5.19s
83:	learn: 0.3106060	total: 475ms	remaining: 5.18s
84:	learn: 0.3101723	total: 479ms	remaining: 5.16s
85:	learn: 0.3097392	total: 484ms	remaining: 5.15s
86:	learn: 0.3094585	total: 489ms	remaining: 5.13s
87:	learn: 0.3091016	total: 494ms	remaining: 5.12s
88:	learn: 0.3087696	total: 498ms	remaining: 5.1s
89:	learn: 0.3084121	total: 503ms	remaining: 5.08s
90:	learn: 0.3081581	total: 507ms	remaining: 5.07s
91:	learn: 0.3078190	total: 512ms	remaining: 5.05s
92:	learn: 0.3074500	total: 516ms	remaining: 5.04s
93:	learn: 0.3071878	total: 521ms	remaining: 5.02s
94:	learn: 0.3067724	total: 525ms	remaining: 5s
95:	learn: 0.3064456	total: 529ms	remaining: 4.98s
96:	learn: 0.3061112	total: 533ms	remaining: 4.96s
97:	learn: 0.3058421	total: 537ms	remaining: 4.95s
98:	learn: 0.3055698	total: 542ms	remaining: 4.93s
99:	learn: 0.3052925	total: 547ms	remaining: 4.92s
100:	learn: 0.3050070	total: 551ms	remaining: 4.9s
101:	learn: 0.3047493	total: 555ms	remaining: 4.88s
102:	learn: 0.3044761	total: 559ms	remaining: 4.87s
103:	learn: 0.3042074	total: 565ms	remaining: 4.87s
104:	learn: 0.3039628	total: 571ms	remaining: 4.87s
105:	learn: 0.3037193	total: 577ms	remaining: 4.86s
106:	learn: 0.3035065	total: 581ms	remaining: 4.85s

107:	learn: 0.3032646	total: 585ms	remaining: 4.83s
108:	learn: 0.3029860	total: 590ms	remaining: 4.82s
109:	learn: 0.3027088	total: 594ms	remaining: 4.8s
110:	learn: 0.3025229	total: 598ms	remaining: 4.79s
111:	learn: 0.3023400	total: 602ms	remaining: 4.77s
112:	learn: 0.3020395	total: 607ms	remaining: 4.77s
113:	learn: 0.3018901	total: 615ms	remaining: 4.78s
114:	learn: 0.3017108	total: 620ms	remaining: 4.77s
115:	learn: 0.3015517	total: 627ms	remaining: 4.78s
116:	learn: 0.3012891	total: 631ms	remaining: 4.76s
117:	learn: 0.3010240	total: 635ms	remaining: 4.75s
118:	learn: 0.3007856	total: 640ms	remaining: 4.74s
119:	learn: 0.3004783	total: 644ms	remaining: 4.72s
120:	learn: 0.3002942	total: 648ms	remaining: 4.71s
121:	learn: 0.3000529	total: 652ms	remaining: 4.69s
122:	learn: 0.2998639	total: 656ms	remaining: 4.68s
123:	learn: 0.2996544	total: 661ms	remaining: 4.67s
124:	learn: 0.2994038	total: 665ms	remaining: 4.65s
125:	learn: 0.2992109	total: 669ms	remaining: 4.64s
126:	learn: 0.2989910	total: 673ms	remaining: 4.63s
127:	learn: 0.2987907	total: 677ms	remaining: 4.61s
128:	learn: 0.2986668	total: 682ms	remaining: 4.6s
129:	learn: 0.2984855	total: 691ms	remaining: 4.62s
130:	learn: 0.2983822	total: 704ms	remaining: 4.67s
131:	learn: 0.2982268	total: 709ms	remaining: 4.66s
132:	learn: 0.2980705	total: 713ms	remaining: 4.64s
133:	learn: 0.2979513	total: 717ms	remaining: 4.63s
134:	learn: 0.2978157	total: 722ms	remaining: 4.63s
135:	learn: 0.2976776	total: 726ms	remaining: 4.62s
136:	learn: 0.2975325	total: 731ms	remaining: 4.6s
137:	learn: 0.2973979	total: 736ms	remaining: 4.59s
138:	learn: 0.2972505	total: 740ms	remaining: 4.58s
139:	learn: 0.2970664	total: 745ms	remaining: 4.57s
140:	learn: 0.2969151	total: 749ms	remaining: 4.56s
141:	learn: 0.2967223	total: 753ms	remaining: 4.55s
142:	learn: 0.2964960	total: 757ms	remaining: 4.54s
143:	learn: 0.2963086	total: 761ms	remaining: 4.53s
144:	learn: 0.2961754	total: 766ms	remaining: 4.51s
145:	learn: 0.2959509	total: 772ms	remaining: 4.51s
146:	learn: 0.2958605	total: 776ms	remaining: 4.5s
147:	learn: 0.2957566	total: 781ms	remaining: 4.5s
148:	learn: 0.2956305	total: 786ms	remaining: 4.49s
149:	learn: 0.2954349	total: 790ms	remaining: 4.47s
150:	learn: 0.2953192	total: 793ms	remaining: 4.46s
151:	learn: 0.2950973	total: 798ms	remaining: 4.45s
152:	learn: 0.2948228	total: 802ms	remaining: 4.44s
153:	learn: 0.2946788	total: 806ms	remaining: 4.43s
154:	learn: 0.2945426	total: 810ms	remaining: 4.42s

155:	learn: 0.2944002	total: 814ms	remaining: 4.41s
156:	learn: 0.2942367	total: 819ms	remaining: 4.39s
157:	learn: 0.2941332	total: 823ms	remaining: 4.38s
158:	learn: 0.2940833	total: 826ms	remaining: 4.37s
159:	learn: 0.2939204	total: 830ms	remaining: 4.36s
160:	learn: 0.2937662	total: 836ms	remaining: 4.36s
161:	learn: 0.2936134	total: 840ms	remaining: 4.35s
162:	learn: 0.2935134	total: 844ms	remaining: 4.34s
163:	learn: 0.2933757	total: 849ms	remaining: 4.33s
164:	learn: 0.2932478	total: 853ms	remaining: 4.32s
165:	learn: 0.2931119	total: 857ms	remaining: 4.3s
166:	learn: 0.2929941	total: 861ms	remaining: 4.3s
167:	learn: 0.2928536	total: 866ms	remaining: 4.29s
168:	learn: 0.2926824	total: 870ms	remaining: 4.28s
169:	learn: 0.2924907	total: 879ms	remaining: 4.29s
170:	learn: 0.2923757	total: 884ms	remaining: 4.29s
171:	learn: 0.2923160	total: 889ms	remaining: 4.28s
172:	learn: 0.2921758	total: 893ms	remaining: 4.27s
173:	learn: 0.2921077	total: 897ms	remaining: 4.26s
174:	learn: 0.2920204	total: 901ms	remaining: 4.25s
175:	learn: 0.2918941	total: 905ms	remaining: 4.24s
176:	learn: 0.2917785	total: 909ms	remaining: 4.23s
177:	learn: 0.2916844	total: 913ms	remaining: 4.22s
178:	learn: 0.2915439	total: 923ms	remaining: 4.23s
179:	learn: 0.2914205	total: 927ms	remaining: 4.22s
180:	learn: 0.2913160	total: 932ms	remaining: 4.22s
181:	learn: 0.2911680	total: 941ms	remaining: 4.23s
182:	learn: 0.2908973	total: 946ms	remaining: 4.22s
183:	learn: 0.2908203	total: 950ms	remaining: 4.21s
184:	learn: 0.2907162	total: 954ms	remaining: 4.2s
185:	learn: 0.2905932	total: 958ms	remaining: 4.19s
186:	learn: 0.2904749	total: 962ms	remaining: 4.18s
187:	learn: 0.2903725	total: 967ms	remaining: 4.17s
188:	learn: 0.2902980	total: 971ms	remaining: 4.17s
189:	learn: 0.2902139	total: 975ms	remaining: 4.16s
190:	learn: 0.2901104	total: 979ms	remaining: 4.15s
191:	learn: 0.2900019	total: 983ms	remaining: 4.14s
192:	learn: 0.2899596	total: 987ms	remaining: 4.13s
193:	learn: 0.2897246	total: 992ms	remaining: 4.12s
194:	learn: 0.2895893	total: 996ms	remaining: 4.11s
195:	learn: 0.2893683	total: 1s	remaining: 4.1s
196:	learn: 0.2892634	total: 1s	remaining: 4.09s
197:	learn: 0.2891924	total: 1.01s	remaining: 4.09s
198:	learn: 0.2891307	total: 1.01s	remaining: 4.09s
199:	learn: 0.2889180	total: 1.02s	remaining: 4.08s
200:	learn: 0.2888433	total: 1.02s	remaining: 4.07s
201:	learn: 0.2887396	total: 1.03s	remaining: 4.06s
202:	learn: 0.2886670	total: 1.03s	remaining: 4.05s

203:	learn: 0.2884300	total: 1.04s	remaining: 4.04s
204:	learn: 0.2882284	total: 1.04s	remaining: 4.04s
205:	learn: 0.2880997	total: 1.04s	remaining: 4.03s
206:	learn: 0.2879996	total: 1.05s	remaining: 4.02s
207:	learn: 0.2879317	total: 1.05s	remaining: 4.01s
208:	learn: 0.2878652	total: 1.06s	remaining: 4.01s
209:	learn: 0.2877525	total: 1.06s	remaining: 4.01s
210:	learn: 0.2876066	total: 1.07s	remaining: 4s
211:	learn: 0.2874602	total: 1.07s	remaining: 3.99s
212:	learn: 0.2873380	total: 1.08s	remaining: 3.98s
213:	learn: 0.2872537	total: 1.08s	remaining: 3.97s
214:	learn: 0.2871813	total: 1.08s	remaining: 3.96s
215:	learn: 0.2871149	total: 1.09s	remaining: 3.96s
216:	learn: 0.2870579	total: 1.1s	remaining: 3.97s
217:	learn: 0.2869528	total: 1.1s	remaining: 3.96s
218:	learn: 0.2867641	total: 1.11s	remaining: 3.96s
219:	learn: 0.2866470	total: 1.11s	remaining: 3.95s
220:	learn: 0.2865309	total: 1.12s	remaining: 3.94s
221:	learn: 0.2864494	total: 1.12s	remaining: 3.93s
222:	learn: 0.2863645	total: 1.13s	remaining: 3.92s
223:	learn: 0.2862834	total: 1.13s	remaining: 3.92s
224:	learn: 0.2861191	total: 1.13s	remaining: 3.91s
225:	learn: 0.2860096	total: 1.14s	remaining: 3.9s
226:	learn: 0.2859487	total: 1.14s	remaining: 3.9s
227:	learn: 0.2858354	total: 1.15s	remaining: 3.89s
228:	learn: 0.2857531	total: 1.15s	remaining: 3.88s
229:	learn: 0.2856655	total: 1.16s	remaining: 3.87s
230:	learn: 0.2855867	total: 1.16s	remaining: 3.86s
231:	learn: 0.2855323	total: 1.16s	remaining: 3.85s
232:	learn: 0.2854454	total: 1.17s	remaining: 3.85s
233:	learn: 0.2853827	total: 1.17s	remaining: 3.84s
234:	learn: 0.2853399	total: 1.18s	remaining: 3.83s
235:	learn: 0.2851575	total: 1.18s	remaining: 3.82s
236:	learn: 0.2850541	total: 1.19s	remaining: 3.81s
237:	learn: 0.2849581	total: 1.19s	remaining: 3.81s
238:	learn: 0.2848588	total: 1.19s	remaining: 3.8s
239:	learn: 0.2847010	total: 1.2s	remaining: 3.79s
240:	learn: 0.2845826	total: 1.2s	remaining: 3.78s
241:	learn: 0.2845169	total: 1.21s	remaining: 3.77s
242:	learn: 0.2844772	total: 1.21s	remaining: 3.77s
243:	learn: 0.2843860	total: 1.22s	remaining: 3.77s
244:	learn: 0.2842622	total: 1.22s	remaining: 3.76s
245:	learn: 0.2841525	total: 1.23s	remaining: 3.76s
246:	learn: 0.2840772	total: 1.23s	remaining: 3.75s
247:	learn: 0.2840018	total: 1.23s	remaining: 3.74s
248:	learn: 0.2839145	total: 1.24s	remaining: 3.73s
249:	learn: 0.2838493	total: 1.24s	remaining: 3.73s
250:	learn: 0.2837293	total: 1.25s	remaining: 3.72s

251:	learn: 0.2836975	total: 1.26s	remaining: 3.74s
252:	learn: 0.2835808	total: 1.26s	remaining: 3.73s
253:	learn: 0.2833869	total: 1.27s	remaining: 3.72s
254:	learn: 0.2832679	total: 1.27s	remaining: 3.71s
255:	learn: 0.2831602	total: 1.27s	remaining: 3.71s
256:	learn: 0.2831158	total: 1.28s	remaining: 3.7s
257:	learn: 0.2830148	total: 1.28s	remaining: 3.69s
258:	learn: 0.2828964	total: 1.29s	remaining: 3.68s
259:	learn: 0.2827724	total: 1.29s	remaining: 3.68s
260:	learn: 0.2826092	total: 1.3s	remaining: 3.67s
261:	learn: 0.2825169	total: 1.3s	remaining: 3.66s
262:	learn: 0.2825068	total: 1.3s	remaining: 3.65s
263:	learn: 0.2823437	total: 1.31s	remaining: 3.65s
264:	learn: 0.2822723	total: 1.31s	remaining: 3.64s
265:	learn: 0.2822283	total: 1.32s	remaining: 3.63s
266:	learn: 0.2821279	total: 1.32s	remaining: 3.62s
267:	learn: 0.2820321	total: 1.32s	remaining: 3.62s
268:	learn: 0.2819805	total: 1.33s	remaining: 3.61s
269:	learn: 0.2819027	total: 1.33s	remaining: 3.6s
270:	learn: 0.2817813	total: 1.34s	remaining: 3.6s
271:	learn: 0.2816787	total: 1.34s	remaining: 3.59s
272:	learn: 0.2816012	total: 1.35s	remaining: 3.58s
273:	learn: 0.2815913	total: 1.35s	remaining: 3.58s
274:	learn: 0.2814843	total: 1.35s	remaining: 3.57s
275:	learn: 0.2813363	total: 1.36s	remaining: 3.57s
276:	learn: 0.2812837	total: 1.36s	remaining: 3.56s
277:	learn: 0.2812109	total: 1.37s	remaining: 3.55s
278:	learn: 0.2811211	total: 1.37s	remaining: 3.55s
279:	learn: 0.2810401	total: 1.38s	remaining: 3.54s
280:	learn: 0.2808487	total: 1.38s	remaining: 3.54s
281:	learn: 0.2807766	total: 1.39s	remaining: 3.53s
282:	learn: 0.2807093	total: 1.39s	remaining: 3.52s
283:	learn: 0.2804914	total: 1.39s	remaining: 3.52s
284:	learn: 0.2804028	total: 1.4s	remaining: 3.51s
285:	learn: 0.2803458	total: 1.4s	remaining: 3.5s
286:	learn: 0.2802583	total: 1.41s	remaining: 3.5s
287:	learn: 0.2801276	total: 1.41s	remaining: 3.49s
288:	learn: 0.2800534	total: 1.41s	remaining: 3.48s
289:	learn: 0.2799695	total: 1.42s	remaining: 3.47s
290:	learn: 0.2798810	total: 1.43s	remaining: 3.48s
291:	learn: 0.2797832	total: 1.43s	remaining: 3.47s
292:	learn: 0.2796558	total: 1.44s	remaining: 3.47s
293:	learn: 0.2795945	total: 1.44s	remaining: 3.46s
294:	learn: 0.2795202	total: 1.45s	remaining: 3.45s
295:	learn: 0.2794293	total: 1.45s	remaining: 3.45s
296:	learn: 0.2793809	total: 1.45s	remaining: 3.44s
297:	learn: 0.2793387	total: 1.46s	remaining: 3.44s
298:	learn: 0.2793005	total: 1.47s	remaining: 3.45s

299:	learn: 0.2791566	total: 1.48s	remaining: 3.45s
300:	learn: 0.2790807	total: 1.48s	remaining: 3.44s
301:	learn: 0.2789431	total: 1.49s	remaining: 3.43s
302:	learn: 0.2788047	total: 1.49s	remaining: 3.43s
303:	learn: 0.2786151	total: 1.49s	remaining: 3.42s
304:	learn: 0.2785693	total: 1.5s	remaining: 3.41s
305:	learn: 0.2784898	total: 1.5s	remaining: 3.41s
306:	learn: 0.2783937	total: 1.51s	remaining: 3.4s
307:	learn: 0.2782893	total: 1.51s	remaining: 3.39s
308:	learn: 0.2782514	total: 1.51s	remaining: 3.38s
309:	learn: 0.2781282	total: 1.52s	remaining: 3.38s
310:	learn: 0.2780981	total: 1.52s	remaining: 3.37s
311:	learn: 0.2779761	total: 1.53s	remaining: 3.37s
312:	learn: 0.2779209	total: 1.53s	remaining: 3.36s
313:	learn: 0.2777680	total: 1.53s	remaining: 3.35s
314:	learn: 0.2776253	total: 1.54s	remaining: 3.35s
315:	learn: 0.2775151	total: 1.54s	remaining: 3.34s
316:	learn: 0.2774829	total: 1.55s	remaining: 3.33s
317:	learn: 0.2773785	total: 1.55s	remaining: 3.33s
318:	learn: 0.2772631	total: 1.55s	remaining: 3.32s
319:	learn: 0.2771501	total: 1.56s	remaining: 3.31s
320:	learn: 0.2770916	total: 1.56s	remaining: 3.31s
321:	learn: 0.2770459	total: 1.57s	remaining: 3.31s
322:	learn: 0.2769531	total: 1.58s	remaining: 3.31s
323:	learn: 0.2768020	total: 1.58s	remaining: 3.3s
324:	learn: 0.2767142	total: 1.58s	remaining: 3.29s
325:	learn: 0.2766926	total: 1.59s	remaining: 3.29s
326:	learn: 0.2766399	total: 1.59s	remaining: 3.28s
327:	learn: 0.2765410	total: 1.6s	remaining: 3.27s
328:	learn: 0.2763519	total: 1.6s	remaining: 3.27s
329:	learn: 0.2762532	total: 1.6s	remaining: 3.26s
330:	learn: 0.2762137	total: 1.61s	remaining: 3.25s
331:	learn: 0.2761432	total: 1.61s	remaining: 3.25s
332:	learn: 0.2760289	total: 1.62s	remaining: 3.24s
333:	learn: 0.2759334	total: 1.62s	remaining: 3.23s
334:	learn: 0.2758379	total: 1.63s	remaining: 3.23s
335:	learn: 0.2757152	total: 1.63s	remaining: 3.22s
336:	learn: 0.2755666	total: 1.64s	remaining: 3.22s
337:	learn: 0.2755002	total: 1.64s	remaining: 3.21s
338:	learn: 0.2754050	total: 1.64s	remaining: 3.21s
339:	learn: 0.2753514	total: 1.65s	remaining: 3.2s
340:	learn: 0.2752574	total: 1.65s	remaining: 3.19s
341:	learn: 0.2751516	total: 1.66s	remaining: 3.19s
342:	learn: 0.2750929	total: 1.66s	remaining: 3.18s
343:	learn: 0.2750206	total: 1.67s	remaining: 3.17s
344:	learn: 0.2748901	total: 1.67s	remaining: 3.17s
345:	learn: 0.2747709	total: 1.67s	remaining: 3.16s
346:	learn: 0.2746374	total: 1.68s	remaining: 3.16s

347:	learn: 0.2745663	total: 1.68s	remaining: 3.15s
348:	learn: 0.2744456	total: 1.69s	remaining: 3.14s
349:	learn: 0.2742668	total: 1.69s	remaining: 3.14s
350:	learn: 0.2741899	total: 1.69s	remaining: 3.13s
351:	learn: 0.2741470	total: 1.7s	remaining: 3.13s
352:	learn: 0.2740850	total: 1.71s	remaining: 3.13s
353:	learn: 0.2739253	total: 1.71s	remaining: 3.12s
354:	learn: 0.2738744	total: 1.71s	remaining: 3.11s
355:	learn: 0.2738365	total: 1.72s	remaining: 3.11s
356:	learn: 0.2737249	total: 1.73s	remaining: 3.11s
357:	learn: 0.2736461	total: 1.73s	remaining: 3.1s
358:	learn: 0.2735561	total: 1.74s	remaining: 3.1s
359:	learn: 0.2735050	total: 1.74s	remaining: 3.09s
360:	learn: 0.2732989	total: 1.74s	remaining: 3.08s
361:	learn: 0.2732255	total: 1.75s	remaining: 3.08s
362:	learn: 0.2732005	total: 1.75s	remaining: 3.07s
363:	learn: 0.2731355	total: 1.75s	remaining: 3.07s
364:	learn: 0.2730965	total: 1.76s	remaining: 3.06s
365:	learn: 0.2730186	total: 1.76s	remaining: 3.06s
366:	learn: 0.2728931	total: 1.77s	remaining: 3.05s
367:	learn: 0.2728400	total: 1.77s	remaining: 3.04s
368:	learn: 0.2727351	total: 1.78s	remaining: 3.04s
369:	learn: 0.2726118	total: 1.78s	remaining: 3.03s
370:	learn: 0.2725260	total: 1.79s	remaining: 3.03s
371:	learn: 0.2724132	total: 1.79s	remaining: 3.03s
372:	learn: 0.2722923	total: 1.8s	remaining: 3.02s
373:	learn: 0.2721751	total: 1.8s	remaining: 3.02s
374:	learn: 0.2720819	total: 1.81s	remaining: 3.01s
375:	learn: 0.2720245	total: 1.81s	remaining: 3.01s
376:	learn: 0.2719490	total: 1.82s	remaining: 3s
377:	learn: 0.2718320	total: 1.82s	remaining: 3s
378:	learn: 0.2716546	total: 1.83s	remaining: 2.99s
379:	learn: 0.2715959	total: 1.83s	remaining: 2.99s
380:	learn: 0.2714922	total: 1.83s	remaining: 2.98s
381:	learn: 0.2713620	total: 1.84s	remaining: 2.98s
382:	learn: 0.2711762	total: 1.84s	remaining: 2.97s
383:	learn: 0.2710903	total: 1.85s	remaining: 2.96s
384:	learn: 0.2709811	total: 1.85s	remaining: 2.96s
385:	learn: 0.2708683	total: 1.86s	remaining: 2.95s
386:	learn: 0.2707950	total: 1.86s	remaining: 2.95s
387:	learn: 0.2707607	total: 1.86s	remaining: 2.94s
388:	learn: 0.2706512	total: 1.87s	remaining: 2.93s
389:	learn: 0.2705690	total: 1.87s	remaining: 2.93s
390:	learn: 0.2704074	total: 1.88s	remaining: 2.92s
391:	learn: 0.2702712	total: 1.88s	remaining: 2.92s
392:	learn: 0.2701901	total: 1.89s	remaining: 2.91s
393:	learn: 0.2700823	total: 1.89s	remaining: 2.91s
394:	learn: 0.2699994	total: 1.9s	remaining: 2.9s

395:	learn: 0.2699307	total: 1.9s	remaining: 2.9s
396:	learn: 0.2698784	total: 1.9s	remaining: 2.89s
397:	learn: 0.2698244	total: 1.91s	remaining: 2.89s
398:	learn: 0.2697642	total: 1.91s	remaining: 2.88s
399:	learn: 0.2695883	total: 1.92s	remaining: 2.88s
400:	learn: 0.2694965	total: 1.92s	remaining: 2.87s
401:	learn: 0.2694558	total: 1.93s	remaining: 2.86s
402:	learn: 0.2693862	total: 1.93s	remaining: 2.86s
403:	learn: 0.2692279	total: 1.93s	remaining: 2.85s
404:	learn: 0.2691589	total: 1.94s	remaining: 2.85s
405:	learn: 0.2690430	total: 1.94s	remaining: 2.84s
406:	learn: 0.2689284	total: 1.95s	remaining: 2.84s
407:	learn: 0.2688488	total: 1.95s	remaining: 2.83s
408:	learn: 0.2687389	total: 1.96s	remaining: 2.83s
409:	learn: 0.2686538	total: 1.96s	remaining: 2.82s
410:	learn: 0.2684985	total: 1.97s	remaining: 2.82s
411:	learn: 0.2684620	total: 1.97s	remaining: 2.81s
412:	learn: 0.2683312	total: 1.98s	remaining: 2.81s
413:	learn: 0.2682117	total: 1.98s	remaining: 2.8s
414:	learn: 0.2680596	total: 1.98s	remaining: 2.8s
415:	learn: 0.2679850	total: 1.99s	remaining: 2.79s
416:	learn: 0.2679142	total: 1.99s	remaining: 2.78s
417:	learn: 0.2678328	total: 2s	remaining: 2.78s
418:	learn: 0.2677617	total: 2s	remaining: 2.77s
419:	learn: 0.2676223	total: 2s	remaining: 2.77s
420:	learn: 0.2675644	total: 2.01s	remaining: 2.76s
421:	learn: 0.2674405	total: 2.01s	remaining: 2.75s
422:	learn: 0.2673193	total: 2.02s	remaining: 2.75s
423:	learn: 0.2672078	total: 2.02s	remaining: 2.74s
424:	learn: 0.2671753	total: 2.02s	remaining: 2.74s
425:	learn: 0.2670390	total: 2.04s	remaining: 2.75s
426:	learn: 0.2669521	total: 2.04s	remaining: 2.74s
427:	learn: 0.2667978	total: 2.05s	remaining: 2.73s
428:	learn: 0.2666656	total: 2.05s	remaining: 2.73s
429:	learn: 0.2664674	total: 2.05s	remaining: 2.72s
430:	learn: 0.2663146	total: 2.06s	remaining: 2.72s
431:	learn: 0.2661979	total: 2.06s	remaining: 2.71s
432:	learn: 0.2660461	total: 2.07s	remaining: 2.71s
433:	learn: 0.2658627	total: 2.07s	remaining: 2.7s
434:	learn: 0.2656930	total: 2.08s	remaining: 2.69s
435:	learn: 0.2654501	total: 2.08s	remaining: 2.69s
436:	learn: 0.2652762	total: 2.08s	remaining: 2.68s
437:	learn: 0.2651556	total: 2.09s	remaining: 2.68s
438:	learn: 0.2649373	total: 2.09s	remaining: 2.67s
439:	learn: 0.2647703	total: 2.1s	remaining: 2.67s
440:	learn: 0.2647357	total: 2.1s	remaining: 2.66s
441:	learn: 0.2645782	total: 2.1s	remaining: 2.66s
442:	learn: 0.2643660	total: 2.11s	remaining: 2.65s

443:	learn: 0.2642309	total: 2.11s	remaining: 2.65s
444:	learn: 0.2640945	total: 2.12s	remaining: 2.64s
445:	learn: 0.2639866	total: 2.12s	remaining: 2.63s
446:	learn: 0.2639108	total: 2.12s	remaining: 2.63s
447:	learn: 0.2637714	total: 2.13s	remaining: 2.62s
448:	learn: 0.2637159	total: 2.13s	remaining: 2.62s
449:	learn: 0.2636516	total: 2.14s	remaining: 2.61s
450:	learn: 0.2635214	total: 2.14s	remaining: 2.6s
451:	learn: 0.2633970	total: 2.14s	remaining: 2.6s
452:	learn: 0.2632751	total: 2.15s	remaining: 2.59s
453:	learn: 0.2632419	total: 2.15s	remaining: 2.59s
454:	learn: 0.2630705	total: 2.16s	remaining: 2.58s
455:	learn: 0.2630413	total: 2.16s	remaining: 2.58s
456:	learn: 0.2629229	total: 2.17s	remaining: 2.57s
457:	learn: 0.2628388	total: 2.17s	remaining: 2.57s
458:	learn: 0.2627783	total: 2.17s	remaining: 2.56s
459:	learn: 0.2626977	total: 2.18s	remaining: 2.56s
460:	learn: 0.2625855	total: 2.18s	remaining: 2.55s
461:	learn: 0.2625216	total: 2.19s	remaining: 2.55s
462:	learn: 0.2624613	total: 2.19s	remaining: 2.54s
463:	learn: 0.2623461	total: 2.2s	remaining: 2.54s
464:	learn: 0.2622401	total: 2.2s	remaining: 2.53s
465:	learn: 0.2621519	total: 2.2s	remaining: 2.53s
466:	learn: 0.2620593	total: 2.21s	remaining: 2.52s
467:	learn: 0.2618895	total: 2.21s	remaining: 2.52s
468:	learn: 0.2618123	total: 2.23s	remaining: 2.52s
469:	learn: 0.2616784	total: 2.24s	remaining: 2.53s
470:	learn: 0.2615874	total: 2.25s	remaining: 2.52s
471:	learn: 0.2615093	total: 2.25s	remaining: 2.52s
472:	learn: 0.2614731	total: 2.25s	remaining: 2.51s
473:	learn: 0.2614158	total: 2.26s	remaining: 2.5s
474:	learn: 0.2612984	total: 2.26s	remaining: 2.5s
475:	learn: 0.2612583	total: 2.27s	remaining: 2.5s
476:	learn: 0.2611957	total: 2.27s	remaining: 2.49s
477:	learn: 0.2610584	total: 2.28s	remaining: 2.49s
478:	learn: 0.2609592	total: 2.28s	remaining: 2.48s
479:	learn: 0.2608191	total: 2.29s	remaining: 2.48s
480:	learn: 0.2606573	total: 2.29s	remaining: 2.47s
481:	learn: 0.2606021	total: 2.29s	remaining: 2.46s
482:	learn: 0.2604594	total: 2.3s	remaining: 2.46s
483:	learn: 0.2603066	total: 2.3s	remaining: 2.45s
484:	learn: 0.2602765	total: 2.31s	remaining: 2.45s
485:	learn: 0.2601793	total: 2.31s	remaining: 2.44s
486:	learn: 0.2601188	total: 2.31s	remaining: 2.44s
487:	learn: 0.2600707	total: 2.32s	remaining: 2.43s
488:	learn: 0.2599435	total: 2.32s	remaining: 2.43s
489:	learn: 0.2598334	total: 2.33s	remaining: 2.42s
490:	learn: 0.2597085	total: 2.33s	remaining: 2.42s

491:	learn: 0.2595649	total: 2.33s	remaining: 2.41s
492:	learn: 0.2594491	total: 2.34s	remaining: 2.4s
493:	learn: 0.2592716	total: 2.34s	remaining: 2.4s
494:	learn: 0.2591422	total: 2.35s	remaining: 2.4s
495:	learn: 0.2590207	total: 2.36s	remaining: 2.4s
496:	learn: 0.2588486	total: 2.36s	remaining: 2.39s
497:	learn: 0.2587153	total: 2.37s	remaining: 2.38s
498:	learn: 0.2586292	total: 2.37s	remaining: 2.38s
499:	learn: 0.2584772	total: 2.37s	remaining: 2.37s
500:	learn: 0.2583363	total: 2.38s	remaining: 2.37s
501:	learn: 0.2582178	total: 2.38s	remaining: 2.36s
502:	learn: 0.2581067	total: 2.39s	remaining: 2.36s
503:	learn: 0.2580181	total: 2.39s	remaining: 2.35s
504:	learn: 0.2579388	total: 2.4s	remaining: 2.35s
505:	learn: 0.2579136	total: 2.4s	remaining: 2.34s
506:	learn: 0.2577579	total: 2.4s	remaining: 2.34s
507:	learn: 0.2576471	total: 2.41s	remaining: 2.33s
508:	learn: 0.2574927	total: 2.41s	remaining: 2.33s
509:	learn: 0.2574618	total: 2.42s	remaining: 2.32s
510:	learn: 0.2573351	total: 2.42s	remaining: 2.32s
511:	learn: 0.2571768	total: 2.42s	remaining: 2.31s
512:	learn: 0.2571231	total: 2.43s	remaining: 2.31s
513:	learn: 0.2569989	total: 2.43s	remaining: 2.3s
514:	learn: 0.2568289	total: 2.44s	remaining: 2.29s
515:	learn: 0.2567832	total: 2.44s	remaining: 2.29s
516:	learn: 0.2566498	total: 2.44s	remaining: 2.29s
517:	learn: 0.2565397	total: 2.45s	remaining: 2.28s
518:	learn: 0.2563869	total: 2.45s	remaining: 2.27s
519:	learn: 0.2563722	total: 2.46s	remaining: 2.27s
520:	learn: 0.2562482	total: 2.46s	remaining: 2.26s
521:	learn: 0.2561258	total: 2.47s	remaining: 2.26s
522:	learn: 0.2560855	total: 2.47s	remaining: 2.25s
523:	learn: 0.2558933	total: 2.47s	remaining: 2.25s
524:	learn: 0.2558259	total: 2.48s	remaining: 2.24s
525:	learn: 0.2557118	total: 2.48s	remaining: 2.24s
526:	learn: 0.2556921	total: 2.49s	remaining: 2.23s
527:	learn: 0.2555119	total: 2.49s	remaining: 2.23s
528:	learn: 0.2554155	total: 2.5s	remaining: 2.22s
529:	learn: 0.2553309	total: 2.5s	remaining: 2.22s
530:	learn: 0.2552121	total: 2.51s	remaining: 2.22s
531:	learn: 0.2551631	total: 2.52s	remaining: 2.21s
532:	learn: 0.2550403	total: 2.52s	remaining: 2.21s
533:	learn: 0.2549383	total: 2.52s	remaining: 2.2s
534:	learn: 0.2548254	total: 2.53s	remaining: 2.2s
535:	learn: 0.2546976	total: 2.53s	remaining: 2.19s
536:	learn: 0.2546020	total: 2.54s	remaining: 2.19s
537:	learn: 0.2544997	total: 2.54s	remaining: 2.18s
538:	learn: 0.2543631	total: 2.55s	remaining: 2.18s

539:	learn: 0.2542762	total: 2.55s	remaining: 2.17s
540:	learn: 0.2541843	total: 2.56s	remaining: 2.17s
541:	learn: 0.2541158	total: 2.56s	remaining: 2.16s
542:	learn: 0.2539751	total: 2.56s	remaining: 2.16s
543:	learn: 0.2538931	total: 2.57s	remaining: 2.15s
544:	learn: 0.2538210	total: 2.57s	remaining: 2.15s
545:	learn: 0.2537727	total: 2.58s	remaining: 2.14s
546:	learn: 0.2537369	total: 2.58s	remaining: 2.14s
547:	learn: 0.2535797	total: 2.58s	remaining: 2.13s
548:	learn: 0.2534644	total: 2.59s	remaining: 2.13s
549:	learn: 0.2534076	total: 2.59s	remaining: 2.12s
550:	learn: 0.2532643	total: 2.6s	remaining: 2.12s
551:	learn: 0.2531687	total: 2.6s	remaining: 2.11s
552:	learn: 0.2530565	total: 2.6s	remaining: 2.1s
553:	learn: 0.2529316	total: 2.61s	remaining: 2.1s
554:	learn: 0.2528134	total: 2.62s	remaining: 2.1s
555:	learn: 0.2526678	total: 2.62s	remaining: 2.09s
556:	learn: 0.2525662	total: 2.63s	remaining: 2.09s
557:	learn: 0.2524557	total: 2.63s	remaining: 2.08s
558:	learn: 0.2523753	total: 2.63s	remaining: 2.08s
559:	learn: 0.2522612	total: 2.64s	remaining: 2.07s
560:	learn: 0.2521518	total: 2.64s	remaining: 2.07s
561:	learn: 0.2520387	total: 2.65s	remaining: 2.06s
562:	learn: 0.2519318	total: 2.65s	remaining: 2.06s
563:	learn: 0.2518438	total: 2.66s	remaining: 2.05s
564:	learn: 0.2517494	total: 2.66s	remaining: 2.05s
565:	learn: 0.2516364	total: 2.67s	remaining: 2.04s
566:	learn: 0.2516076	total: 2.67s	remaining: 2.04s
567:	learn: 0.2514604	total: 2.67s	remaining: 2.03s
568:	learn: 0.2513344	total: 2.68s	remaining: 2.03s
569:	learn: 0.2512447	total: 2.68s	remaining: 2.02s
570:	learn: 0.2511355	total: 2.69s	remaining: 2.02s
571:	learn: 0.2510138	total: 2.69s	remaining: 2.02s
572:	learn: 0.2509874	total: 2.7s	remaining: 2.01s
573:	learn: 0.2509091	total: 2.7s	remaining: 2.01s
574:	learn: 0.2507517	total: 2.71s	remaining: 2s
575:	learn: 0.2506609	total: 2.71s	remaining: 2s
576:	learn: 0.2505612	total: 2.71s	remaining: 1.99s
577:	learn: 0.2504912	total: 2.72s	remaining: 1.99s
578:	learn: 0.2504030	total: 2.72s	remaining: 1.98s
579:	learn: 0.2503360	total: 2.73s	remaining: 1.98s
580:	learn: 0.2502963	total: 2.73s	remaining: 1.97s
581:	learn: 0.2501950	total: 2.73s	remaining: 1.96s
582:	learn: 0.2501056	total: 2.74s	remaining: 1.96s
583:	learn: 0.2500449	total: 2.74s	remaining: 1.95s
584:	learn: 0.2499642	total: 2.75s	remaining: 1.95s
585:	learn: 0.2499285	total: 2.75s	remaining: 1.94s
586:	learn: 0.2498328	total: 2.76s	remaining: 1.94s

587:	learn: 0.2497572	total: 2.76s	remaining: 1.94s
588:	learn: 0.2496486	total: 2.77s	remaining: 1.93s
589:	learn: 0.2495774	total: 2.77s	remaining: 1.93s
590:	learn: 0.2494260	total: 2.77s	remaining: 1.92s
591:	learn: 0.2493090	total: 2.78s	remaining: 1.91s
592:	learn: 0.2492339	total: 2.78s	remaining: 1.91s
593:	learn: 0.2491031	total: 2.79s	remaining: 1.9s
594:	learn: 0.2489703	total: 2.79s	remaining: 1.9s
595:	learn: 0.2489494	total: 2.79s	remaining: 1.89s
596:	learn: 0.2488737	total: 2.8s	remaining: 1.89s
597:	learn: 0.2487797	total: 2.8s	remaining: 1.88s
598:	learn: 0.2486515	total: 2.81s	remaining: 1.88s
599:	learn: 0.2485264	total: 2.81s	remaining: 1.87s
600:	learn: 0.2483911	total: 2.82s	remaining: 1.87s
601:	learn: 0.2483578	total: 2.82s	remaining: 1.86s
602:	learn: 0.2482694	total: 2.83s	remaining: 1.86s
603:	learn: 0.2481642	total: 2.83s	remaining: 1.85s
604:	learn: 0.2480681	total: 2.84s	remaining: 1.85s
605:	learn: 0.2479818	total: 2.84s	remaining: 1.85s
606:	learn: 0.2478457	total: 2.84s	remaining: 1.84s
607:	learn: 0.2477257	total: 2.85s	remaining: 1.84s
608:	learn: 0.2476057	total: 2.85s	remaining: 1.83s
609:	learn: 0.2475002	total: 2.86s	remaining: 1.83s
610:	learn: 0.2473594	total: 2.86s	remaining: 1.82s
611:	learn: 0.2472655	total: 2.86s	remaining: 1.82s
612:	learn: 0.2472352	total: 2.87s	remaining: 1.81s
613:	learn: 0.2470995	total: 2.87s	remaining: 1.81s
614:	learn: 0.2470226	total: 2.88s	remaining: 1.8s
615:	learn: 0.2469332	total: 2.88s	remaining: 1.8s
616:	learn: 0.2468310	total: 2.88s	remaining: 1.79s
617:	learn: 0.2467131	total: 2.89s	remaining: 1.79s
618:	learn: 0.2466010	total: 2.89s	remaining: 1.78s
619:	learn: 0.2465214	total: 2.9s	remaining: 1.78s
620:	learn: 0.2464173	total: 2.9s	remaining: 1.77s
621:	learn: 0.2463697	total: 2.91s	remaining: 1.77s
622:	learn: 0.2462781	total: 2.91s	remaining: 1.76s
623:	learn: 0.2462583	total: 2.91s	remaining: 1.75s
624:	learn: 0.2461716	total: 2.92s	remaining: 1.75s
625:	learn: 0.2460558	total: 2.92s	remaining: 1.75s
626:	learn: 0.2459192	total: 2.93s	remaining: 1.74s
627:	learn: 0.2458228	total: 2.93s	remaining: 1.74s
628:	learn: 0.2457577	total: 2.93s	remaining: 1.73s
629:	learn: 0.2456975	total: 2.94s	remaining: 1.73s
630:	learn: 0.2456117	total: 2.94s	remaining: 1.72s
631:	learn: 0.2454485	total: 2.95s	remaining: 1.72s
632:	learn: 0.2453455	total: 2.95s	remaining: 1.71s
633:	learn: 0.2452995	total: 2.96s	remaining: 1.71s
634:	learn: 0.2452020	total: 2.96s	remaining: 1.7s

635:	learn: 0.2451390	total: 2.96s	remaining: 1.7s
636:	learn: 0.2450561	total: 2.97s	remaining: 1.69s
637:	learn: 0.2449980	total: 2.97s	remaining: 1.69s
638:	learn: 0.2449352	total: 2.98s	remaining: 1.68s
639:	learn: 0.2448566	total: 2.98s	remaining: 1.68s
640:	learn: 0.2447609	total: 2.99s	remaining: 1.67s
641:	learn: 0.2446418	total: 2.99s	remaining: 1.67s
642:	learn: 0.2445493	total: 3s	remaining: 1.67s
643:	learn: 0.2444040	total: 3s	remaining: 1.66s
644:	learn: 0.2443408	total: 3.01s	remaining: 1.66s
645:	learn: 0.2442866	total: 3.01s	remaining: 1.65s
646:	learn: 0.2441773	total: 3.02s	remaining: 1.65s
647:	learn: 0.2441172	total: 3.02s	remaining: 1.64s
648:	learn: 0.2439909	total: 3.03s	remaining: 1.64s
649:	learn: 0.2438922	total: 3.03s	remaining: 1.63s
650:	learn: 0.2437894	total: 3.04s	remaining: 1.63s
651:	learn: 0.2436862	total: 3.04s	remaining: 1.62s
652:	learn: 0.2435952	total: 3.04s	remaining: 1.62s
653:	learn: 0.2435773	total: 3.05s	remaining: 1.61s
654:	learn: 0.2434924	total: 3.05s	remaining: 1.61s
655:	learn: 0.2433427	total: 3.06s	remaining: 1.6s
656:	learn: 0.2432354	total: 3.06s	remaining: 1.6s
657:	learn: 0.2431162	total: 3.07s	remaining: 1.59s
658:	learn: 0.2429768	total: 3.07s	remaining: 1.59s
659:	learn: 0.2429455	total: 3.08s	remaining: 1.58s
660:	learn: 0.2427771	total: 3.08s	remaining: 1.58s
661:	learn: 0.2426691	total: 3.08s	remaining: 1.57s
662:	learn: 0.2425780	total: 3.09s	remaining: 1.57s
663:	learn: 0.2425021	total: 3.09s	remaining: 1.56s
664:	learn: 0.2424653	total: 3.1s	remaining: 1.56s
665:	learn: 0.2424031	total: 3.1s	remaining: 1.55s
666:	learn: 0.2423234	total: 3.1s	remaining: 1.55s
667:	learn: 0.2422012	total: 3.11s	remaining: 1.54s
668:	learn: 0.2420848	total: 3.11s	remaining: 1.54s
669:	learn: 0.2420074	total: 3.12s	remaining: 1.54s
670:	learn: 0.2419515	total: 3.12s	remaining: 1.53s
671:	learn: 0.2418760	total: 3.13s	remaining: 1.53s
672:	learn: 0.2417668	total: 3.14s	remaining: 1.52s
673:	learn: 0.2416849	total: 3.14s	remaining: 1.52s
674:	learn: 0.2415432	total: 3.15s	remaining: 1.51s
675:	learn: 0.2415173	total: 3.15s	remaining: 1.51s
676:	learn: 0.2413821	total: 3.15s	remaining: 1.5s
677:	learn: 0.2412997	total: 3.16s	remaining: 1.5s
678:	learn: 0.2412044	total: 3.16s	remaining: 1.5s
679:	learn: 0.2410694	total: 3.17s	remaining: 1.49s
680:	learn: 0.2410109	total: 3.17s	remaining: 1.49s
681:	learn: 0.2408687	total: 3.18s	remaining: 1.48s
682:	learn: 0.2407379	total: 3.18s	remaining: 1.48s

683:	learn: 0.2406855	total: 3.19s	remaining: 1.47s
684:	learn: 0.2406559	total: 3.19s	remaining: 1.47s
685:	learn: 0.2405346	total: 3.19s	remaining: 1.46s
686:	learn: 0.2404110	total: 3.2s	remaining: 1.46s
687:	learn: 0.2403673	total: 3.2s	remaining: 1.45s
688:	learn: 0.2402493	total: 3.21s	remaining: 1.45s
689:	learn: 0.2401259	total: 3.21s	remaining: 1.44s
690:	learn: 0.2400241	total: 3.22s	remaining: 1.44s
691:	learn: 0.2399267	total: 3.22s	remaining: 1.43s
692:	learn: 0.2398717	total: 3.23s	remaining: 1.43s
693:	learn: 0.2397821	total: 3.23s	remaining: 1.42s
694:	learn: 0.2397106	total: 3.23s	remaining: 1.42s
695:	learn: 0.2396256	total: 3.24s	remaining: 1.41s
696:	learn: 0.2395480	total: 3.24s	remaining: 1.41s
697:	learn: 0.2395243	total: 3.25s	remaining: 1.4s
698:	learn: 0.2394139	total: 3.25s	remaining: 1.4s
699:	learn: 0.2393281	total: 3.26s	remaining: 1.4s
700:	learn: 0.2392202	total: 3.26s	remaining: 1.39s
701:	learn: 0.2390804	total: 3.27s	remaining: 1.39s
702:	learn: 0.2389484	total: 3.27s	remaining: 1.38s
703:	learn: 0.2388676	total: 3.28s	remaining: 1.38s
704:	learn: 0.2387399	total: 3.29s	remaining: 1.38s
705:	learn: 0.2385966	total: 3.29s	remaining: 1.37s
706:	learn: 0.2384617	total: 3.3s	remaining: 1.37s
707:	learn: 0.2383553	total: 3.3s	remaining: 1.36s
708:	learn: 0.2382304	total: 3.31s	remaining: 1.36s
709:	learn: 0.2381370	total: 3.31s	remaining: 1.35s
710:	learn: 0.2380171	total: 3.32s	remaining: 1.35s
711:	learn: 0.2379382	total: 3.33s	remaining: 1.35s
712:	learn: 0.2378635	total: 3.34s	remaining: 1.34s
713:	learn: 0.2377428	total: 3.35s	remaining: 1.34s
714:	learn: 0.2376204	total: 3.35s	remaining: 1.34s
715:	learn: 0.2375711	total: 3.35s	remaining: 1.33s
716:	learn: 0.2375494	total: 3.36s	remaining: 1.33s
717:	learn: 0.2374141	total: 3.36s	remaining: 1.32s
718:	learn: 0.2372852	total: 3.37s	remaining: 1.32s
719:	learn: 0.2372129	total: 3.37s	remaining: 1.31s
720:	learn: 0.2371005	total: 3.38s	remaining: 1.31s
721:	learn: 0.2369503	total: 3.38s	remaining: 1.3s
722:	learn: 0.2368564	total: 3.38s	remaining: 1.3s
723:	learn: 0.2367261	total: 3.39s	remaining: 1.29s
724:	learn: 0.2365850	total: 3.39s	remaining: 1.29s
725:	learn: 0.2364697	total: 3.4s	remaining: 1.28s
726:	learn: 0.2363892	total: 3.4s	remaining: 1.28s
727:	learn: 0.2362563	total: 3.41s	remaining: 1.27s
728:	learn: 0.2362085	total: 3.41s	remaining: 1.27s
729:	learn: 0.2361217	total: 3.42s	remaining: 1.26s
730:	learn: 0.2359972	total: 3.42s	remaining: 1.26s

731:	learn: 0.2359289	total: 3.42s	remaining: 1.25s
732:	learn: 0.2358250	total: 3.43s	remaining: 1.25s
733:	learn: 0.2357710	total: 3.43s	remaining: 1.24s
734:	learn: 0.2356860	total: 3.44s	remaining: 1.24s
735:	learn: 0.2356300	total: 3.44s	remaining: 1.24s
736:	learn: 0.2355316	total: 3.45s	remaining: 1.23s
737:	learn: 0.2354117	total: 3.46s	remaining: 1.23s
738:	learn: 0.2353325	total: 3.46s	remaining: 1.22s
739:	learn: 0.2352845	total: 3.46s	remaining: 1.22s
740:	learn: 0.2352398	total: 3.47s	remaining: 1.21s
741:	learn: 0.2351484	total: 3.47s	remaining: 1.21s
742:	learn: 0.2350426	total: 3.48s	remaining: 1.2s
743:	learn: 0.2349611	total: 3.48s	remaining: 1.2s
744:	learn: 0.2348327	total: 3.49s	remaining: 1.19s
745:	learn: 0.2347441	total: 3.49s	remaining: 1.19s
746:	learn: 0.2347286	total: 3.5s	remaining: 1.18s
747:	learn: 0.2345647	total: 3.5s	remaining: 1.18s
748:	learn: 0.2345143	total: 3.5s	remaining: 1.17s
749:	learn: 0.2344305	total: 3.51s	remaining: 1.17s
750:	learn: 0.2343510	total: 3.51s	remaining: 1.16s
751:	learn: 0.2342496	total: 3.52s	remaining: 1.16s
752:	learn: 0.2341667	total: 3.52s	remaining: 1.15s
753:	learn: 0.2341519	total: 3.52s	remaining: 1.15s
754:	learn: 0.2340957	total: 3.53s	remaining: 1.15s
755:	learn: 0.2339937	total: 3.53s	remaining: 1.14s
756:	learn: 0.2339005	total: 3.54s	remaining: 1.14s
757:	learn: 0.2338483	total: 3.54s	remaining: 1.13s
758:	learn: 0.2337836	total: 3.54s	remaining: 1.13s
759:	learn: 0.2336981	total: 3.55s	remaining: 1.12s
760:	learn: 0.2336181	total: 3.55s	remaining: 1.12s
761:	learn: 0.2335623	total: 3.56s	remaining: 1.11s
762:	learn: 0.2334250	total: 3.56s	remaining: 1.11s
763:	learn: 0.2333797	total: 3.56s	remaining: 1.1s
764:	learn: 0.2333449	total: 3.57s	remaining: 1.1s
765:	learn: 0.2332691	total: 3.57s	remaining: 1.09s
766:	learn: 0.2332262	total: 3.58s	remaining: 1.09s
767:	learn: 0.2331237	total: 3.58s	remaining: 1.08s
768:	learn: 0.2330779	total: 3.58s	remaining: 1.08s
769:	learn: 0.2329714	total: 3.59s	remaining: 1.07s
770:	learn: 0.2329265	total: 3.59s	remaining: 1.07s
771:	learn: 0.2328485	total: 3.6s	remaining: 1.06s
772:	learn: 0.2327295	total: 3.6s	remaining: 1.06s
773:	learn: 0.2326102	total: 3.61s	remaining: 1.05s
774:	learn: 0.2324657	total: 3.61s	remaining: 1.05s
775:	learn: 0.2324207	total: 3.62s	remaining: 1.04s
776:	learn: 0.2323687	total: 3.62s	remaining: 1.04s
777:	learn: 0.2323516	total: 3.63s	remaining: 1.03s
778:	learn: 0.2322661	total: 3.63s	remaining: 1.03s

779:	learn: 0.2321246	total: 3.64s	remaining: 1.03s
780:	learn: 0.2320130	total: 3.64s	remaining: 1.02s
781:	learn: 0.2318948	total: 3.65s	remaining: 1.02s
782:	learn: 0.2317394	total: 3.65s	remaining: 1.01s
783:	learn: 0.2316499	total: 3.65s	remaining: 1.01s
784:	learn: 0.2315425	total: 3.66s	remaining: 1s
785:	learn: 0.2315178	total: 3.66s	remaining: 998ms
786:	learn: 0.2314773	total: 3.67s	remaining: 993ms
787:	learn: 0.2314492	total: 3.67s	remaining: 988ms
788:	learn: 0.2313765	total: 3.68s	remaining: 983ms
789:	learn: 0.2313438	total: 3.68s	remaining: 978ms
790:	learn: 0.2312243	total: 3.69s	remaining: 974ms
791:	learn: 0.2310777	total: 3.69s	remaining: 969ms
792:	learn: 0.2310086	total: 3.69s	remaining: 964ms
793:	learn: 0.2309024	total: 3.7s	remaining: 960ms
794:	learn: 0.2308074	total: 3.7s	remaining: 955ms
795:	learn: 0.2307357	total: 3.71s	remaining: 950ms
796:	learn: 0.2306515	total: 3.71s	remaining: 945ms
797:	learn: 0.2305806	total: 3.71s	remaining: 941ms
798:	learn: 0.2304887	total: 3.72s	remaining: 936ms
799:	learn: 0.2304000	total: 3.72s	remaining: 931ms
800:	learn: 0.2302765	total: 3.73s	remaining: 926ms
801:	learn: 0.2301975	total: 3.73s	remaining: 921ms
802:	learn: 0.2300683	total: 3.74s	remaining: 917ms
803:	learn: 0.2300344	total: 3.74s	remaining: 912ms
804:	learn: 0.2300014	total: 3.74s	remaining: 907ms
805:	learn: 0.2299461	total: 3.75s	remaining: 902ms
806:	learn: 0.2298999	total: 3.77s	remaining: 903ms
807:	learn: 0.2298304	total: 3.78s	remaining: 898ms
808:	learn: 0.2297642	total: 3.79s	remaining: 894ms
809:	learn: 0.2297124	total: 3.79s	remaining: 889ms
810:	learn: 0.2296123	total: 3.79s	remaining: 884ms
811:	learn: 0.2295455	total: 3.8s	remaining: 879ms
812:	learn: 0.2294806	total: 3.8s	remaining: 875ms
813:	learn: 0.2293903	total: 3.81s	remaining: 870ms
814:	learn: 0.2292722	total: 3.81s	remaining: 866ms
815:	learn: 0.2292611	total: 3.82s	remaining: 861ms
816:	learn: 0.2291938	total: 3.82s	remaining: 856ms
817:	learn: 0.2291205	total: 3.83s	remaining: 852ms
818:	learn: 0.2290163	total: 3.83s	remaining: 847ms
819:	learn: 0.2289142	total: 3.84s	remaining: 842ms
820:	learn: 0.2288674	total: 3.84s	remaining: 838ms
821:	learn: 0.2288285	total: 3.85s	remaining: 833ms
822:	learn: 0.2286940	total: 3.85s	remaining: 828ms
823:	learn: 0.2286592	total: 3.86s	remaining: 824ms
824:	learn: 0.2285675	total: 3.86s	remaining: 819ms
825:	learn: 0.2284709	total: 3.86s	remaining: 814ms
826:	learn: 0.2283489	total: 3.87s	remaining: 809ms

827:	learn: 0.2282423	total: 3.87s	remaining: 804ms
828:	learn: 0.2281286	total: 3.88s	remaining: 800ms
829:	learn: 0.2280057	total: 3.88s	remaining: 795ms
830:	learn: 0.2279388	total: 3.88s	remaining: 790ms
831:	learn: 0.2278292	total: 3.89s	remaining: 785ms
832:	learn: 0.2277392	total: 3.89s	remaining: 781ms
833:	learn: 0.2276643	total: 3.9s	remaining: 776ms
834:	learn: 0.2275149	total: 3.9s	remaining: 771ms
835:	learn: 0.2274418	total: 3.9s	remaining: 766ms
836:	learn: 0.2273198	total: 3.91s	remaining: 762ms
837:	learn: 0.2272520	total: 3.92s	remaining: 757ms
838:	learn: 0.2271805	total: 3.92s	remaining: 752ms
839:	learn: 0.2271323	total: 3.92s	remaining: 747ms
840:	learn: 0.2270378	total: 3.93s	remaining: 744ms
841:	learn: 0.2269771	total: 3.94s	remaining: 739ms
842:	learn: 0.2268770	total: 3.94s	remaining: 734ms
843:	learn: 0.2268460	total: 3.94s	remaining: 729ms
844:	learn: 0.2268012	total: 3.95s	remaining: 724ms
845:	learn: 0.2267403	total: 3.95s	remaining: 720ms
846:	learn: 0.2266653	total: 3.96s	remaining: 715ms
847:	learn: 0.2266478	total: 3.96s	remaining: 710ms
848:	learn: 0.2265855	total: 3.96s	remaining: 705ms
849:	learn: 0.2265180	total: 3.97s	remaining: 700ms
850:	learn: 0.2263816	total: 3.97s	remaining: 696ms
851:	learn: 0.2262752	total: 3.98s	remaining: 691ms
852:	learn: 0.2262345	total: 3.98s	remaining: 686ms
853:	learn: 0.2261541	total: 3.99s	remaining: 681ms
854:	learn: 0.2260413	total: 3.99s	remaining: 677ms
855:	learn: 0.2259880	total: 3.99s	remaining: 672ms
856:	learn: 0.2258904	total: 4s	remaining: 667ms
857:	learn: 0.2257969	total: 4s	remaining: 663ms
858:	learn: 0.2257152	total: 4.01s	remaining: 658ms
859:	learn: 0.2256633	total: 4.01s	remaining: 653ms
860:	learn: 0.2256213	total: 4.02s	remaining: 649ms
861:	learn: 0.2255740	total: 4.02s	remaining: 644ms
862:	learn: 0.2255433	total: 4.03s	remaining: 639ms
863:	learn: 0.2254787	total: 4.03s	remaining: 635ms
864:	learn: 0.2254510	total: 4.04s	remaining: 630ms
865:	learn: 0.2253249	total: 4.04s	remaining: 625ms
866:	learn: 0.2252986	total: 4.04s	remaining: 620ms
867:	learn: 0.2252437	total: 4.05s	remaining: 616ms
868:	learn: 0.2251908	total: 4.05s	remaining: 611ms
869:	learn: 0.2251028	total: 4.06s	remaining: 606ms
870:	learn: 0.2249859	total: 4.06s	remaining: 601ms
871:	learn: 0.2249586	total: 4.07s	remaining: 597ms
872:	learn: 0.2248448	total: 4.07s	remaining: 592ms
873:	learn: 0.2247639	total: 4.07s	remaining: 587ms
874:	learn: 0.2247198	total: 4.08s	remaining: 583ms

875:	learn: 0.2245896	total: 4.09s	remaining: 579ms
876:	learn: 0.2245419	total: 4.1s	remaining: 575ms
877:	learn: 0.2244641	total: 4.1s	remaining: 570ms
878:	learn: 0.2243776	total: 4.11s	remaining: 565ms
879:	learn: 0.2242690	total: 4.11s	remaining: 561ms
880:	learn: 0.2241784	total: 4.11s	remaining: 556ms
881:	learn: 0.2241008	total: 4.12s	remaining: 551ms
882:	learn: 0.2240589	total: 4.12s	remaining: 546ms
883:	learn: 0.2240356	total: 4.13s	remaining: 542ms
884:	learn: 0.2240064	total: 4.13s	remaining: 537ms
885:	learn: 0.2239609	total: 4.13s	remaining: 532ms
886:	learn: 0.2238799	total: 4.14s	remaining: 527ms
887:	learn: 0.2238683	total: 4.14s	remaining: 523ms
888:	learn: 0.2237816	total: 4.15s	remaining: 518ms
889:	learn: 0.2237327	total: 4.15s	remaining: 513ms
890:	learn: 0.2236911	total: 4.16s	remaining: 508ms
891:	learn: 0.2236429	total: 4.16s	remaining: 504ms
892:	learn: 0.2235339	total: 4.17s	remaining: 500ms
893:	learn: 0.2235146	total: 4.17s	remaining: 495ms
894:	learn: 0.2234429	total: 4.18s	remaining: 490ms
895:	learn: 0.2233881	total: 4.18s	remaining: 485ms
896:	learn: 0.2233103	total: 4.18s	remaining: 481ms
897:	learn: 0.2232425	total: 4.19s	remaining: 476ms
898:	learn: 0.2231495	total: 4.19s	remaining: 471ms
899:	learn: 0.2230450	total: 4.2s	remaining: 466ms
900:	learn: 0.2229459	total: 4.2s	remaining: 462ms
901:	learn: 0.2228508	total: 4.21s	remaining: 457ms
902:	learn: 0.2227118	total: 4.21s	remaining: 452ms
903:	learn: 0.2226266	total: 4.21s	remaining: 448ms
904:	learn: 0.2224978	total: 4.22s	remaining: 443ms
905:	learn: 0.2223745	total: 4.22s	remaining: 438ms
906:	learn: 0.2223209	total: 4.23s	remaining: 433ms
907:	learn: 0.2222321	total: 4.23s	remaining: 429ms
908:	learn: 0.2222101	total: 4.24s	remaining: 424ms
909:	learn: 0.2221871	total: 4.24s	remaining: 419ms
910:	learn: 0.2220574	total: 4.25s	remaining: 415ms
911:	learn: 0.2220366	total: 4.25s	remaining: 411ms
912:	learn: 0.2219995	total: 4.26s	remaining: 406ms
913:	learn: 0.2219674	total: 4.26s	remaining: 401ms
914:	learn: 0.2218841	total: 4.27s	remaining: 397ms
915:	learn: 0.2218368	total: 4.28s	remaining: 392ms
916:	learn: 0.2217445	total: 4.28s	remaining: 387ms
917:	learn: 0.2217052	total: 4.28s	remaining: 383ms
918:	learn: 0.2215675	total: 4.29s	remaining: 378ms
919:	learn: 0.2214576	total: 4.29s	remaining: 374ms
920:	learn: 0.2213452	total: 4.3s	remaining: 369ms
921:	learn: 0.2213285	total: 4.3s	remaining: 364ms
922:	learn: 0.2211607	total: 4.31s	remaining: 359ms

923:	learn: 0.2210734	total: 4.31s	remaining: 355ms
924:	learn: 0.2209818	total: 4.32s	remaining: 350ms
925:	learn: 0.2209414	total: 4.32s	remaining: 345ms
926:	learn: 0.2208060	total: 4.33s	remaining: 341ms
927:	learn: 0.2207501	total: 4.33s	remaining: 336ms
928:	learn: 0.2206743	total: 4.33s	remaining: 331ms
929:	learn: 0.2206217	total: 4.34s	remaining: 327ms
930:	learn: 0.2205869	total: 4.34s	remaining: 322ms
931:	learn: 0.2205290	total: 4.35s	remaining: 317ms
932:	learn: 0.2203779	total: 4.35s	remaining: 313ms
933:	learn: 0.2203355	total: 4.36s	remaining: 308ms
934:	learn: 0.2202550	total: 4.36s	remaining: 303ms
935:	learn: 0.2202032	total: 4.37s	remaining: 299ms
936:	learn: 0.2201523	total: 4.37s	remaining: 294ms
937:	learn: 0.2200008	total: 4.38s	remaining: 289ms
938:	learn: 0.2199491	total: 4.38s	remaining: 284ms
939:	learn: 0.2198851	total: 4.38s	remaining: 280ms
940:	learn: 0.2198068	total: 4.39s	remaining: 275ms
941:	learn: 0.2197600	total: 4.39s	remaining: 270ms
942:	learn: 0.2197209	total: 4.39s	remaining: 266ms
943:	learn: 0.2196689	total: 4.4s	remaining: 261ms
944:	learn: 0.2195412	total: 4.4s	remaining: 256ms
945:	learn: 0.2194804	total: 4.41s	remaining: 252ms
946:	learn: 0.2193977	total: 4.42s	remaining: 247ms
947:	learn: 0.2193597	total: 4.42s	remaining: 243ms
948:	learn: 0.2192361	total: 4.42s	remaining: 238ms
949:	learn: 0.2191516	total: 4.43s	remaining: 233ms
950:	learn: 0.2191203	total: 4.44s	remaining: 229ms
951:	learn: 0.2190650	total: 4.44s	remaining: 224ms
952:	learn: 0.2189925	total: 4.45s	remaining: 219ms
953:	learn: 0.2189326	total: 4.45s	remaining: 215ms
954:	learn: 0.2189098	total: 4.45s	remaining: 210ms
955:	learn: 0.2188143	total: 4.46s	remaining: 205ms
956:	learn: 0.2187722	total: 4.46s	remaining: 201ms
957:	learn: 0.2187059	total: 4.47s	remaining: 196ms
958:	learn: 0.2186228	total: 4.47s	remaining: 191ms
959:	learn: 0.2185408	total: 4.47s	remaining: 186ms
960:	learn: 0.2185256	total: 4.48s	remaining: 182ms
961:	learn: 0.2184887	total: 4.48s	remaining: 177ms
962:	learn: 0.2184347	total: 4.49s	remaining: 172ms
963:	learn: 0.2182894	total: 4.49s	remaining: 168ms
964:	learn: 0.2182457	total: 4.5s	remaining: 163ms
965:	learn: 0.2181341	total: 4.5s	remaining: 158ms
966:	learn: 0.2180216	total: 4.51s	remaining: 154ms
967:	learn: 0.2179816	total: 4.51s	remaining: 149ms
968:	learn: 0.2178842	total: 4.52s	remaining: 145ms
969:	learn: 0.2177958	total: 4.52s	remaining: 140ms
970:	learn: 0.2176845	total: 4.53s	remaining: 135ms

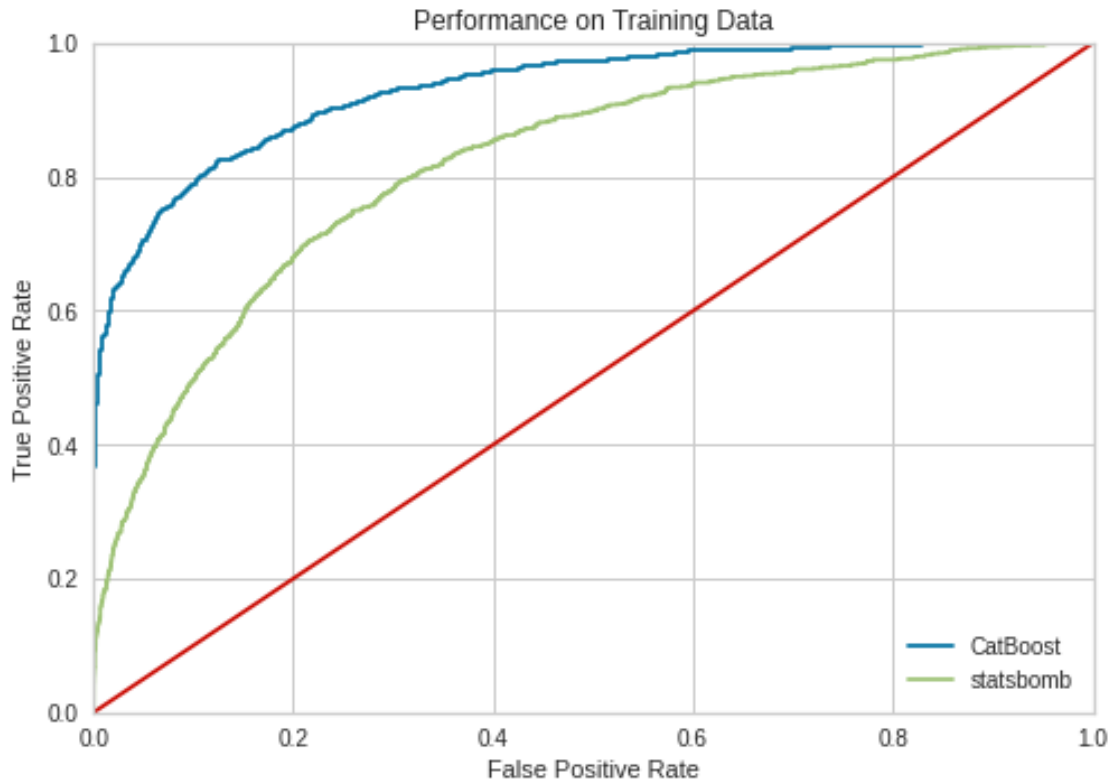
971:	learn: 0.2175973	total: 4.54s	remaining: 131ms
972:	learn: 0.2175230	total: 4.54s	remaining: 126ms
973:	learn: 0.2174739	total: 4.54s	remaining: 121ms
974:	learn: 0.2173947	total: 4.55s	remaining: 117ms
975:	learn: 0.2173313	total: 4.56s	remaining: 112ms
976:	learn: 0.2172316	total: 4.57s	remaining: 107ms
977:	learn: 0.2171191	total: 4.57s	remaining: 103ms
978:	learn: 0.2170583	total: 4.57s	remaining: 98.1ms
979:	learn: 0.2170034	total: 4.58s	remaining: 93.4ms
980:	learn: 0.2168997	total: 4.58s	remaining: 88.8ms
981:	learn: 0.2168097	total: 4.59s	remaining: 84.1ms
982:	learn: 0.2167471	total: 4.59s	remaining: 79.4ms
983:	learn: 0.2166978	total: 4.59s	remaining: 74.7ms
984:	learn: 0.2166272	total: 4.6s	remaining: 70ms
985:	learn: 0.2165737	total: 4.6s	remaining: 65.4ms
986:	learn: 0.2165453	total: 4.61s	remaining: 60.7ms
987:	learn: 0.2164605	total: 4.61s	remaining: 56ms
988:	learn: 0.2164166	total: 4.62s	remaining: 51.3ms
989:	learn: 0.2163155	total: 4.62s	remaining: 46.7ms
990:	learn: 0.2162293	total: 4.63s	remaining: 42ms
991:	learn: 0.2161710	total: 4.63s	remaining: 37.3ms
992:	learn: 0.2160824	total: 4.63s	remaining: 32.7ms
993:	learn: 0.2160382	total: 4.64s	remaining: 28ms
994:	learn: 0.2159285	total: 4.64s	remaining: 23.3ms
995:	learn: 0.2158509	total: 4.65s	remaining: 18.7ms
996:	learn: 0.2157704	total: 4.65s	remaining: 14ms
997:	learn: 0.2156741	total: 4.66s	remaining: 9.33ms
998:	learn: 0.2156002	total: 4.66s	remaining: 4.66ms
999:	learn: 0.2155051	total: 4.66s	remaining: 0us

```
[ ]: <catboost.core.CatBoostClassifier at 0x7fb388fc3e10>
```

```
[ ]: y_train_pred = cat.predict(processed_X_train)
     y_train_proba = cat.predict_proba(processed_X_train)
```

```
[ ]: plot_roc_curve(y_train,y_train_proba,statsbomb_proba,model_label='CatBoost')
     plt.title('Performance on Training Data')
```

```
[ ]: Text(0.5, 1.0, 'Performance on Training Data')
```



```
[ ]: print('The auc score for the CatBoost model on the training data is {}'.
        ↳format(round(roc_auc_score(y_train,y_train_proba[:,1]),2)))
print('The auc score for the Statsbomb model on the training data is {}'.
      ↳format(round(roc_auc_score(y_train,statsbomb_proba),2)))
```

The auc score for the CatBoost model on the training data is 0.93
The auc score for the Statsbomb model on the training data is 0.82

```
[ ]: for classifier_name, classifier_obj in classifiers.items():
      results = hyperparameter_opt(classifier=classifier_obj[0],
                                   hyperparameter_dist=classifier_obj[1],
                                   processed_X_train=processed_X_train,
                                   y_train=y_train,
                                   scoring="neg_log_loss",
                                   n_iter=5)
      random_cv_results[classifier_name] = results
```

```
[ ]: import sklearn
sklearn.metrics.SCORERS.keys()
```

```
[ ]: dict_keys(['explained_variance', 'r2', 'max_error', 'neg_median_absolute_error',
               'neg_mean_absolute_error', 'neg_mean_absolute_percentage_error',
```

```
'neg_mean_squared_error', 'neg_mean_squared_log_error',
'neg_root_mean_squared_error', 'neg_mean_poisson_deviance',
'neg_mean_gamma_deviance', 'accuracy', 'top_k_accuracy', 'roc_auc',
'roc_auc_ovr', 'roc_auc_ovo', 'roc_auc_ovr_weighted', 'roc_auc_ovo_weighted',
'balanced_accuracy', 'average_precision', 'neg_log_loss', 'neg_brier_score',
'adjusted_rand_score', 'rand_score', 'homogeneity_score', 'completeness_score',
'v_measure_score', 'mutual_info_score', 'adjusted_mutual_info_score',
'normalized_mutual_info_score', 'fowlkes_mallows_score', 'precision',
'precision_macro', 'precision_micro', 'precision_samples', 'precision_weighted',
'recall', 'recall_macro', 'recall_micro', 'recall_samples', 'recall_weighted',
'f1', 'f1_macro', 'f1_micro', 'f1_samples', 'f1_weighted', 'jaccard',
'jaccard_macro', 'jaccard_micro', 'jaccard_samples', 'jaccard_weighted']])
```

```
[ ]: from sklearn.model_selection import validation_curve
validation_set_predictions = {'lightgbm' : None, 'xgboost' : None, 'logistic_
    ↪regression' : None, 'catboost' : None }

for classifier_name, classifier_obj in random_cv_results.items():
    try:
        classifier = random_cv_results[classifier_name][1]
        y_train_pred = classifier.predict_proba(processed_X_train)
        y_valid_pred = classifier.predict_proba(processed_X_test)

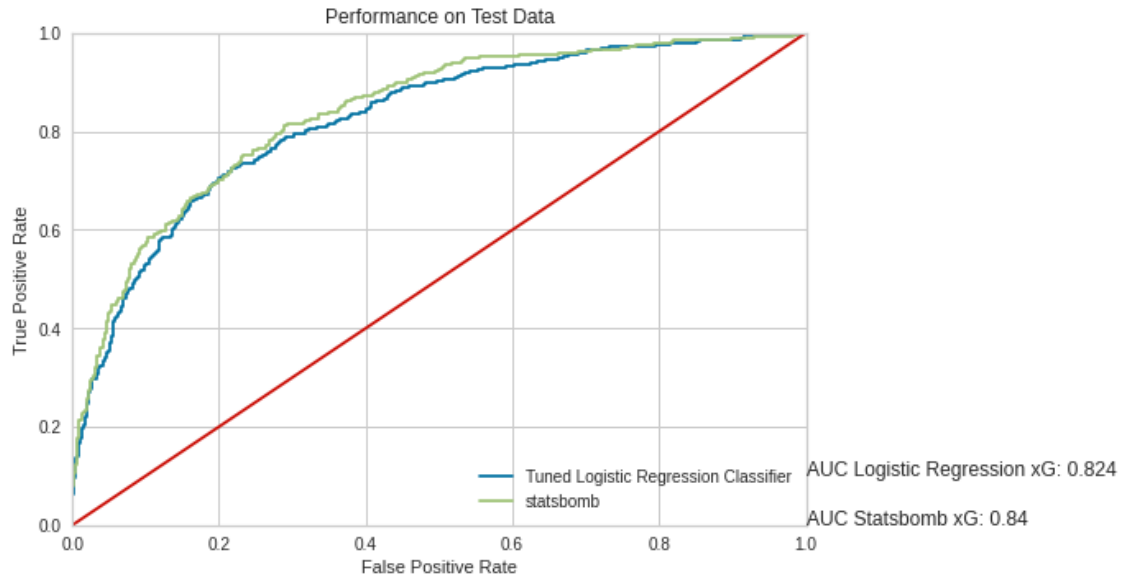
        validation_set_predictions[classifier_name] = y_valid_pred

        print(classifier_name)
        print("training set results: {0}".format(log_loss(y_train, y_train_pred,
    ↪normalize=True)))
        print("validation set results: {0}".format(log_loss(y_valid, y_valid_pred,
    ↪normalize=True)))

    except TypeError:
        continue
```

```
catboost
training set results: 0.2952140014842253
validation set results: 0.31736389059916464
lightgbm
training set results: 0.3024277573656228
validation set results: 0.31949260358602866
xgboost
training set results: 0.19243117485859595
validation set results: 0.33565946556925647
logistic regression
training set results: 0.31002072615891196
validation set results: 0.31266230759382185
```

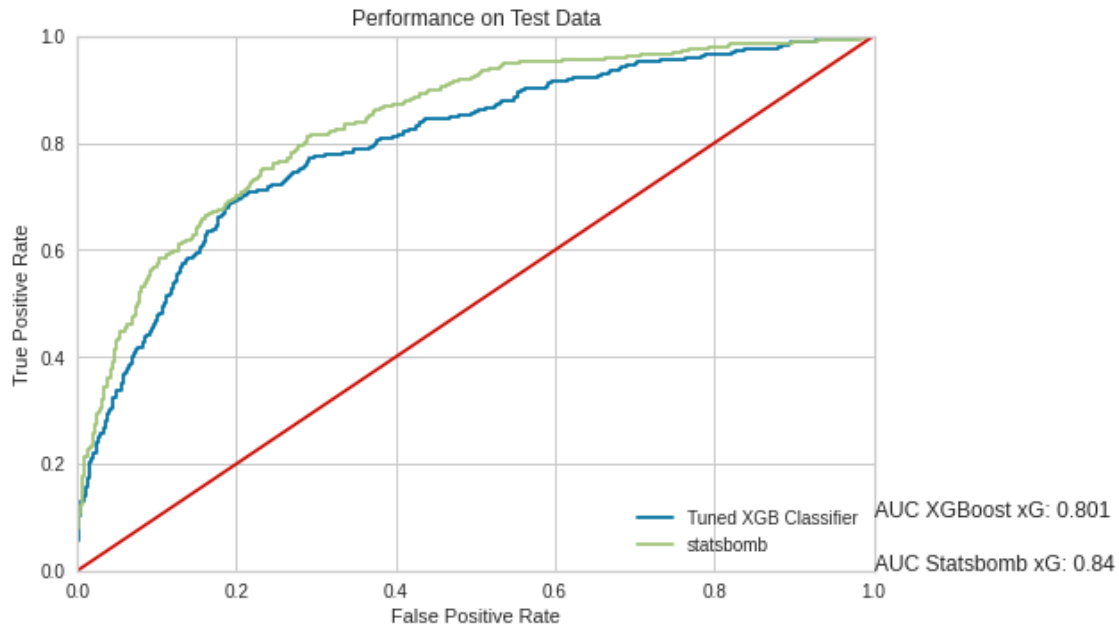
```
[ ]: processed_full_X_train = np.concatenate((processed_X_train, processed_X_test),  
      ↪axis=0)  
binary_xgbmodel = random_cv_results["xgboost"][1].fit(processed_full_X_train,  
      ↪y_train.append(y_valid))  
binary_catmodel = random_cv_results["catboost"][1].fit(processed_full_X_train,  
      ↪y_train.append(y_valid))  
binary_lgbmodel = random_cv_results["lightgbm"][1].fit(processed_full_X_train,  
      ↪y_train.append(y_valid))  
  
[ ]: binary_lr_model = random_cv_results["logistic regression"][1].  
      ↪fit(processed_full_X_train, y_train.append(y_valid))  
  
[ ]: X_test_preprocessed = main_pipeline.fit_transform(test)  
  
[ ]: statsbomb_proba_test= test['official xg']  
  
[ ]: y_test_pred = binary_lr_model.predict(X_test_preprocessed)  
y_test_proba = binary_lr_model.predict_proba(X_test_preprocessed)  
  
[ ]: plot_roc_curve(y_test,y_test_proba,statsbomb_proba_test,model_label='Tuned'  
      ↪'Logistic Regression Classifier')  
plt.title('Performance on Test Data')  
plt.annotate("AUC Logistic Regression xG: {}".  
      ↪format(round(roc_auc_score(y_test,y_test_proba[:,1]),3)), xy = (1, 0.1),  
      ↪size=12)  
plt.annotate("AUC Statsbomb xG: {}".  
      ↪format(round(roc_auc_score(y_test,statsbomb_proba_test),2)), xy = (1, 0),  
      ↪size=12)  
#print('The auc score for the tuned Logistic Regression model on the test data'  
      ↪is {}'.format(round(roc_auc_score(y_test,y_test_proba[:,1]),3)))  
#print('The auc score for the Statsbomb model on the test data is {}'.  
      ↪format(round(roc_auc_score(y_test,statsbomb_proba_test),2)))  
  
[ ]: Text(1, 0, 'AUC Statsbomb xG: 0.84')
```



```
[ ]: y_test_pred = binary_xgbmodel.predict(X_test_preprocessed)
y_test_proba = binary_xgbmodel.predict_proba(X_test_preprocessed)

[ ]: plot_roc_curve(y_test,y_test_proba,statsbomb_proba_test,model_label='Tuned XGB_
    ↪Classifier')
plt.title('Performance on Test Data')
plt.annotate("AUC XGBoost xG: {}".
    ↪format(round(roc_auc_score(y_test,y_test_proba[:,1]),3)), xy = (1, 0.1),_
    ↪size=12)
plt.annotate("AUC Statsbomb xG: {}".
    ↪format(round(roc_auc_score(y_test,statsbomb_proba_test),2)), xy = (1, 0),_
    ↪size=12)
#print('The auc score for the tuned XGBoost model on the test data is {}'.
    ↪format(round(roc_auc_score(y_test,y_test_proba[:,1]),3))
#print('The auc score for the Statsbomb model on the test data is {}'.
    ↪format(round(roc_auc_score(y_test,statsbomb_proba_test),2)))

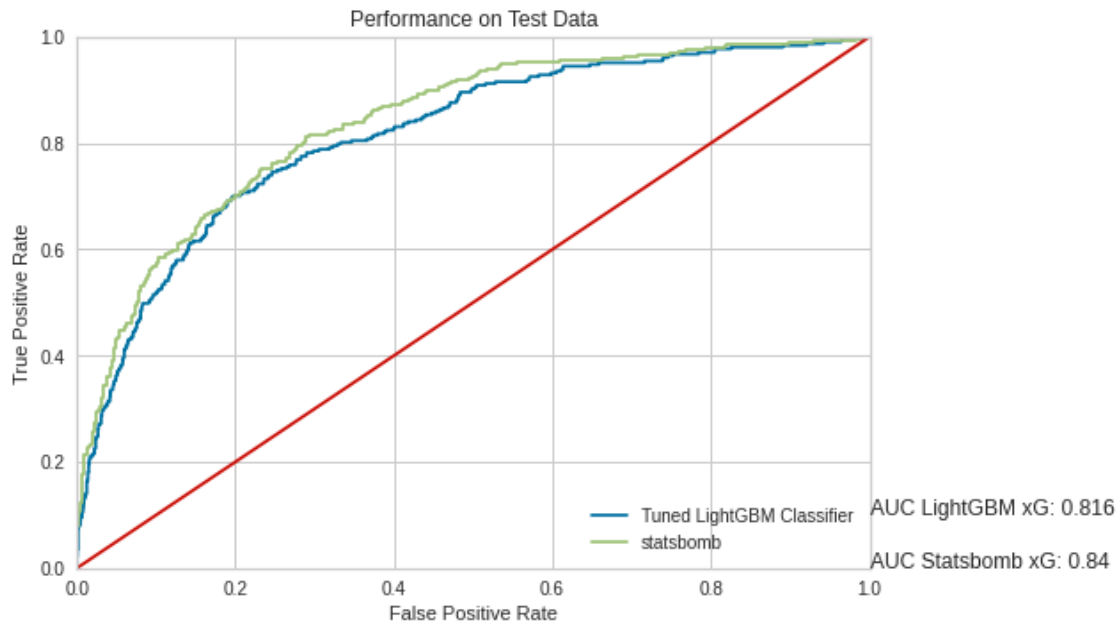
[ ]: Text(1, 0, 'AUC Statsbomb xG: 0.84')
```



```
[ ]: y_test_pred = binary_lgbmodel.predict(X_test_preprocessed)
     y_test_proba = binary_lgbmodel.predict_proba(X_test_preprocessed)

[ ]: plot_roc_curve(y_test,y_test_proba,statsbomb_proba_test,model_label='Tuned_
     ↳LightGBM Classifier')
     plt.title('Performance on Test Data')
     plt.annotate("AUC LightGBM xG: {}".
     ↳format(round(roc_auc_score(y_test,y_test_proba[:,1]),3)), xy = (1, 0.1),
     ↳size=12)
     plt.annotate("AUC Statsbomb xG: {}".
     ↳format(round(roc_auc_score(y_test,statsbomb_proba_test),2)), xy = (1, 0),
     ↳size=12)
     #print('The auc score for the tuned LightGBM model on the test data is {}'.
     ↳format(round(roc_auc_score(y_test,y_test_proba[:,1]),3)))
     #print('The auc score for the Statsbomb model on the test data is {}'.
     ↳format(round(roc_auc_score(y_test,statsbomb_proba_test),2)))

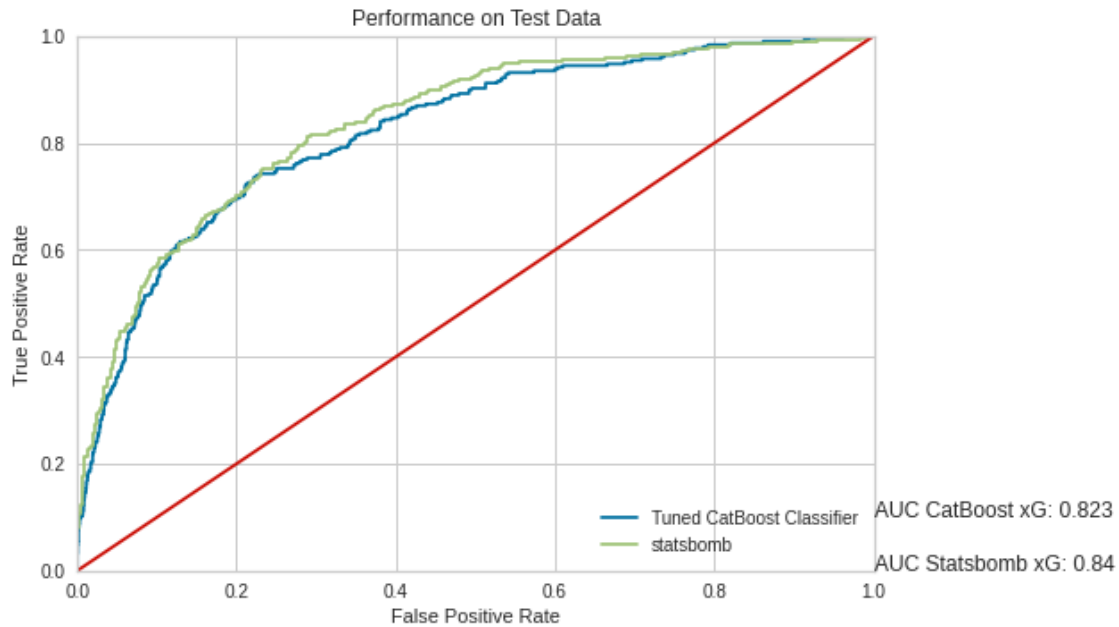
[ ]: Text(1, 0, 'AUC Statsbomb xG: 0.84')
```



```
[ ]: y_test_pred = binary_catmodel.predict(X_test_preprocessed)
     y_test_proba = binary_catmodel.predict_proba(X_test_preprocessed)

[ ]: plot_roc_curve(y_test,y_test_proba,statsbomb_proba_test,model_label='Tuned_
     ↪CatBoost Classifier')
plt.title('Performance on Test Data')
plt.annotate("AUC CatBoost xG: {}".
     ↪format(round(roc_auc_score(y_test,y_test_proba[:,1]),3)), xy = (1, 0.1),
     ↪size=12)
plt.annotate("AUC Statsbomb xG: {}".
     ↪format(round(roc_auc_score(y_test,statsbomb_proba_test),2)), xy = (1, 0),
     ↪size=12)
#print('The auc score for the tuned CatBoost model on the test data is {}'.
     ↪format(round(roc_auc_score(y_test,y_test_proba[:,1]),3)))
#print('The auc score for the Statsbomb model on the test data is {}'.
     ↪format(round(roc_auc_score(y_test,statsbomb_proba_test),2)))

[ ]: Text(1, 0, 'AUC Statsbomb xG: 0.84')
```



```
[ ]: logit_coefs = binary_lr_model.coef_
logit_coefs[0]
```

```
[ ]: array([-0.08241951, -0.91635726, -0.05164229, -0.39384838, -0.40525183,
          -0.14803426, -0.5592553 , -0.43952146, -0.5556064 , -0.44317037,
          -1.52976118,  0.27338906, -0.09968207, -0.39271053, -0.61687289,
           0.07181805, -0.0336103 , -0.39088635, -0.03677007])
```

```
[ ]: xgb_importances = binary_xgbmodel.feature_importances_
xgb_importances
```

```
[ ]: array([0.12537093, 0.          , 0.04136473, 0.05434451, 0.03127528,
          0.04465916, 0.0370964 , 0.          , 0.04551385, 0.          ,
          0.06768027, 0.10602297, 0.03563055, 0.1920847 , 0.03986996,
          0.03823037, 0.04618715, 0.05694861, 0.03772057], dtype=float32)
```

```
[ ]: lgb_importances = binary_lgbmodel.feature_importances_
lgb_importances
```

```
[ ]: array([ 4,  0,  2,  1,  1,  0,  2,  0,  1,  0,  6, 24,  4,  7,  8,  6, 10,
          10, 14])
```

```
[ ]: cat_importances = binary_catmodel.feature_importances_
cat_importances
```

```
[ ]: array([ 2.15015197,  3.12278295,  1.98455298,  1.35832003,  0.07119724,
          1.59383346,  1.16280143,  0.29684719,  1.55772241,  1.34925747,
          12.19741295, 26.19453693,  3.08269503, 11.37641689,  7.20350078,
          4.32074502,  8.58871011, 11.00127916,  1.38723599])
```

```
[ ]: fig,(ax1,ax2,ax3,ax4) = plt.
    ↪subplots(nrows=4,ncols=1,figsize=(20,10),sharex=True)

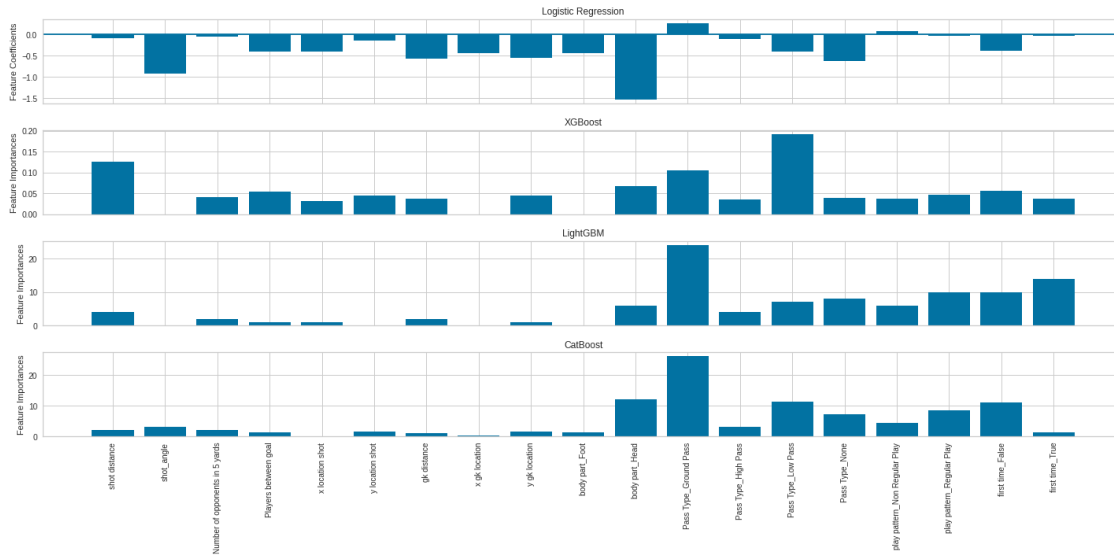
#logistic regression plot
ax1.bar(range(len(logit_coefs[0])),logit_coefs[0],align='center')
ax1.axhline(y=0)
ax1.set_ylabel('Feature Coefficients')
ax1.set_title('Logistic Regression')

#xgboost plot
ax2.bar(range(len(xgb_importances)),xgb_importances,align='center')
ax2.set_ylabel('Feature Importances')
ax2.set_title('XGBoost')

#gradboost plot
ax3.bar(range(len(lgb_importances)),lgb_importances,align='center')
ax3.set_ylabel('Feature Importances')
ax3.set_title('LightGBM')

ax4.bar(range(len(cat_importances)),cat_importances,align='center')
ax4.set_xticks(np.arange(0,len(xgb_importances),1))
ax4.set_xticklabels(all_features_transformed)
plt.xticks(rotation=90)
ax4.set_ylabel('Feature Importances')
ax4.set_title('CatBoost')

plt.tight_layout()
```



```
[ ]: from lime import lime_tabular
explainer = lime_tabular.LimeTabularExplainer(processed_full_X_train,
    ↪mode="classification",
    class_names=train['Goal_label'].
    ↪unique(),
    ↪feature_names=all_features_transformed,
    )
```

```
[ ]: pickle.dump(binary_xgbmodel, open('binary_xgbmodel.pkl', 'wb'))
pickle.dump(binary_lgbmodel, open('binary_lgbmodel.pkl', 'wb'))
pickle.dump(binary_catmodel, open('binary_catmodel.pkl', 'wb'))
```

```
[ ]: pickle.dump(binary_lr_model, open('binary_lr_model.pkl', 'wb'))
```

```
[ ]: binary_xgbmodel = pickle.load(open('binary_xgbmodel.pkl', 'rb'))
binary_lgbmodel = pickle.load(open('binary_lgbmodel.pkl', 'rb'))
binary_catmodel = pickle.load(open('binary_catmodel.pkl', 'rb'))
```

```
[ ]: binary_lr_model = pickle.load(open('binary_lr_model.pkl', 'rb'))
```

```
[ ]: processed_full_X_train = np.concatenate((processed_X_train, processed_X_test),
    ↪axis=0)
y_train_full = y_train.append(y_valid)

test_set_predictions = {'lightgbm' : None,
                        'xgboost' : None,
                        'catboost' : None,
```

```

        'logistic regression': None}

#for classifier_name, classifier_obj in random_cv_results.items():
#    try:
#classifier = random_cv_results[classifier_name][1]
binary_xgbmodel.fit(processed_full_X_train, y_train.append(y_valid))
y_test_pred = binary_xgbmodel.predict_proba(X_test_preprocessed)

test_set_predictions['xgboost'] = y_test_pred

print('XGBoost')
print("test set results: {0}\n".format(log_loss(y_test, y_test_pred,
↪normalize=True)))

```

XGBoost
test set results: 0.3134145029161513

```

[ ]: eli5.explain_weights_xgboost(binary_xgbmodel, feature_names = np.
↪array(all_features_transformed), top=100)
#xgb, vec=None, top=20, target_names=None, targets=None, feature_names=None,
↪feature_re=None, feature_filter=None, importance_type='gain')

```

```

[ ]: Explanation(estimator='XGBClassifier(eta=0.32889574848851144, max_depth=7,
num_leaves=76)', description='\nXGBoost feature importances; values are numbers
0 <= x <= 1;\nall values sum to 1.\n', error=None, method='feature importances',
is_regression=False, targets=None,
feature_importances=FeatureImportances(importances=[FeatureWeight(feature='Pass
Type_Low Pass', weight=0.1920847, std=None, value=None),
FeatureWeight(feature='shot distance', weight=0.12537093, std=None, value=None),
FeatureWeight(feature='Pass Type_Ground Pass', weight=0.10602297, std=None,
value=None), FeatureWeight(feature='body part_Head', weight=0.06768027,
std=None, value=None), FeatureWeight(feature='first time_False',
weight=0.05694861, std=None, value=None), FeatureWeight(feature='Players between
goal', weight=0.054344505, std=None, value=None), FeatureWeight(feature='play
pattern_Regular Play', weight=0.046187147, std=None, value=None),
FeatureWeight(feature='y gk location', weight=0.04551385, std=None, value=None),
FeatureWeight(feature='y location shot', weight=0.044659164, std=None,
value=None), FeatureWeight(feature='Number of opponents in 5 yards',
weight=0.041364733, std=None, value=None), FeatureWeight(feature='Pass
Type_None', weight=0.03986996, std=None, value=None),
FeatureWeight(feature='play pattern_Non Regular Play', weight=0.03823037,
std=None, value=None), FeatureWeight(feature='first time_True',
weight=0.037720572, std=None, value=None), FeatureWeight(feature='gk distance',
weight=0.037096404, std=None, value=None), FeatureWeight(feature='Pass Type_High
Pass', weight=0.035630547, std=None, value=None), FeatureWeight(feature='x
location shot', weight=0.031275284, std=None, value=None),

```

```

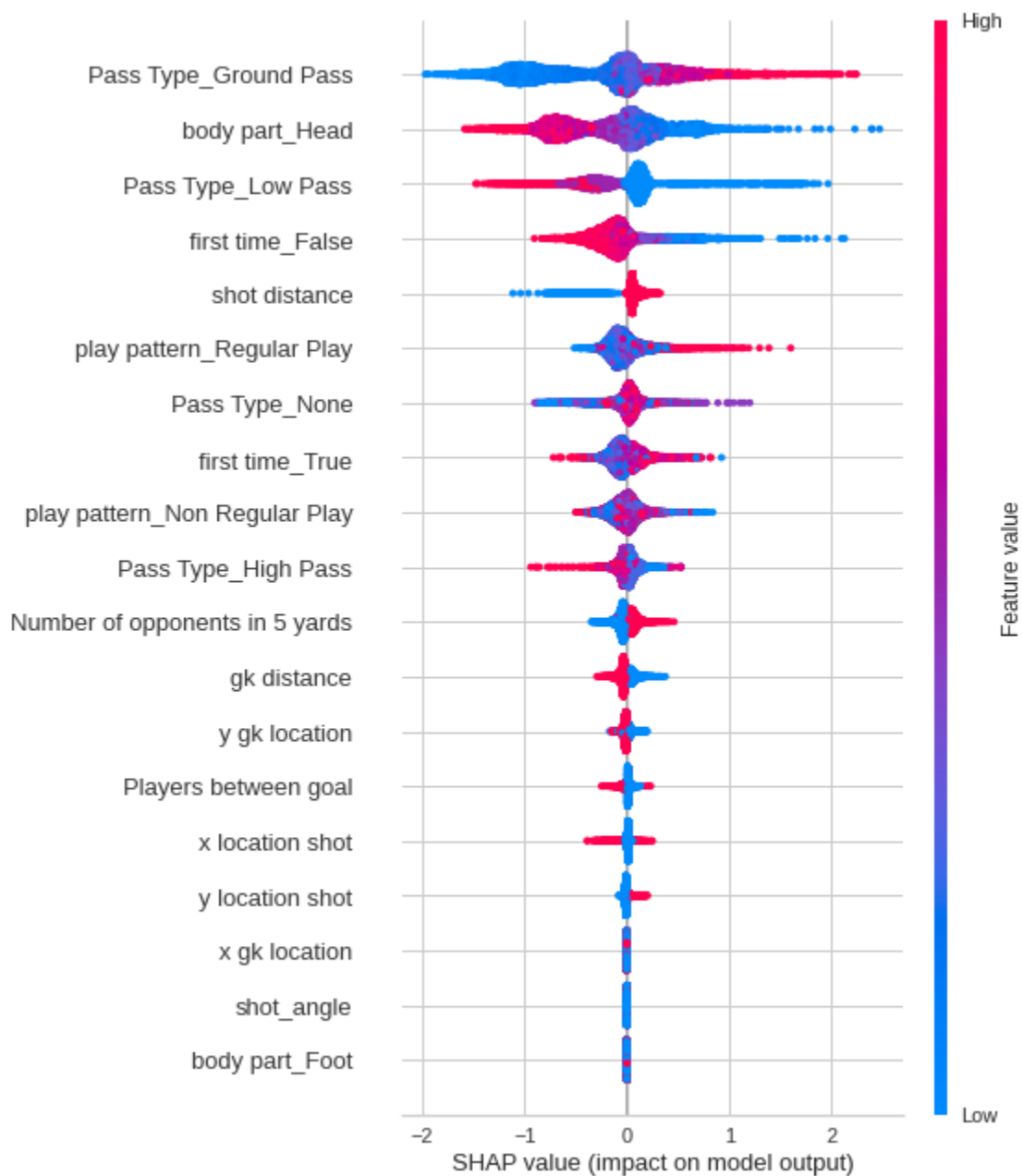
FeatureWeight(feature='x gk location', weight=0.0, std=None, value=None),
FeatureWeight(feature='shot_angle', weight=0.0, std=None, value=None),
FeatureWeight(feature='body part_Foot', weight=0.0, std=None, value=None)],
remaining=0), decision_tree=None, highlight_spaces=None,
transition_features=None, image=None)

```

```

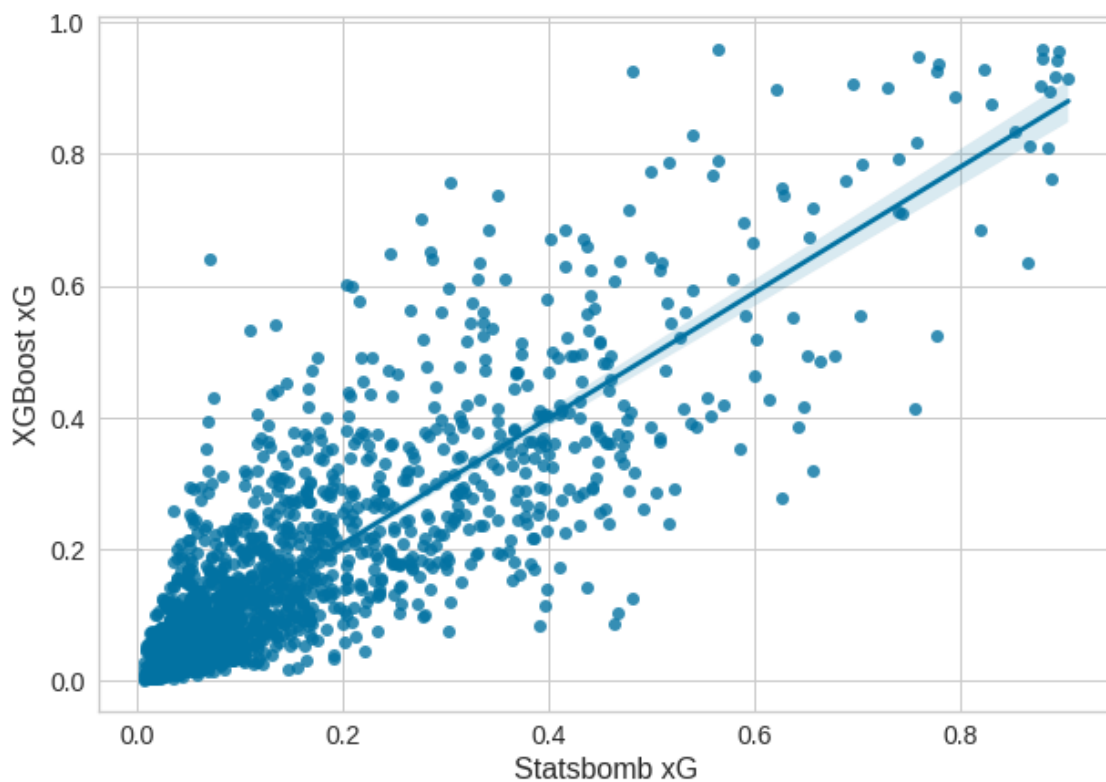
[ ]: explainer = shap.TreeExplainer(binary_xgbmodel)
shap_values = explainer.shap_values(processed_X_train)
shap.summary_plot(shap_values, processed_X_train, feature_names = _
↪all_features_transformed, class_names = train["Goal_label"].unique())

```



```
[ ]: shot_test_set_results = test.copy()
shot_test_set_results["xgboost xg"] = test_set_predictions["xgboost"][:, 1]
fig, ax = plt.subplots(1,1, figsize=[10,7])
#shot_test_set_results.plot("xgboost xg", "official xg", kind="scatter", ax=ax,
    ↪alpha=0.25)
sns.regplot(data=shot_test_set_results,y="xgboost xg",x="official xg")
plt.tick_params(labelsize=14)
plt.xlabel("Statsbomb xG", size=16)
plt.ylabel("XGBoost xG", size=16)
```

```
[ ]: Text(0, 0.5, 'XGBoost xG')
```



```
[ ]: from sklearn.metrics import classification_report, confusion_matrix
processed_full_X_train = np.concatenate((processed_X_train, processed_X_test),
    ↪axis=0)
y_train_full = y_train.append(y_valid)

#for classifier_name, classifier_obj in random_cv_results.items():
# try:
```

```

#classifier = random_cv_results[classifier_name][1]
binary_lr_model.fit(processed_full_X_train, y_train.append(y_valid))
y_test_pred = binary_lr_model.predict_proba(X_test_preprocessed)
y_test_pred_val = binary_lr_model.predict(X_test_preprocessed)
test_set_predictions['logistic regression'] = y_test_pred

print('Logistic Regression')
print("test set results: {0}\n".format(log_loss(y_test, y_test_pred,
↪normalize=True)))
#print(classification_report(y_test, y_test_pred_val))
#print(confusion_matrix(y_test, y_test_pred_val))

```

Logistic Regression

test set results: 0.299873193721975

```

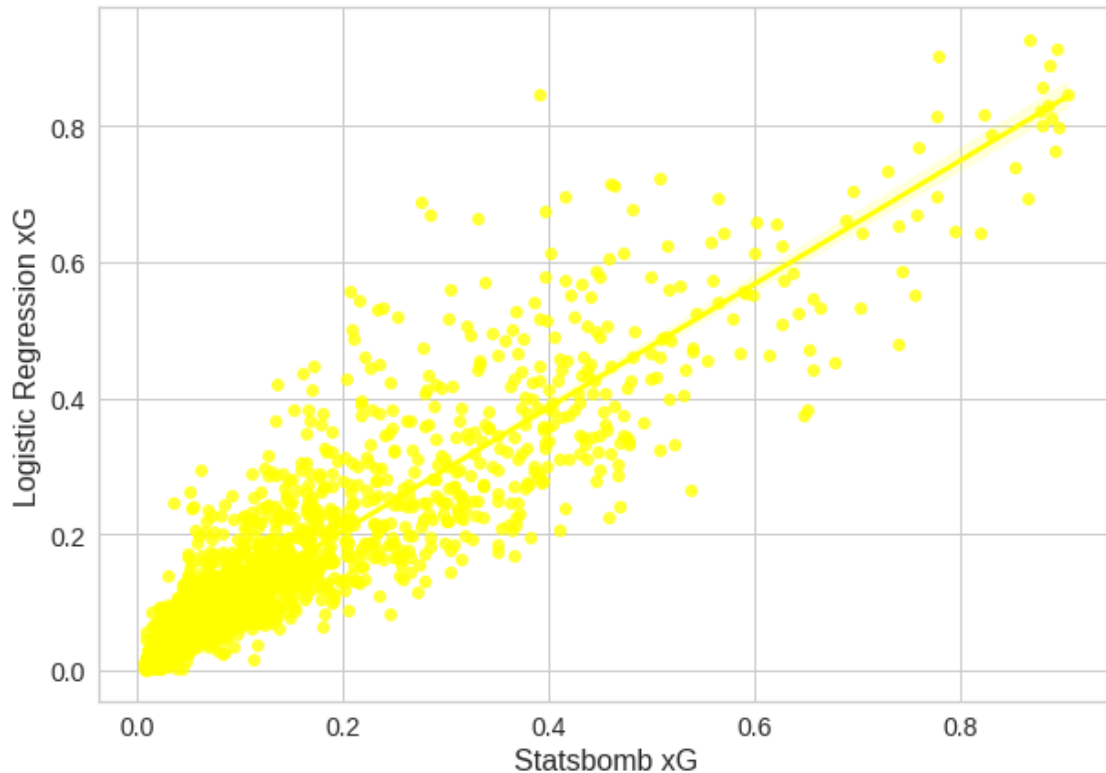
[ ]: shot_test_set_results["logreg xg"] = test_set_predictions["logistic_
↪regression"][:, 1]
fig, ax = plt.subplots(1,1, figsize=[10,7])
sns.regplot(data=shot_test_set_results,y="logreg xg",x="official xg",
↪color='yellow')
plt.tick_params(labelsize=14)
plt.xlabel("Statsbomb xG", size=16)
plt.ylabel("Logistic Regression xG", size=16)

```

```

[ ]: Text(0, 0.5, 'Logistic Regression xG')

```



```
[ ]: eli5.explain_weights(binary_lr_model, feature_names = np.
    ↪array(all_features_transformed), top=100)
```

```
[ ]: Explanation(estimator='LogisticRegression(C=2300.0475375005203,
max_iter=10000)', description="\nFeatures with largest
coefficients.\nCaveats:\n1. Be careful with features which are not\n
independent - weights don't show their importance.\n2. If scale of input
features is different then scale of coefficients\n    will also be different,
making direct comparison between coefficient values\n    incorrect.\n3. Depending
on regularization, rare features sometimes may have high\n    coefficients; this
doesn't mean they contribute much to the\n    classification result for most
examples.\n", error=None, method='linear model', is_regression=False,
targets=[TargetExplanation(target=1,
feature_weights=FeatureWeights(pos=[FeatureWeight(feature='Pass Type_Ground
Pass', weight=0.27338906009759995, std=None, value=None),
FeatureWeight(feature='play pattern_Non Regular Play',
weight=0.07181805271693752, std=None, value=None)],
neg=[FeatureWeight(feature='body part_Head', weight=-1.5297611772521935,
std=None, value=None), FeatureWeight(feature='<BIAS>',
weight=-0.9988388021531359, std=None, value=None),
FeatureWeight(feature='shot_angle', weight=-0.916357258722577, std=None,
value=None), FeatureWeight(feature='Pass Type_None', weight=-0.6168728873833028,
```

```

std=None, value=None), FeatureWeight(feature='gk distance',
weight=-0.5592553049120426, std=None, value=None), FeatureWeight(feature='y gk
location', weight=-0.5556063992603723, std=None, value=None),
FeatureWeight(feature='body part_Foot', weight=-0.44317036579028096, std=None,
value=None), FeatureWeight(feature='x gk location', weight=-0.4395214601386093,
std=None, value=None), FeatureWeight(feature='x location shot',
weight=-0.40525183424412003, std=None, value=None),
FeatureWeight(feature='Players between goal', weight=-0.39384838116793364,
std=None, value=None), FeatureWeight(feature='Pass Type_Low Pass',
weight=-0.39271052550338664, std=None, value=None), FeatureWeight(feature='first
time_False', weight=-0.39088635309416836, std=None, value=None),
FeatureWeight(feature='y location shot', weight=-0.14803426202998016, std=None,
value=None), FeatureWeight(feature='Pass Type_High Pass',
weight=-0.09968206818753188, std=None, value=None), FeatureWeight(feature='shot
distance', weight=-0.08241950632807135, std=None, value=None),
FeatureWeight(feature='Number of opponents in 5 yards',
weight=-0.051642287608615674, std=None, value=None),
FeatureWeight(feature='first time_True', weight=-0.036770065051479374, std=None,
value=None), FeatureWeight(feature='play pattern_Regular Play',
weight=-0.03361030261786003, std=None, value=None)], pos_remaining=0,
neg_remaining=0), proba=None, score=None, weighted_spans=None, heatmap=None)],
feature_importances=None, decision_tree=None, highlight_spaces=None,
transition_features=None, image=None)

```

```

[ ]: processed_full_X_train = np.concatenate((processed_X_train, processed_X_test),
↪axis=0)
y_train_full = y_train.append(y_valid)

#for classifier_name, classifier_obj in random_cv_results.items():
# try:
#classifier = random_cv_results[classifier_name][1]
binary_catmodel.fit(processed_full_X_train, y_train.append(y_valid))
y_test_pred = binary_catmodel.predict_proba(X_test_preprocessed)

test_set_predictions['catboost'] = y_test_pred

print('CatBoost')
print("test set results: {0}\n".format(log_loss(y_test, y_test_pred,
↪normalize=True)))

```

```

[ ]: eli5.explain_weights_catboost(binary_catmodel, feature_names = np.
↪array(all_features_transformed), top=100)

```

```

[ ]: Explanation(estimator='<catboost.core.CatBoostClassifier object at
0x7fb38a14a210>', description='CatBoost feature importances; \nvalues are
numbers 0 <= x <= 1; all values sum to 1.', error=None, method='feature
importances', is_regression=False, targets=None,

```

```

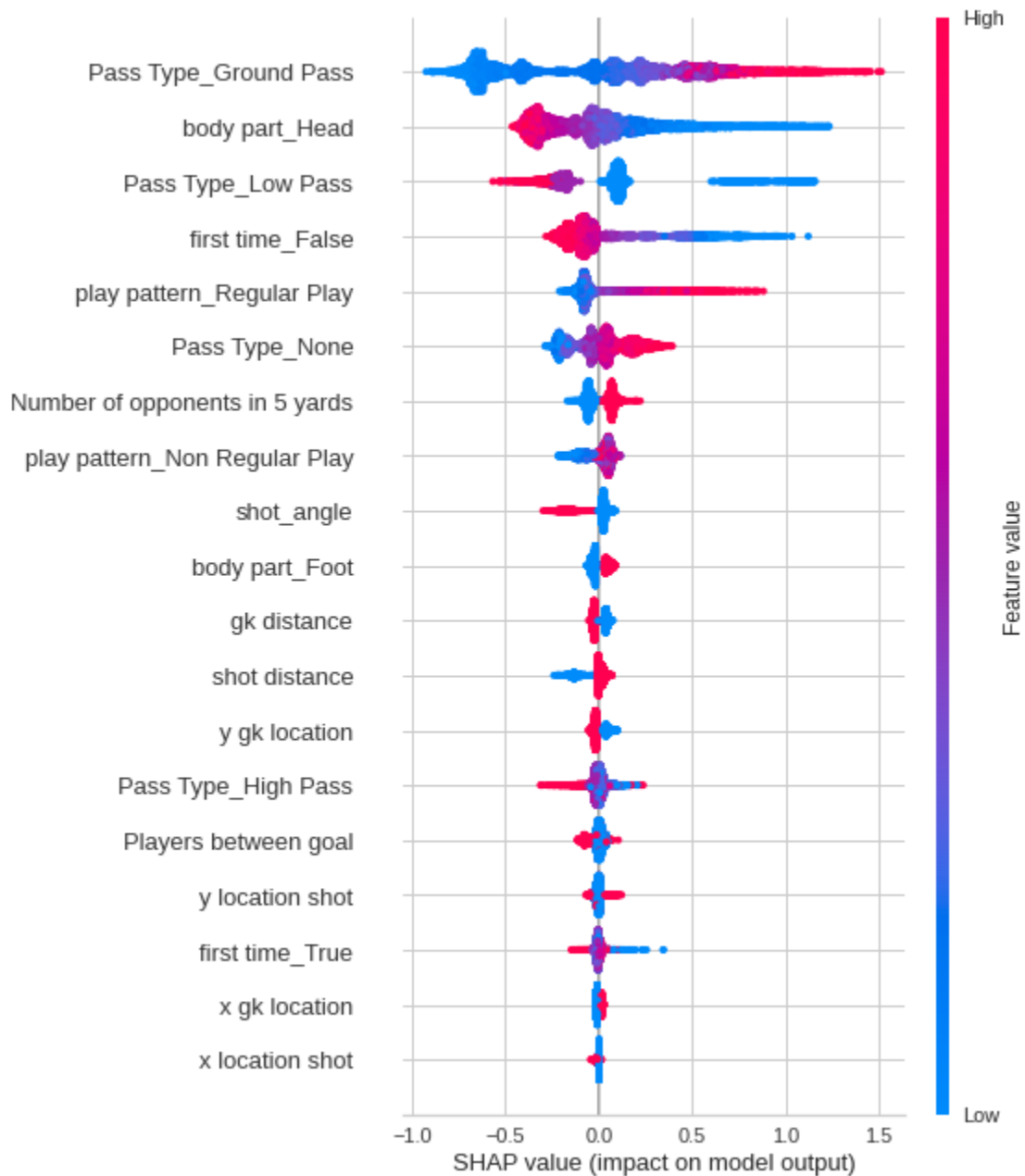
feature_importances=FeatureImportances(importances=[FeatureWeight(feature='Pass
Type_Ground Pass', weight=0.26194537, std=None, value=None),
FeatureWeight(feature='body part_Head', weight=0.121974126, std=None,
value=None), FeatureWeight(feature='Pass Type_Low Pass', weight=0.113764174,
std=None, value=None), FeatureWeight(feature='first time_False',
weight=0.11001279, std=None, value=None), FeatureWeight(feature='play
pattern_Regular Play', weight=0.0858871, std=None, value=None),
FeatureWeight(feature='Pass Type_None', weight=0.07203501, std=None,
value=None), FeatureWeight(feature='play pattern_Non Regular Play',
weight=0.04320745, std=None, value=None), FeatureWeight(feature='shot_angle',
weight=0.031227829, std=None, value=None), FeatureWeight(feature='Pass Type_High
Pass', weight=0.03082695, std=None, value=None), FeatureWeight(feature='shot
distance', weight=0.021501519, std=None, value=None),
FeatureWeight(feature='Number of opponents in 5 yards', weight=0.01984553,
std=None, value=None), FeatureWeight(feature='y location shot',
weight=0.015938334, std=None, value=None), FeatureWeight(feature='y gk
location', weight=0.015577224, std=None, value=None),
FeatureWeight(feature='first time_True', weight=0.01387236, std=None,
value=None), FeatureWeight(feature='Players between goal', weight=0.0135832,
std=None, value=None), FeatureWeight(feature='body part_Foot',
weight=0.013492575, std=None, value=None), FeatureWeight(feature='gk distance',
weight=0.011628014, std=None, value=None), FeatureWeight(feature='x gk
location', weight=0.002968472, std=None, value=None), FeatureWeight(feature='x
location shot', weight=0.0007119724, std=None, value=None)], remaining=0),
decision_tree=None, highlight_spaces=None, transition_features=None, image=None)

```

```

[ ]: explainer = shap.TreeExplainer(binary_catmodel)
shap_values = explainer.shap_values(processed_X_train)
shap.summary_plot(shap_values, processed_X_train, feature_names =
    ↪all_features_transformed, class_names = train["Goal_label"].unique())

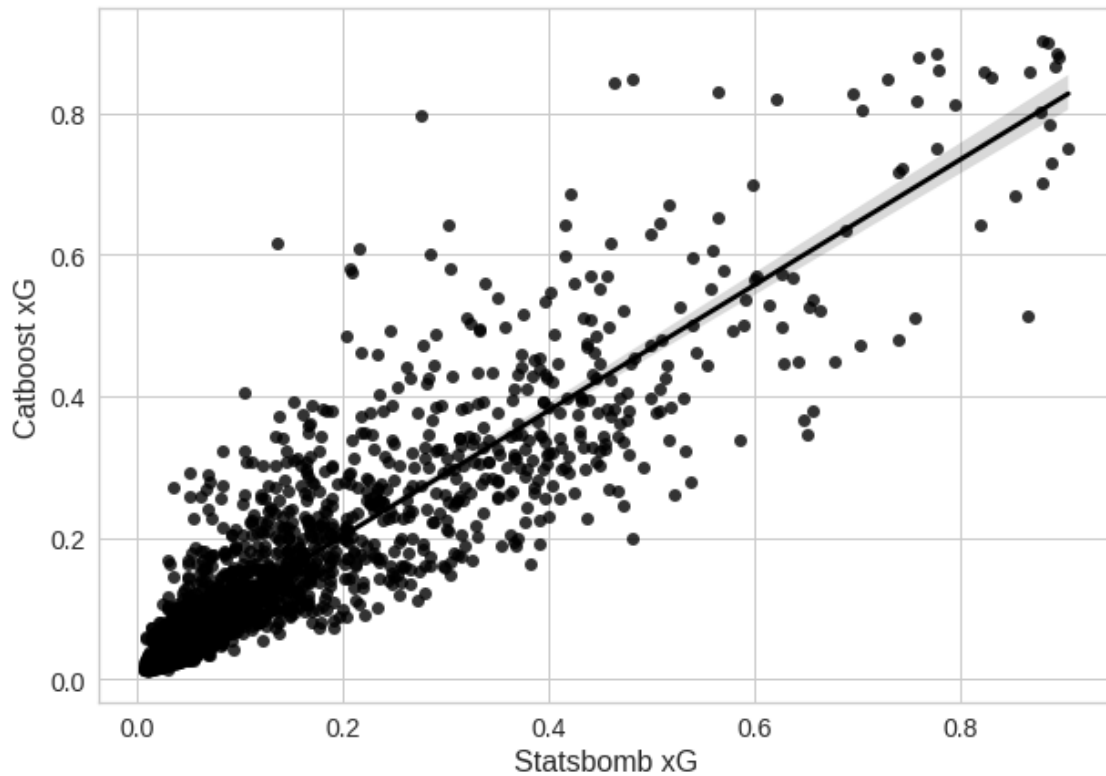
```



```
[ ]: shot_test_set_results["catboost xg"] = test_set_predictions["catboost"][:, 1]
fig, ax = plt.subplots(1,1, figsize=[10,7])
#shot_test_set_results.plot("official xg", "catboost xg", color =_
    ↪ "black",kind="scatter", ax=ax, alpha=0.25)
sns.regplot(data=shot_test_set_results,y="catboost xg",x="official xg",_
    ↪ color='black')
plt.tick_params(labelsize=14)
plt.xlabel("Statsbomb xG", size=16)
```

```
plt.ylabel("Catboost xG", size=16)
```

```
[ ]: Text(0, 0.5, 'Catboost xG')
```



```
[ ]: processed_full_X_train = np.concatenate((processed_X_train, processed_X_test),
↪axis=0)
y_train_full = y_train.append(y_valid)

#for classifier_name, classifier_obj in random_cv_results.items():
# try:
#classifier = random_cv_results[classifier_name][1]
binary_lgbmodel.fit(processed_full_X_train, y_train.append(y_valid))
y_test_pred = binary_lgbmodel.predict_proba(X_test_preprocessed)

test_set_predictions['lightgbm'] = y_test_pred

print('Light Gradient Boost')
print("test set results: {0}\n".format(log_loss(y_test, y_test_pred,
↪normalize=True)))
```

Light Gradient Boost

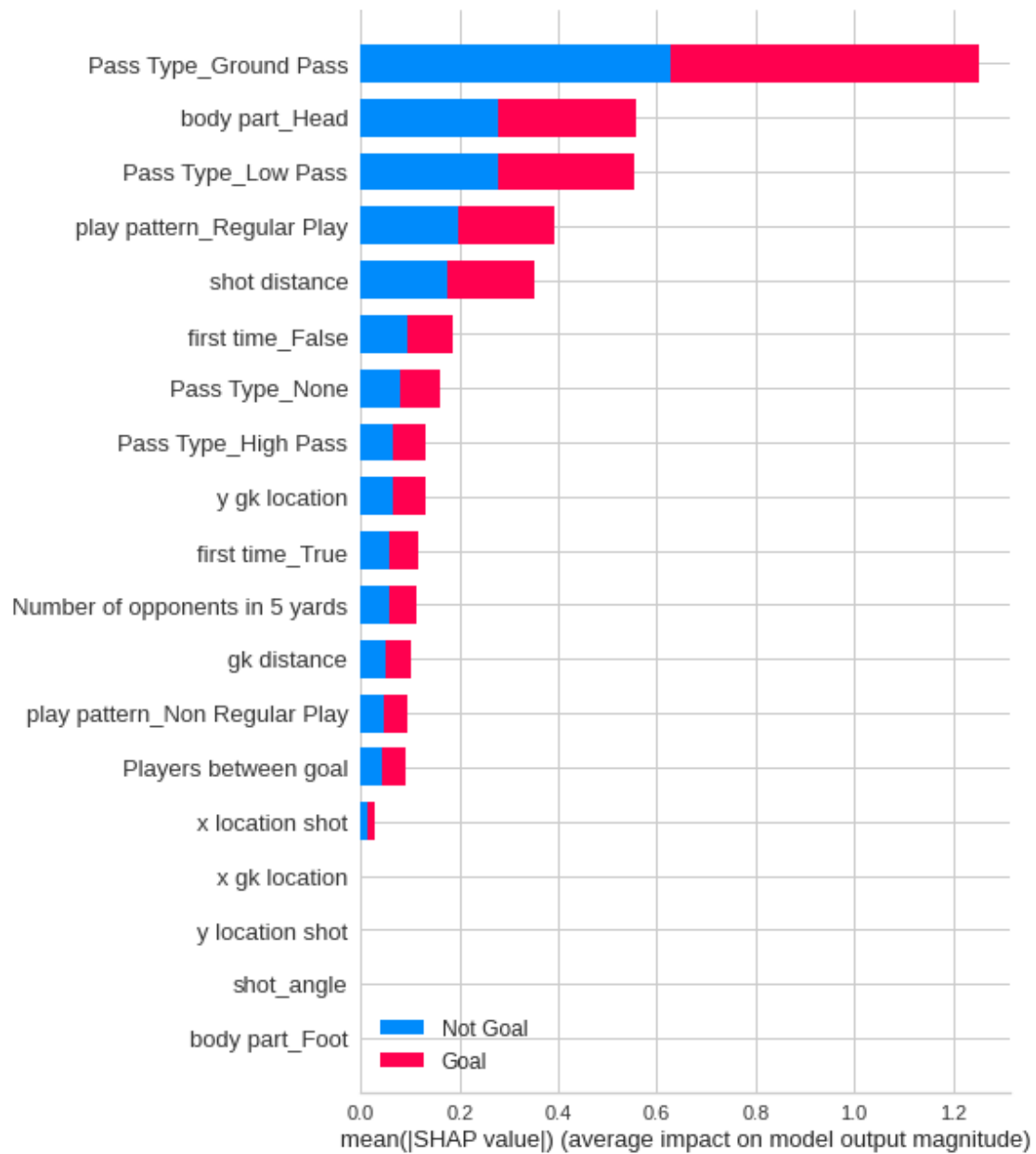
test set results: 0.3057187843739998

```
[ ]: eli5.explain_weights_lightgbm(binary_lgbmodel, feature_names = np.  
    ↪array(all_features_transformed), top=100)
```

```
[ ]: Explanation(estimator='LGBMClassifier(learning_rate=0.46645544479201073,  
max_depth=1, num_leaves=151)', description='\nLightGBM feature importances;  
values are numbers 0 <= x <= 1;\nall values sum to 1.\n', error=None,  
method='feature importances', is_regression=False, targets=None,  
feature_importances=FeatureImportances(importances=[FeatureWeight(feature='Pass  
Type_Ground Pass', weight=0.47464136315761735, std=None, value=None),  
FeatureWeight(feature='play pattern_Regular Play', weight=0.1961807055980082,  
std=None, value=None), FeatureWeight(feature='Pass Type_Low Pass',  
weight=0.12382000514880567, std=None, value=None), FeatureWeight(feature='body  
part_Head', weight=0.07288185374052521, std=None, value=None),  
FeatureWeight(feature='shot distance', weight=0.04870806616780421, std=None,  
value=None), FeatureWeight(feature='first time_False',  
weight=0.024839792548307933, std=None, value=None), FeatureWeight(feature='first  
time_True', weight=0.015687745756329887, std=None, value=None),  
FeatureWeight(feature='Pass Type_None', weight=0.01112721211990801, std=None,  
value=None), FeatureWeight(feature='y_gk location', weight=0.009520665189885192,  
std=None, value=None), FeatureWeight(feature='Pass Type_High Pass',  
weight=0.005753371751336433, std=None, value=None),  
FeatureWeight(feature='Players between goal', weight=0.005344839452203821,  
std=None, value=None), FeatureWeight(feature='play pattern_Non Regular Play',  
weight=0.004301246971267904, std=None, value=None),  
FeatureWeight(feature='Number of opponents in 5 yards',  
weight=0.00322238032008147, std=None, value=None), FeatureWeight(feature='gk  
distance', weight=0.00292665345511595, std=None, value=None),  
FeatureWeight(feature='x location shot', weight=0.0010440986228027506, std=None,  
value=None), FeatureWeight(feature='x_gk location', weight=0.0, std=None,  
value=None), FeatureWeight(feature='y location shot', weight=0.0, std=None,  
value=None), FeatureWeight(feature='shot_angle', weight=0.0, std=None,  
value=None), FeatureWeight(feature='body part_Foot', weight=0.0, std=None,  
value=None)], remaining=0), decision_tree=None, highlight_spaces=None,  
transition_features=None, image=None)
```

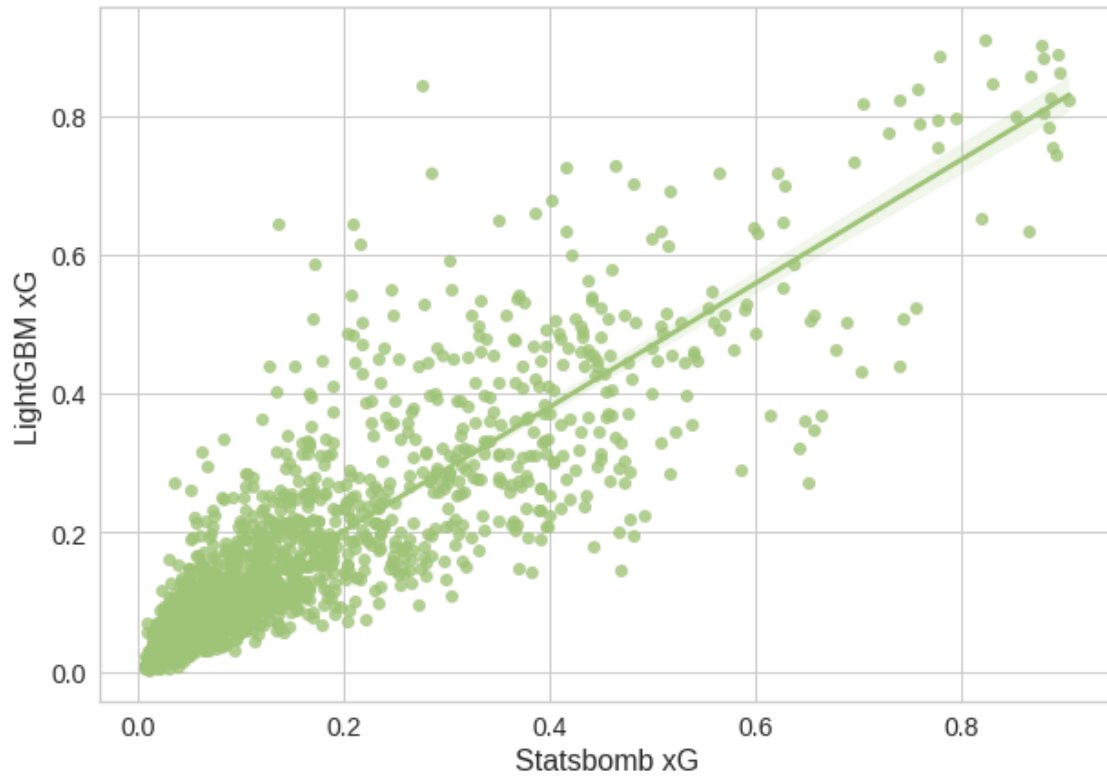
```
[ ]: explainer = shap.TreeExplainer(binary_lgbmodel)  
shap_values = explainer.shap_values(processed_X_train)  
shap.summary_plot(shap_values, processed_X_train, feature_names =  
    ↪all_features_transformed, class_names = train["Goal_label"].unique())
```

LightGBM binary classifier with TreeExplainer shap values output has changed to a list of ndarray



```
[ ]: shot_test_set_results["lightgbm xg"] = test_set_predictions["lightgbm"][:, 1]
fig, ax = plt.subplots(1,1, figsize=[10,7])
sns.regplot(data=shot_test_set_results,y="lightgbm xg",x="official xg",
            color='g')
plt.tick_params(labels=14)
plt.xlabel("Statsbomb xG", size=16)
plt.ylabel("LightGBM xG", size=16)
```

```
[ ]: Text(0, 0.5, 'LightGBM xG')
```



```
[ ]: def freeze_frame1(game_json, shot_id, ax=ax):
    pitchplot()
    goalkeeper_x = 120
    goalkeeper_y = 40
    player_list_x = []
    player_list_y = []
    player_name = []
    team_name = []
    location_of_shot_x = 0
    location_of_shot_y = 0
    shot_end_location_x = 0
    shot_end_location_y = 0
    teammate_x = []
    teammate_y = []

    for events in game_json:
        if events['id'] == shot_id:
            location_of_shot_x = events['location'][0]
            location_of_shot_y = events['location'][1]
            shot_end_location_x = events['shot']['end_location'][0]
            shot_end_location_y = events['shot']['end_location'][1]
```

```

        if "freeze_frame" in events["shot"]:
            for players in events['shot']['freeze_frame']:
                if (players['teammate'] == False):
                    player_list_x.append(players['location'][0])
                    player_list_y.append(players['location'][1])
                    player_name.append(events['player']['name'])
                    team_name.append(events['possession_team'])
                elif (players['teammate'] == True):
                    teammate_x.append(players['location'][0])
                    teammate_y.append(players['location'][1])

            if ((players['position']['name'] == 'Goalkeeper') and
↳(players['teammate'] == False)):
                goalkeeper_x = players['location'][0]
                goalkeeper_y = players['location'][1]

plt.scatter(player_list_x, player_list_y, color = 'black')
plt.scatter(teammate_x, teammate_y, color = 'brown')
plt.scatter(location_of_shot_x, location_of_shot_y, s=200)
plt.scatter(goalkeeper_x, goalkeeper_y, s=100, color='red')
plt.scatter(shot_end_location_x, shot_end_location_y, s=100, color='green')
plt.plot([location_of_shot_x, 120], [location_of_shot_y,36], color = 'red',↳
↳linestyle = 'dashed')
plt.plot([location_of_shot_x, 120], [location_of_shot_y,44], color = 'red',↳
↳linestyle = 'dashed')
plt.title(player_name[0], fontsize = 30)
return plt

```

```

[ ]: def freeze_frame2(game_json, shot_id,ax=ax):
    halfpitchplot()
    goalkeeper_x = 120
    goalkeeper_y = 40
    player_list_x = []
    player_list_y = []
    player_name = []
    team_name = []
    location_of_shot_x = 0
    location_of_shot_y = 0
    shot_end_location_x = 0
    shot_end_location_y = 0
    teammate_x = []
    teammate_y = []

    for events in game_json:
        if events['id'] == shot_id:
            location_of_shot_x = events['location'][0]
            location_of_shot_y = events['location'][1]

```

```

shot_end_location_x = events['shot']['end_location'][0]
shot_end_location_y = events['shot']['end_location'][1]

if "freeze_frame" in events["shot"]:
    for players in events['shot']['freeze_frame']:
        if (players['teammate'] == False):
            player_list_x.append(players['location'][0])
            player_list_y.append(players['location'][1])
            player_name.append(events['player']['name'])
            team_name.append(events['possession_team'])
        elif (players['teammate'] == True):
            teammate_x.append(players['location'][0])
            teammate_y.append(players['location'][1])

            if ((players['position']['name'] == 'Goalkeeper') and
↳(players['teammate'] == False)):
                goalkeeper_x = players['location'][0]
                goalkeeper_y = players['location'][1]

plt.scatter(player_list_x, player_list_y, color = 'black')
plt.scatter(teammate_x, teammate_y, color = 'brown')
plt.scatter(location_of_shot_x, location_of_shot_y, s=200)
plt.scatter(goalkeeper_x, goalkeeper_y, s=100, color='red')
plt.scatter(shot_end_location_x, shot_end_location_y, s=100, color='green')
plt.plot([location_of_shot_x, 120], [location_of_shot_y,36], color = 'red',
↳linestyle = 'dashed')
plt.plot([location_of_shot_x, 120], [location_of_shot_y,44], color = 'red',
↳linestyle = 'dashed')
plt.title(player_name[0], fontsize = 30)
return plt

```

```

[ ]: predicted_xg=0.25*shot_test_set_results['xgboost xg']+0.
↳25*shot_test_set_results['lightgbm xg']+0.25*shot_test_set_results['logreg_
↳xg']+(1-0.75)*shot_test_set_results['catboost xg']

```

```

[ ]: shot_test_set_results['predicted xg'] = predicted_xg

```

```

[ ]: shot_test_set_results.columns

```

```

[ ]: Index(['play pattern', 'x location shot', 'y location shot', 'duration',
'outcome', 'technique used', 'first time', 'x gk location',
'y gk location', 'body part', 'type of shot',
'Number of opponents in 5 yards', 'Players between goal', 'player name',
'Team Name', 'official xg', 'Pass id', 'Pass Type', 'game_id',
'season_id', 'distance_from_goalpost_a', 'distance_from_goalpost_b',
'shot_angle', 'shot distance', 'gk distance', 'Goal_label',
'binary_outcome', 'xgboost xg', 'logreg xg', 'catboost xg',

```

```

'lightgbm xg', 'predicted xg'],
dtype='object')

```

```

[ ]: df_player_level = shot_test_set_results[['player_
↳name', 'binary_outcome', 'predicted xg']]
df_player_level=df_player_level.groupby(['player name']).sum()
df_player_level['xG_Diff']=df_player_level['binary_outcome']-df_player_level['predicted_
↳xg']

df_player_level_xG=df_player_level.sort_values(by=['xG_Diff'],ascending=False)
df_player_level_xG=df_player_level_xG.round(1)
df_player_level_xG.columns=['goals', 'xG_sum', 'xG_Diff']
df_player_level_xG.head(10)

```

```

[ ]:

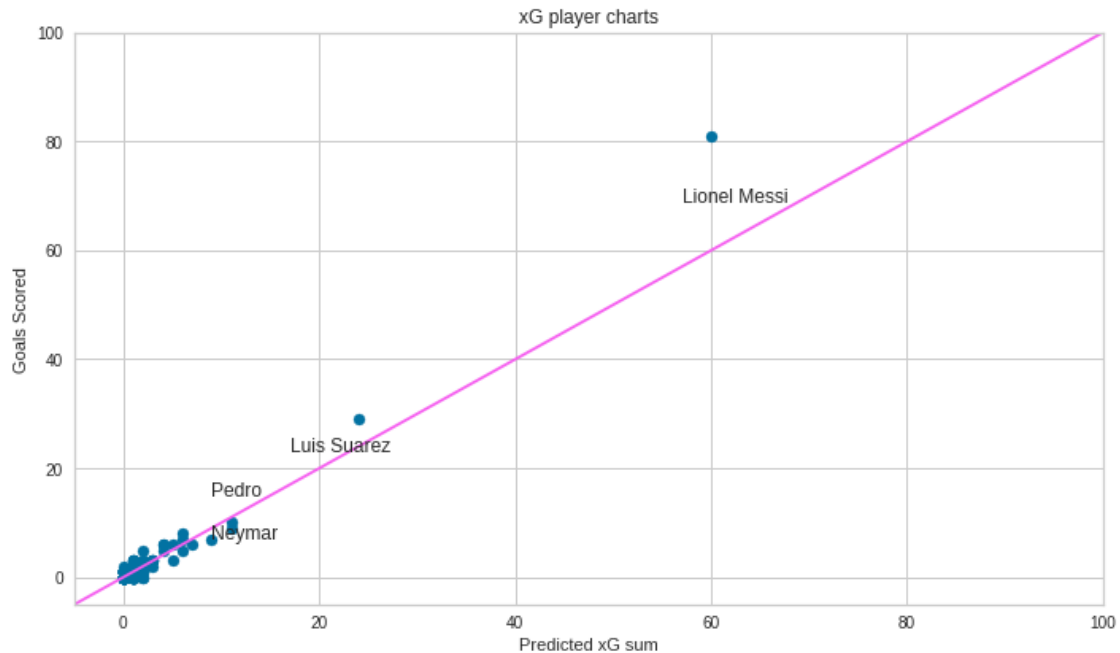
```

	goals	xG_sum	xG_Diff
player name			
Lionel Andrés Messi Cuccittini	81	60.1	20.9
Luis Alberto Suárez Díaz	29	24.2	4.8
Carles Puyol i Saforcada	5	2.5	2.5
Alexis Alejandro Sánchez Sánchez	8	5.6	2.4
Rubén Castro Martín	3	0.8	2.2
Sergio Busquets i Burgos	3	1.1	1.9
Arturo Erasmo Vidal Pardo	3	1.2	1.8
Antoine Griezmann	6	4.3	1.7
Eiður Smári Guðjohnsen	3	1.4	1.6
Francesc Fàbregas i Soler	6	4.4	1.6

```

[ ]: plt.figure(figsize=(10,6))
plt.scatter(round(df_player_level_xG['xG_sum'],0),df_player_level_xG['goals'])
plt.plot([-5,100],[-5,100],color="#f65ef0")
plt.xlabel('Predicted xG sum')
plt.ylabel('Goals Scored')
plt.xlim([-5,100])
plt.ylim([-5,100])
plt.annotate('Lionel Messi',xy=(57,69),xytext=(57,69))
plt.annotate('Luis Suarez',xy=(17,23),xytext=(17,23))
plt.annotate('Pedro',xy=(9,15),xytext=(9,15))
plt.annotate('Neymar',xy=(9,7),xytext=(9,7))
plt.title('xG player charts')
plt.tight_layout()
plt.savefig('xg_player_charts.png')

```



```
[ ]: match_level_xg = shot_test_set_results[['x location shot','y location_
↳shot','Team Name','player name','binary_outcome','predicted xg']]

[ ]: season_xg=match_level_xg[['player name','binary_outcome','predicted xg','Team_
↳Name']].groupby('Team Name').sum()

season_xg = season_xg.round(1)
season_xg['xgDiff']=season_xg['binary_outcome']-season_xg['predicted xg']
season_xg.columns=['Goals','Cumulative xG','xgDiff']
season_xg=season_xg.sort_values(by='Cumulative xG',ascending=False).iloc[0:19]
season_xg['Actual League Position'] =_
↳[1,4,9,6,3,10,8,2,5,7,11,16,18,12,19,13,14,15,17]
season_xg
```

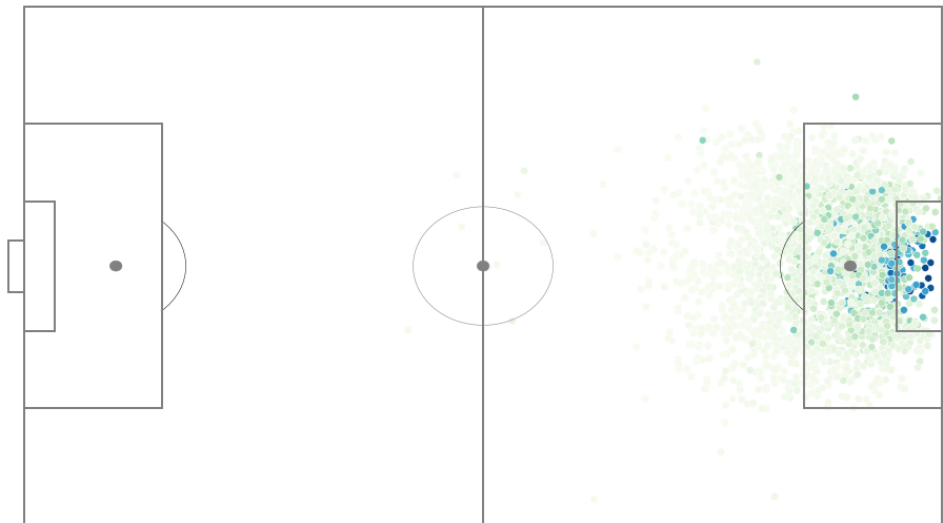
```
[ ]:
Team Name      Goals  Cumulative xG  xgDiff  Actual League Position
Barcelona      242      220.4      21.6              1
Real Madrid    2       7.0      -5.0              4
Valencia       8       5.8       2.2              9
Athletic Club  1       5.7      -4.7              6
Espanyol       3       5.4      -2.4              3
Sevilla        2       5.3      -3.3             10
Atlético Madrid 4       5.2      -1.2              8
Real Betis     8       4.3       3.7              2
Real Sociedad  3       3.9      -0.9              5
```

Villarreal	5	3.9	1.1	7
Deportivo La Coruna	3	3.9	-0.9	11
Celta Vigo	2	3.8	-1.8	16
Getafe	2	3.4	-1.4	18
Levante	1	3.4	-2.4	12
Real Zaragoza	3	2.9	0.1	19
Eibar	1	2.7	-1.7	13
Rayo Vallecano	1	2.6	-1.6	14
Granada	3	2.4	0.6	15
Osasuna	3	2.3	0.7	17

```
[ ]: plt1 = pitchplot()
plt1 = sns.scatterplot(shot_test_set_results['x location_␣
↪shot'],shot_test_set_results['y location_␣
↪shot'],c=shot_test_set_results['predicted xg'],cmap='GnBu')
plt.axis('off')
plt.title('Overall xG shot map',size=40)
plt.tight_layout()
plt.savefig('overallxGmap.png')
```

Pass the following variables as keyword args: x, y. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.

Overall xG shot map



```
[ ]: from matplotlib.patches import Arc
```

```

[ ]: i=1
fig=plt.figure(figsize=(25,10))
with plt.style.context('bmh'):
    for team in set(shot_test_set_results['Team Name'].values):
        if team=='Barcelona' or team=='Atlético Madrid' or team=='Real Madrid' or
        team=='Sevilla':
            filt1 = shot_test_set_results['Team Name'] == team
            df_team = shot_test_set_results[filt1]

            ax = fig.add_subplot(2,6,i)

            ax.axis([60, 125, -10, 90])
            ax.grid(False)

            ax.plot([60, 60], [0, 80], color='black')
            ax.plot([120, 120], [0, 80], color="black")
            ax.plot([60, 120], [0, 0], color='black')
            ax.plot([60, 120], [80, 80], color='black')

            centreCircle = plt.Circle((60, 40), 10, color='black', fill=False)

            # right penalty area
            ax.plot([120, 102], [18, 18], color='black')
            ax.plot([102, 102], [18, 62], color='black')
            ax.plot([102, 120], [62, 62], color='black')

            # right six yard box
            ax.plot([120, 114], [30, 30], color='black')
            ax.plot([114, 114], [30, 50], color='black')
            ax.plot([114, 120], [50, 50], color='black')

            # right goal posts
            ax.plot([120, 122], [36, 36], color='black')
            ax.plot([120, 122], [44, 44], color='black')
            ax.plot([122, 122], [36, 44], color='black')

            # right Arc
            rightArc = Arc((108, 40), height=18.3, width=18.3, angle=0,
                           theta1=130, theta2=230, color='black')

            ax = plt.gca()
            ax.add_patch(centreCircle)
            ax.add_patch(rightArc)
            ax.set_ylim(ax.get_ylim()[::-1])

```

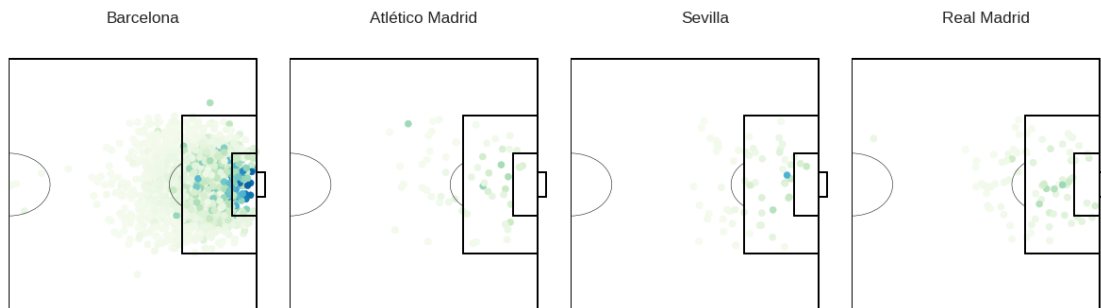
```

        ax.scatter(df_team['x location shot'],df_team['y location_
↪shot'],c=df_team['predicted xg'],cmap='GnBu',vmin=0,vmax=1)
        ax.axis('off')

        ax.set_title(team)
        i+=1

plt.tight_layout()
plt.savefig('xg_allteams.png')

```



```

[ ]: ascending=False
season_ids = shot_test_set_results.sort_values("xgboost xg",
↪ascending=ascending).reset_index().iloc[0:100]["season_id"]
game_ids = shot_test_set_results.sort_values("xgboost xg", ascending=ascending).
↪reset_index().iloc[0:100]["game_id"]
shot_ids = shot_test_set_results.sort_values("xgboost xg", ascending=ascending).
↪reset_index().iloc[0:100]["shot id"]
xg_vals_xgb = shot_test_set_results.sort_values("xgboost xg",
↪ascending=ascending).reset_index().iloc[0:100]["xgboost xg"]
xg_vals_cat = shot_test_set_results.sort_values("xgboost xg",
↪ascending=ascending).reset_index().iloc[0:100]["catboost xg"]
xg_vals_lgb = shot_test_set_results.sort_values("xgboost xg",
↪ascending=ascending).reset_index().iloc[0:100]["lightgbm xg"]
xg_vals_lr = shot_test_set_results.sort_values("xgboost xg",
↪ascending=ascending).reset_index().iloc[0:100]["logreg xg"]
xg_vals_sb = shot_test_set_results.sort_values("xgboost xg",
↪ascending=ascending).reset_index().iloc[0:100]["official xg"]

freeze_frame1(laliga[season_ids[70]][game_ids[70]].json_file,
↪shot_ids[70],ax=ax)

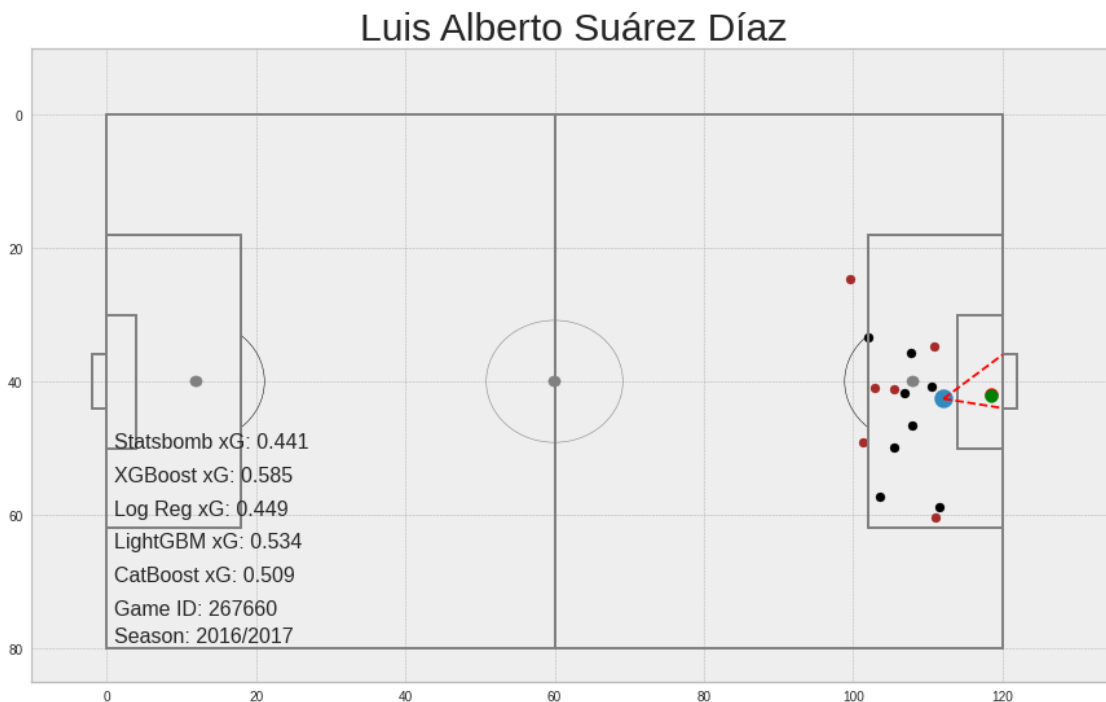
```

```

plt.annotate("Season: {}".format(season_ids[70]), xy = (1,79), size=16)
plt.annotate("Game ID: {}".format(game_ids[70]), xy = (1,75), size=16)
plt.annotate("XGBoost xG: {:.3f}".format(round(xg_vals_xgb[70], 3)), xy = (1,55), size=16)
plt.annotate("CatBoost xG: {:.3f}".format(round(xg_vals_cat[70], 3)), xy = (1,70), size=16)
plt.annotate("LightGBM xG: {:.3f}".format(round(xg_vals_lgb[70], 3)), xy = (1,65), size=16)
plt.annotate("Log Reg xG: {:.3f}".format(round(xg_vals_lr[70], 3)), xy = (1,60), size=16)
plt.annotate("Statsbomb xG: {:.3f}".format(round(xg_vals_sb[70], 3)), xy = (1,50), size=16)

```

```
[ ]: Text(1, 50, 'Statsbomb xG: 0.441')
```



```

[ ]: ascending=False
messi_xg = shot_test_set_results[shot_test_set_results['player name']=='Lionel_
    Andrés Messi Cuccittini']
season_ids = messi_xg.sort_values("season_id", ascending=ascending).
    reset_index().iloc[0:100]["season_id"]
game_ids = messi_xg.sort_values("season_id", ascending=ascending).reset_index().
    iloc[0:100]["game_id"]
shot_ids = messi_xg.sort_values("season_id", ascending=ascending).reset_index().
    iloc[0:100]["shot id"]

```

```

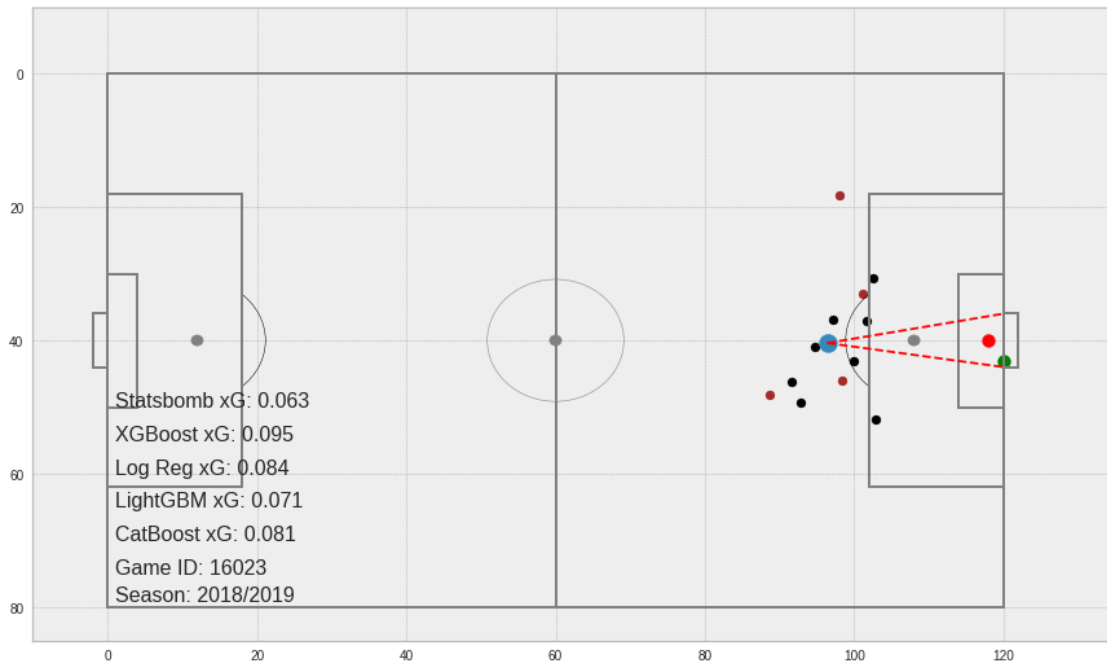
xg_vals_xgb = messi_xg.sort_values("season_id", ascending=ascending).
↳reset_index().iloc[0:100]["xgboost xg"]
xg_vals_cat = messi_xg.sort_values("season_id", ascending=ascending).
↳reset_index().iloc[0:100]["catboost xg"]
xg_vals_lgb = messi_xg.sort_values("season_id", ascending=ascending).
↳reset_index().iloc[0:100]["lightgbm xg"]
xg_vals_sb = messi_xg.sort_values("season_id", ascending=ascending).
↳reset_index().iloc[0:100]["official xg"]
xg_vals_lr = messi_xg.sort_values("season_id", ascending=ascending).
↳reset_index().iloc[0:100]["logreg xg"]
#messi_xg = messi_xg.sort_values("season_id", ascending=ascending).
↳reset_index().iloc[0:100]["player name"]=="Lionel Andrés Messi Cuccittini"
freeze_frame1(laliga[season_ids[50]][game_ids[50]].json_file,
↳shot_ids[50],ax=ax)

plt.annotate("Season: {}".format(season_ids[50]), xy = (1,79), size=16)
plt.annotate("Game ID: {}".format(game_ids[50]), xy = (1,75), size=16)
plt.annotate("XGBoost xG: {:.3f}".format(round(xg_vals_xgb[50], 3)), xy = (1,
↳55), size=16)
plt.annotate("CatBoost xG: {:.3f}".format(round(xg_vals_cat[50], 3)), xy = (1,
↳70), size=16)
plt.annotate("LightGBM xG: {:.3f}".format(round(xg_vals_lgb[50], 3)), xy = (1,
↳65), size=16)
plt.annotate("Log Reg xG: {:.3f}".format(round(xg_vals_lr[50], 3)), xy = (1,
↳60), size=16)
plt.annotate("Statsbomb xG: {:.3f}".format(round(xg_vals_sb[50], 3)), xy = (1,
↳50), size=16)

```

[]: Text(1, 50, 'Statsbomb xG: 0.063')

Lionel Andrés Messi Cuccittini



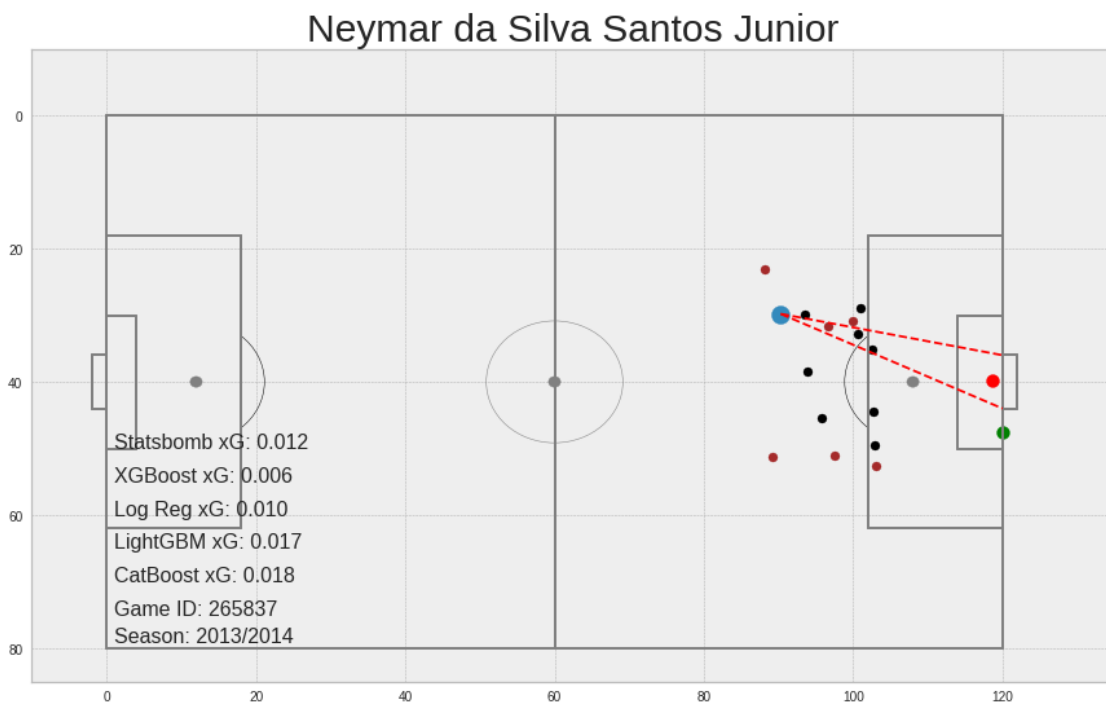
```
[ ]: ascending=True
season_ids = shot_test_set_results.sort_values("xgboost xg",
    ↪ascending=ascending).reset_index().iloc[0:100]["season_id"]
game_ids = shot_test_set_results.sort_values("xgboost xg", ascending=ascending).
    ↪reset_index().iloc[0:100]["game_id"]
shot_ids = shot_test_set_results.sort_values("xgboost xg", ascending=ascending).
    ↪reset_index().iloc[0:100]["shot_id"]
xg_vals_xgb = shot_test_set_results.sort_values("xgboost xg",
    ↪ascending=ascending).reset_index().iloc[0:100]["xgboost xg"]
xg_vals_cat = shot_test_set_results.sort_values("xgboost xg",
    ↪ascending=ascending).reset_index().iloc[0:100]["catboost xg"]
xg_vals_lgb = shot_test_set_results.sort_values("xgboost xg",
    ↪ascending=ascending).reset_index().iloc[0:100]["lightgbm xg"]
xg_vals_lr = shot_test_set_results.sort_values("xgboost xg",
    ↪ascending=ascending).reset_index().iloc[0:100]["logreg xg"]
xg_vals_sb = shot_test_set_results.sort_values("xgboost xg",
    ↪ascending=ascending).reset_index().iloc[0:100]["official xg"]

freeze_frame1(laliga[season_ids[55]][game_ids[55]].json_file,
    ↪shot_ids[55],ax=ax)

plt.annotate("Season: {}".format(season_ids[55]), xy = (1,79), size=16)
plt.annotate("Game ID: {}".format(game_ids[55]), xy = (1,75), size=16)
```

```
plt.annotate("XGBoost xG: {:.3f}".format(round(xg_vals_xgb[55], 3)), xy = (1, 55), size=16)
plt.annotate("CatBoost xG: {:.3f}".format(round(xg_vals_cat[55], 3)), xy = (1, 70), size=16)
plt.annotate("LightGBM xG: {:.3f}".format(round(xg_vals_lgb[55], 3)), xy = (1, 65), size=16)
plt.annotate("Log Reg xG: {:.3f}".format(round(xg_vals_lr[55], 3)), xy = (1, 60), size=16)
plt.annotate("Statsbomb xG: {:.3f}".format(round(xg_vals_sb[55], 3)), xy = (1, 50), size=16)
```

```
[ ]: Text(1, 50, 'Statsbomb xG: 0.012')
```

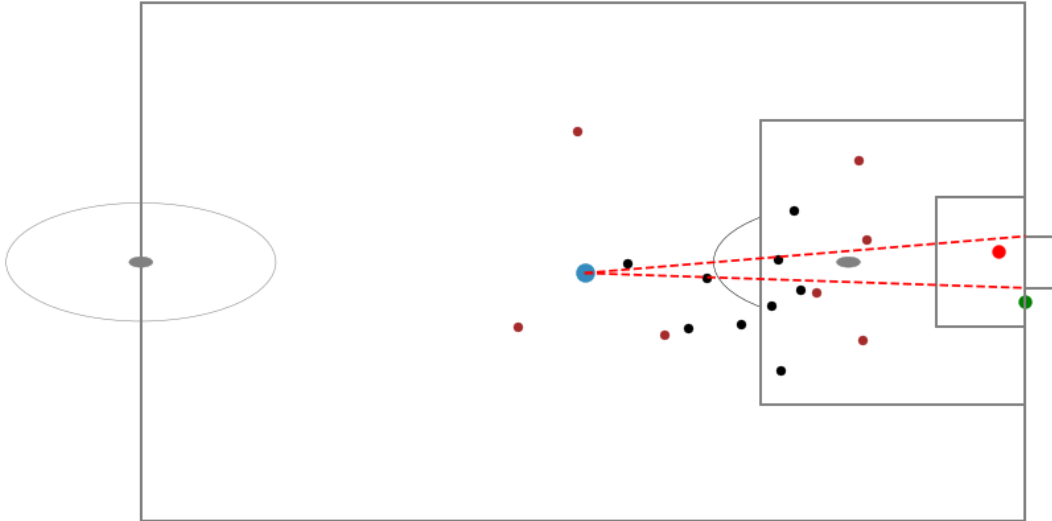


```
[ ]: ascending=True
season_ids = shot_test_set_results.sort_values("xgboost xg",
ascending=ascending).reset_index().iloc[0:100]["season_id"]
game_ids = shot_test_set_results.sort_values("xgboost xg", ascending=ascending).
reset_index().iloc[0:100]["game_id"]
shot_ids = shot_test_set_results.sort_values("xgboost xg", ascending=ascending).
reset_index().iloc[0:100]["shot_id"]

freeze_frame2(laliga[season_ids[44]][game_ids[44]].json_file,
shot_ids[44], ax=ax)
plt.axis('off')
```

```
[ ]: (50.0, 125.0, 90.0, -10.0)
```

Andrés Iniesta Luján



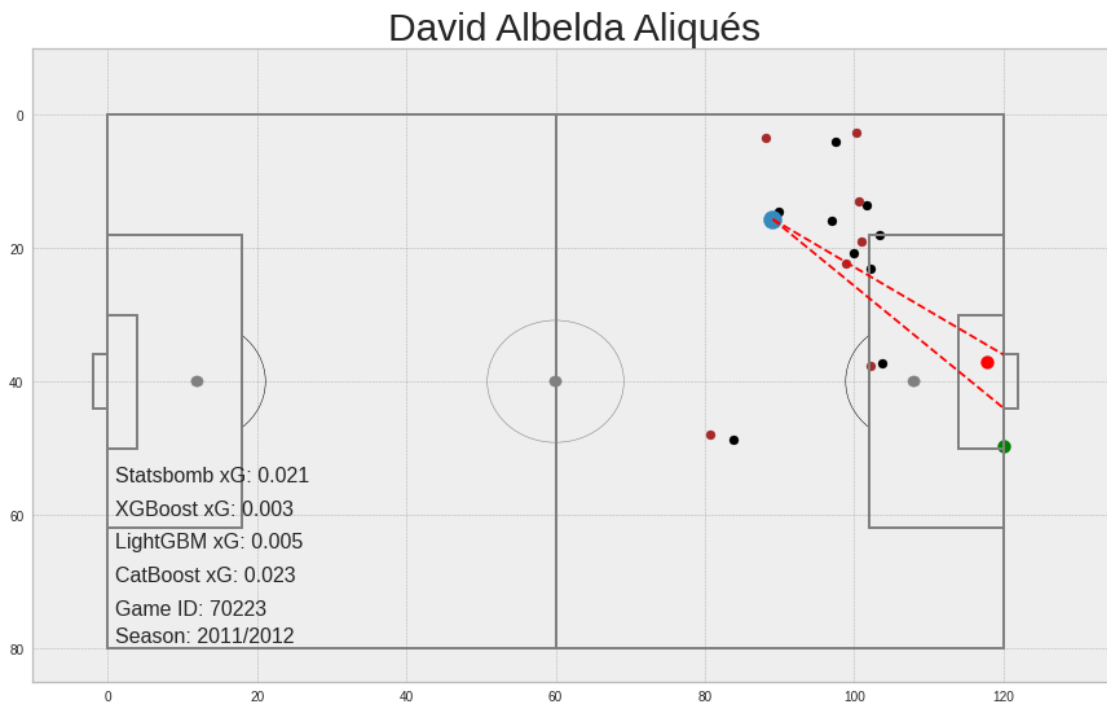
```
[ ]: ascending=False
season_ids = shot_test_set_results.sort_values("shot distance",
    ↳ascending=ascending).reset_index().iloc[0:100]["season_id"]
game_ids = shot_test_set_results.sort_values("shot distance",
    ↳ascending=ascending).reset_index().iloc[0:100]["game_id"]
shot_ids = shot_test_set_results.sort_values("shot distance",
    ↳ascending=ascending).reset_index().iloc[0:100]["shot_id"]
xg_vals_xgb = shot_test_set_results.sort_values("shot distance",
    ↳ascending=ascending).reset_index().iloc[0:100]["xgboost xg"]
xg_vals_cat = shot_test_set_results.sort_values("shot distance",
    ↳ascending=ascending).reset_index().iloc[0:100]["catboost xg"]
xg_vals_lgb = shot_test_set_results.sort_values("shot distance",
    ↳ascending=ascending).reset_index().iloc[0:100]["lightgbm xg"]
xg_vals_sb = shot_test_set_results.sort_values("shot distance",
    ↳ascending=ascending).reset_index().iloc[0:100]["official xg"]
outcomeval = shot_test_set_results.sort_values("shot distance",
    ↳ascending=ascending).reset_index().iloc[0:100]["outcome"]

freeze_frame1(laliga[season_ids[25]][game_ids[25]].json_file,
    ↳shot_ids[25],ax=ax)

plt.annotate("Season: {}".format(season_ids[25]), xy = (1,79), size=16)
```

```
plt.annotate("Game ID: {}".format(game_ids[25]), xy = (1,75), size=16)
plt.annotate("XGBoost xG: {:.3f}".format(round(xg_vals_xgb[25], 3)), xy = (1,60), size=16)
plt.annotate("CatBoost xG: {:.3f}".format(round(xg_vals_cat[25], 3)), xy = (1,70), size=16)
plt.annotate("LightGBM xG: {:.3f}".format(round(xg_vals_lgb[25], 3)), xy = (1,65), size=16)
plt.annotate("Statsbomb xG: {:.3f}".format(round(xg_vals_sb[25], 3)), xy = (1,55), size=16)
```

```
[ ]: Text(1, 55, 'Statsbomb xG: 0.021')
```



```
[ ]: def xGgoals(fig,df):
    halfpitchplot()
    sns.scatterplot(data=df, x="x location shot", y="y location shot",
        hue="Goal_label", size = 'xgboost xg', sizes=(20, 200), palette='GnBu')
    #px.density_heatmap(df_regular_play, x="x location shot", y="y location
    shot")
    playername = df["player name"][1]
    plt.
    title(playername, fontsize=40, pad=None, backgroundcolor='black', color='white')
    xG = sum(df['xgboost xg'])
    plt.annotate("xG: {:.3f}".format(round(xG, 3)), xy = (62, 65), size=16)
```

```

plt.annotate("Goals: {:.1f}".format(sum(df['binary_outcome'])), xy = (62, 70), size=16)
plt.annotate("xG per shot: {:.3f}".format(round((xG/100),3)), xy = (62, 75), size=16)
plt.legend(loc='upper right')
plt.axis('off')
return plt

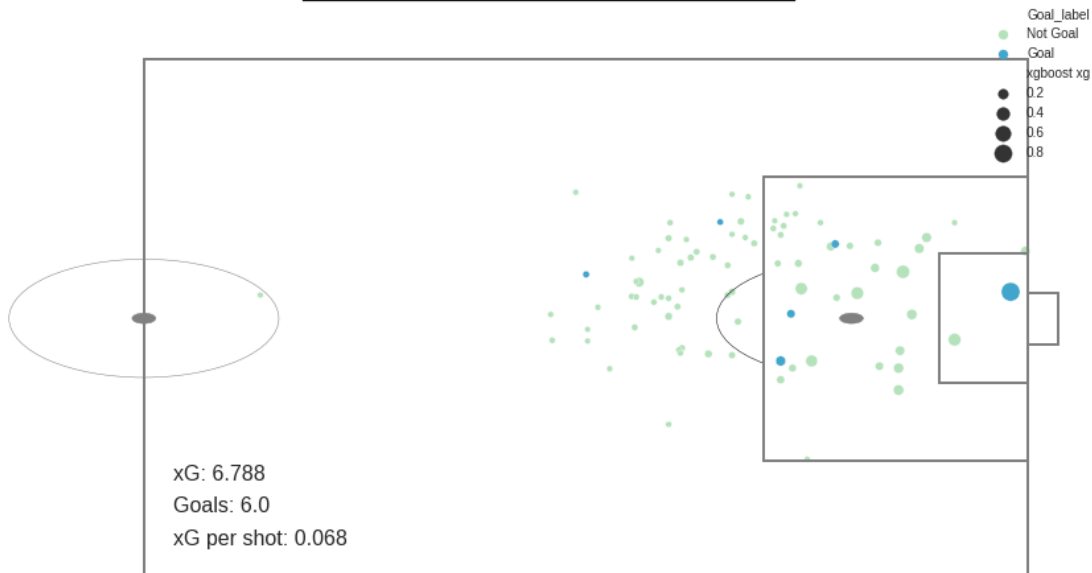
```

```

[ ]: iniesta_xg = shot_test_set_results[shot_test_set_results['player_name'] == 'Andrés Iniesta Luján'].iloc[0:100]
#pitch = goals(plt, messi_xg[['x location shot', 'y location shot']], values='grey', 'shots')
pitch = xGgoals(plt, iniesta_xg)
plt.show()

```

Andrés Iniesta Luján

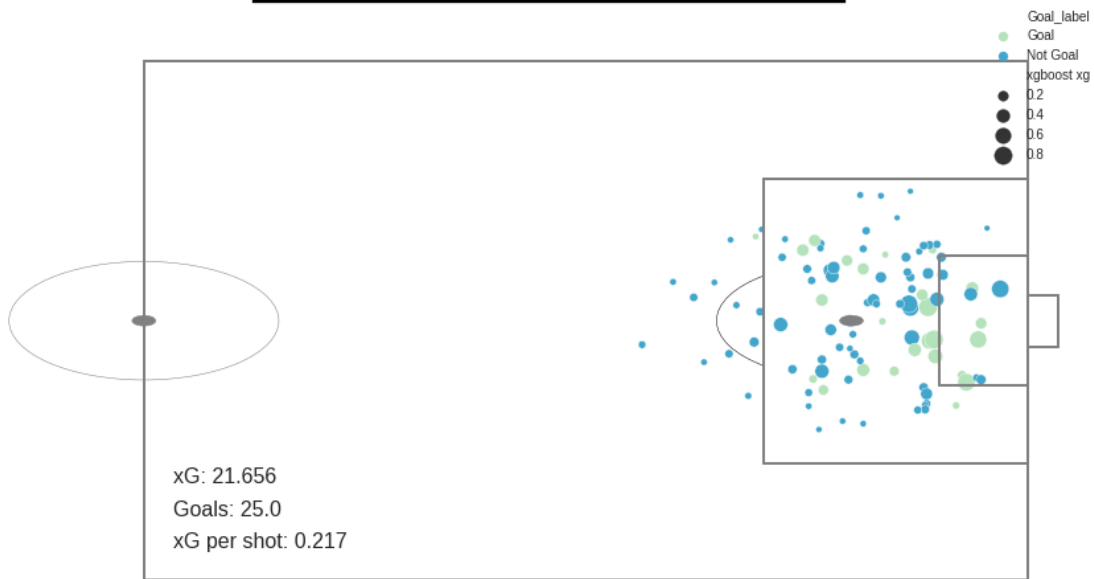


```

[ ]: suarez_xg = shot_test_set_results[shot_test_set_results['player name'] == 'Luis Alberto Suárez Díaz'].iloc[0:100]
#pitch = goals(plt, messi_xg[['x location shot', 'y location shot']], values='grey', 'shots')
pitch = xGgoals(plt, suarez_xg)
plt.legend(loc='upper right')
plt.show()

```

Luis Alberto Suárez Díaz



```
[ ]: map = shot_test_set_results.corr()[["official xg", "lightgbm xg", "xgboost_
    ↪xg", "catboost xg"]].loc[["official xg", "lightgbm xg", "xgboost_
    ↪xg", "catboost xg"]]
```

```
[ ]: sns.heatmap(map, cmap="YlGnBu", annot=True)
plt.title('Binary Probability')
```

```
[ ]: Text(0.5, 1.0, 'Binary Probability')
```

